

Non-Profit JSC “L.N.Gumilyov Eurasian National University”
Institute of Digital Sciences and Artificial Intelligence
Department of “Computer and Software Engineering”

EDUCATIONAL-METHODICAL COMPLEX OF DISCIPLINE

CSSE 33106 Software Development
(code and name of the module)

on discipline TPD 3301 Tools of program development
(code and name of the discipline)

6B06104 – “Computer Engineering and Software”
(code and name of the educational program)

**Non-Profit Joint-Stock Company «L.N. Gumilyov Eurasian National University»
«Institute of Digital Sciences and Artificial Intelligence»
Department Computer and software engineering education department**

Syllabus

CSSE 33106 Programming Languages for Software Development and device

on the discipline
TPD 3301 Tools of program development for students of education program
6B06104 - Computer Engineering and Software

Астана
2026



This document, in accordance with paragraph 1 of Article 7 of the Law of the Republic of Kazakhstan dated January 7, 2003 «On electronic document and electronic digital signature» is equivalent to a document on paper

Document signed

№	Full name	Position
1	Senkovskaya Anastasiya	senior lecturer
2	Dyussekeyev Kanagat	Head of the кафедра
3	Davletova Ainash	Associate Professor (Associate Professor)
4	Orazaliyev Bakhytzhan	Director
5	Seilov Shakhmaran	Director of the Institute

Developer:

Сеньковская Анастасия Александровна, senior lecturer

Working curriculum (Syllabus) for the discipline TPD 3301 Tools of program development is developed on the basis of the educational program 6B06104 - "Computer Engineering and Software"

Considered at the meeting of the department of Computer and Software Engineering protocol # 10 from "04" May 2026 y.

Approved at the meeting of the Institute's Teaching and Methodical Commission Protocol # 9 from "14" May 2026 y.

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

EXPLANATORY NOTE

1. Brief description of the discipline

The course focuses on mastering methods of software system design and ensuring the life cycle of programs. It offers an introduction to the basics of business process modeling and the acquisition of practical skills in the application of modern design technologies.

Purpose of the discipline	Result of training (RT) of the educational program *	Expected Training Outcomes (ET) for the discipline
The objectives of the course are to familiarize students with issues related to software system design and ensuring the life cycle of; teaching them the basics of business process modeling and acquiring practical skills in the application of modern design technologies.	RT₈ – Be able to make an optimal choice of data structures, be able to develop algorithms, and implement them in programming languages, debug and analyze software source code and prepare reporting documentation PC₅ - The ability to form knowledge and skills in the development, modification, control of software, design of structure and content and / or writing computer code necessary for the creation and operation of: system software, application software, databases, mobile applications and services.	– Knowledge of specifications, visualizations, design, and documentation of software systems developed based on object-oriented technologies and a component-based approach; – Possess skills in functional decomposition, in which the structure of the system is described in terms of the hierarchy of its functions and the transfer of information between individual functional elements; – Ability to apply methods for modeling software systems of any complexity based on graphical diagrams of the UML (Unified Modeling Language), Aris, and MS Visio languages. – Knowledge of the basics of modeling software systems of any complexity based on graphical diagrams of the structural type languages BPWin, ERWin, and ModelMart.

* According to EP 6B06104 - "Computer Engineering and Software" 2024 y

2. Prerequisites

To master this discipline, you need the knowledge, skills, and abilities acquired while studying the following disciplines: C++ programming, Information and communication technologies.

Post-requisites

The knowledge, skills, and abilities acquired while studying this discipline are necessary for mastering the following disciplines: «IT project management», «Big Data», «Mixed, augmented, and virtual reality».

3. Extract from the curriculum

Course	3
Semester	5
Number of credits ECTS	5

Types of classes	Total number of hours
Lectures	15
Practical classes	
Seminar classes	
Laboratory classes	30
Independent work of the student (IWS)	105
Total	150

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

4. Thematic plan of the discipline by modules (in academic hours)

Module No.	Module name
1	Design methods and program life cycle. Classification of tools
2	Functional design and methods for developing database structures. Unified modeling language

Lectures				
Week No.	Module No.	Title of lecture topic	Number of hours	Types and methods of teaching
1	1	Introduction. Purpose and objectives of tools. Classification of personal computer software. Classification and main features of modern tools.	1	Explanatory-illustrative method
2	1	Fundamentals of business process modeling. Software life cycle. Life cycle models.	1	Partial-search method
3	1	General requirements for methodology and technology. System design stages. Requirements for document content.	1	Problem-based presentation method
4	1	CASE technology for designing CASE systems of structural and object-oriented types. Practical recommendations for mastering and implementing CASE tools, criteria for their selection, and comparative analysis.	1	Explanatory-illustrative method
5	1		1	Partial-search method
6	1	Structural approach to software design. SADT functional modeling methodology. Functional model of work organization - AS IS. TO BE model and its functional cost analysis.	1	Problem-based presentation method
7	1	IDEFO functional modeling methodology. CASE – BPWin structural tool.	1	Explanatory-illustrative method
8	2	Displaying the logical and physical levels of the model in ERWin. CASE – a structural tool of the ERWin type.	1	Partial-search method
9	2	Unified Modeling Language (UML). Key elements of the object-oriented approach.	1	Problem-based presentation method
10	2	UML diagrams. Description of requirements in the context of the precedent model.	1	Explanatory-illustrative method
11	2	Classes. Class diagram. Modeling interactions between objects.	1	Partial-search method
12	2	Modeling collaborations. Collaboration diagrams, state diagrams.	1	Problem-based presentation method
13	2	UML diagrams used by different roles at different stages of software development	1	Explanatory-illustrative method
14	2	Application of UML diagrams of each type for modeling system behavior.	1	Partial-search method
15	2	Designing systems in UML in Rational Rose, Aris, and MS Office Visio environments.	1	Problem-based presentation method
TOTAL			15	

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

Laboratory classes				
Week no.	Module no.	Name of topics of laboratory classes	Number of hours	Types and methods of training
1	1	BPwin 4.0 tools. Creating a functional model using BPWin 4.0.	2	Heuristic method
2	1		2	Research method
3	1	Using Erwin to develop a data model. Logical model. Entities and relationships.	2	Group work
4	1	Features of the IBM Rational Rose program user interface.	2	Heuristic method
5	1	Building a Use Case Diagram (Diagrams of Use Cases).	2	Research method
6	1	Formulation of technical specifications. Writing system requirements.	2	Group work
7	1	Activity diagram - Construction of activity diagrams.	2	Heuristic method
8	2	Modeling interactions between objects. Interaction diagrams: Sequence diagrams. Collaboration diagrams.	2	Research method
9	2		2	Group work
10	2	State diagrams.	2	Heuristic method
11	2	Classes. Class diagram	2	Research method
12	2		2	Group work
13	2	Developing apps using no-code platforms. AppMaster platform.	2	Heuristic method
14	2	Developing an application with database connectivity on the AppMaster platform	2	Research method
15	2		2	Group work
TOTAL			30	

IWS				
Week no.	Module no.	Name of topics of IWS. Deadlines for submitting IWS	Number of hours	Types and methods of training
1	1	Preliminary survey and analysis of the subject area. Justification and selection of automated tasks (task setting). Preparation of an individual task with a verbal description of the algorithm. Deadline: Week 2	8	summary
2	1		8	presentation
3	1	Creating a business process model. Creating a business analysis model. Deadline: Week 5	7	report
4	1		7	abstract
5	1		7	summary
6	1	Functional modeling and construction of a model of the selected subject area using Platinum BPWin. Individual. Deadline: Week 7	7	presentation
7	1		7	report
8	2	Functional modeling and database structure design for a selected subject area using Platinum ERWin. Individual. Individual assignment. Deadline: Week 10	7	abstract
9	2		7	summary
10	2		7	presentation

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

11	2	Designing a model of the selected subject area in the Rational Rose.	7	report
12	2	Deadline: Week 12	7	summary
13	2	Designing a model of the selected subject area in the Aris.	7	presentation
		Deadline: Week 13		
14	2	Designing a model of a selected subject area on the AppMaster platform.	7	report
15	2	Deadline: Week 15	7	abstract
TOTAL			105	

5. Brief organizational and methodological characteristics of the discipline

Types of control of learning achievements:

Routine control 1 colloquium

Routine control 2 colloquium

Final control: written exam

Course Policies and Procedures:

- Mandatory attendance of students to all classes according to the schedule;
- Preliminary preparation for classes;
- Timely fulfillment and submission of SLOs;
- Preparation for all types of classes should be of an independent, creative nature;
- Active work and creativity during classes;
- Participation in all types of supervision;
- Commitment to the University's Academic Integrity Policy.

For students with special needs, Individual work with students by phone, additional group classes with a specific list of questions, consultation hours at the department, lecture presentations are provided, and additional time is allocated for rating knowledge control and interim assessment. Ms Teams has opened a channel for online consultations and downloading teaching materials and completed laboratory work.

6. Educational and methodological support of the discipline

№	Author, title, publisher, year of publication	Information carrier.	Available (units.)		
			In the library	World Digital Libraries	In the department
1	2	3	4		5
Main literature					
1	Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. – 366с. – ISBN 978-5-4461-1213-5.	Monograph	1		
2	Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL:	Textbook	-	Лань: электронно-библиотечная система. – URL: https://e.lanbook.com/book/208946	In electronic form

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

	https://e.lanbook.com/book/208946 (дата обращения: 30.05.2025)				
3	Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. - М.: Академия, 2015. - 351с. - ISBN 978-5-4468-2440-3.	Textbook	29		
4	Брусникин Г.Н., Соколова Н.Ю. Разработка UML-моделей при проектировании информационных систем: учебное пособие. - М.: МИЭТ, 2023. - 52 с. - ISBN 978-5-7256-1016-1.	Textbook	-	Лань: электронно-библиотечная система. — URL: https://e.lanbook.com/book/461570	In electronic form
Additional literature					
5	Грекул В.И., Денищенко Г.Н., Коровникова Н.Л. Проектирование информационных систем. - М.: Национальный открытый Университет «ИНТУИТ», 2016. - 571с.	Monograph		https://library.enu.kz/ProtectedView/Book/ViewBook/83641	
6	Шевчук Е.В. Интеллектуальные информационные системы: учебно-методическое пособие. - Алматы: CyberSmith, 2017. - 137с. - ISBN 978-601-310-319-8.	Textbook	25		
7	Цуканова О.А. Методология и инструментарий моделирования бизнес-процессов: учебное пособие. - Министерство образования и науки Российской Федерации, Университет ИТМО. - Санкт-Петербург: Университет ИТМО, 2015. - 101 с.	Textbook		https://library.enu.kz/ProtectedView/Book/ViewBook/82803	
Electronic and Internet resources					
1	Унифицированный язык моделирования UML https://www.lucidchart.com/pages/ru/uml				
2	Полный курс UML https://blog.skillfactory.ru/glossary/uml/				
3	https://www.youtube.com/user/ariscommunity				
4	https://www.ariscommunity.com				

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

7. The system of assessment of the results of students' educational achievements

Students' knowledge, skills and abilities are assessed according to the following system

Letter grade	Numerical equivalent of points	% content	Evaluation according to the traditional system	Scoring criteria
A	4,0	95-100	Excellent	An A grade in the subject “Tools of program development” is awarded when the student: understands the software life cycle and can clearly distinguish between waterfall, iterative, spiral, and iterative models; confidently builds AS IS / TO BE models, performs functional and cost analysis; freely uses CASE tools (BPWin, ERWin, Rational Rose, Ms Office Visio, Aris) for modeling structural and object-oriented systems; creates correct UML diagrams (use case, class, sequence, activity, etc.), taking into account the requirements and interaction of components; demonstrates excellent command of IDEF0 and SADT: builds complete functional models; competently prepares documentation in accordance with GOST/IEEE requirements.
A-	3,67	90-94		An A- grade is given when the student: understands the software life cycle and can clearly distinguish between waterfall, iterative, spiral, and iterative models; confidently builds AS IS / TO BE models, performs functional and cost analysis; freely uses CASE tools (BPWin, ERWin, Rational Rose, Ms Office Visio, Aris) for modeling structural and object-oriented systems, but allows minor errors; models (UML, IDEF0, ER models) are logically correct, but may not take into account rare scenarios; demonstrates excellent command of IDEF0 and SADT: builds complete functional models.
B+	3,33	85-89	Good	A B+ grade is given when the student knows the key stages of design, applies SADT and UML methodologies at a good level; minor details or interrelationships may be omitted in UML and IDEF0 models; able to work in CASE environments, but with prompts; documentation is complete, but not always structured.
B	3,0	80-84		A grade of B is given when the student: understands the principles of the life cycle and modeling, but sometimes confuses stages or designations; UML diagrams are drawn correctly, but superficially; has a basic command of CASE tools, prefers templates.
B-	2,67	75-79		A B- grade is given when the student: partially understands the difference between structural and object-oriented approaches; has difficulty constructing coherent models; diagrams are incomplete; uses only basic CASE tool functions; project tasks are completed formally, models require further refinement.
C+	2,33	70-74		A C+ grade is given when a student: has limited knowledge of basic concepts (life cycle, UML diagrams, SADT); uses CASE tools uncertainly, often fails to follow notation; the project contains errors in the logic or structure of models; documentation is incomplete or formal.
C	2,0	65-69	Fair	A C grade is given when the student: demonstrates a general understanding of the discipline, but cannot apply it in practice; creates diagrams with errors, does not distinguish between types of UML diagrams; uses CASE

	«L.N.Gumilyov Eurasian National University»	Working curriculum (Syllabus)	Edition: third
--	---	-------------------------------	----------------

				tools as a drawing tool, without understanding the logic of models.
C-	1,67	60-64		A C- grade is given when the student: knows the terms but does not know how to apply them: IDEF0, SADT, UML, CASE; Errors even in basic diagrams (e.g., in use case or class diagrams); Does not know how to use CASE tools; Lab work completed with outside help; Documentation poorly formatted, no structure.
D+	1,33	55-59		A D+ grade is given when the student: demonstrates weak knowledge: can recall topics but cannot apply them in practice; diagrams and models do not correspond to methodology, logic is lacking; CASE tools are rarely used.
D	1,0	50-54		A D grade is given when the student: understands only individual concepts, but does not master the basics of modeling; does not use CASE tools at all or works with errors; Laboratory assignments are not completed or defended; the project does not meet the requirements.
FX	0,5	25-49		The grade “poor” corresponds to the letters FX and F , which have a numerical equivalent of 0 and a percentage content of 0-49. This grade is given if the student has gaps in their knowledge of the basic material covered by the program, has not mastered more than half of the module (discipline) program, has made fundamental errors in their answers, has not completed individual assignments provided for in the forms of current, intermediate, and final control, or has not studied all the basic literature provided for in the program.
F	0	0-24	Poor	

Theses lectures By topics academic discipline

Lecture 1. Introduction Classification of personal computer software. Classification and main features of modern tools.

The set of programs required for operating a personal computer is called personal computer software. There are various categories programs that differ significantly in their purpose, of which the following can be distinguished :

1. Systemic programs - intended For exploitation And technical maintenance COMPUTER, management and organization of the computing process when solving any specific problem on a computer, etc. These include:

1. Operating systems
2. Shells operating systems
3. Drivers devices
4. Utility programs
5. Antivirus programs
6. Serving programs

2. Programming systems are complexes of programs and other tools, each intended for the development and exploitation of programs in a specific programming language for a specific type of computer. A programming system usually includes some version of a programming language, a program translator presented in this language, etc. For example, Qbasic, FoxPro, etc.

3. Tool programs - programs used How tool for creating documents in everyday activities. These include:

1. Programs For works With texts (Text editors)
2. Graphic editors
3. Electronic tables (Tabular processors)
4. Systems management bases data (DBMS)
5. Integrated environments
6. Educational And educational programs.
7. Applied programs.
8. Games.
9. Multimedia.

Instrumental means developments programs

There are currently quite a few different visual application development tools available on the software market. They can be divided into two classes:

- ❑ specialized tools – oriented exclusively towards working with a very specific DBMS and not intended for developing regular applications that do not use databases.
- ❑ universal tools that can be used for developing information applications, interacting With bases data, So And For developments any others applications, Not using databases. Of these, The most well-known systems are Borland Delphi from Borland and Visual Basic from Microsoft.

CASE tools developments programs

For the last decade V areas technical funds programming formed new direction - *CASE technology* (Computer-Aided Software/ System Engineering). *CASE technology* represents by yourself a set of methodologies for analyzing, designing, developing, and maintaining complex systems and is supported by a set of interconnected automation tools. When using structural analysis methodologies, a number of limitations emerged (complexity of understanding, high labor intensity and cost of use, inconvenience of making changes to design specifications, etc.). CASE technologies have been developed from the very beginning precisely with the goal of overcoming these limitations by automating analysis processes and integrating supporting tools.

Main possibilities CASE tools

Modern CASE tools cover a wide range of support for numerous information system design technologies: from simple analysis and documentation tools to full-scale automation tools covering the entire software life cycle.

The most labor-intensive stages of developing an information system are the stages of analysis and design, during which CASE tools ensure the quality of the technical decisions made and the preparation project documentation. At this big role are playing methods visual performances information. This involves constructing structural or other diagrams in real time, using a diverse color palette, and end-to-end checking of syntax rules. Graphic tools for modeling a subject area allow developers to visually study an existing information system, rebuild it in accordance with the goals set and existing limitations.

IN most full view CASE tools possess next characteristic features:

- *single graphic language*. CASE technologies provide all participants project, including customers, a single strict, clear and intuitive graphical language that allows for the production of clear components With simple And clear structure. At this programs introduce themselves two-dimensional schemes (which easier

in use than multi-page descriptions), allowing the customer to participate in the development process, and developers to communicate with subject matter experts, separating the activities of system analysts, designers and programmers, making it easier for them to defend the project to management, and also ensuring ease of maintenance and making changes to the system;

- *single project database.* The basis of CASE technology is the use of a project database (repository) to store all information about the project, which can be shared by developers in accordance With their rights access. Content repositories includes Not only informational objects different types, but also the relationships between their components, as well as the rules for using or processing these components. Repository Maybe keep objects various types: structural diagrams, definitions screens And menu, report designs, data descriptions, logic processing, data models, their organization and processing, source codes, data elements, etc.;

- *integration of tools.* The repository is used to integrate CASE tools and separate the system information between developers. At the same time, the possibilities repositories provide several levels of integration: a common user interface for all tools, data transfer between tools, integration of development stages through a single system for representing life cycle phases, data and tool transfer between different platforms;

- *support collective development and project management.* CASE technology supports group work on a project, providing the ability to work online, export and import any fragments of the project for their development and/or modification, as well as planning, control, management and interaction, i.e. functions necessary in the process of developing and maintaining projects. These functions are also implemented on the basis of the repository. IN in particular, through repository can be carried out control security (restrictions And access privileges), control versions and changes and etc.; About *layout.* CASE technology makes it possible quickly build mock-ups (prototypes) of the future system, which allows the customer to evaluate at the early stages of development how satisfied he is with it and how acceptable it is for future users;

- *generation documentation.* All documentation By project is generated automatically based on repository (usually in accordance with the requirements of current standards). The undoubted advantage of CASE technology is V volume, What documentation Always answers current state affairs, because any changes V the project is automatically reflected in the repository (it is known that with traditional approaches to software development, documentation is at best lagging behind, and a number of modifications are not reflected in it at all);

- *verification project.* CASE technology provides automatic verification and control of the project for completeness and consistency at the early stages of development, which affects the success of the development as a whole;

- *automatic program code generation.* Program code generation is based on a repository and allows for automatic construction of up to 85-90% of texts in high-level languages.

- *Maintenance and reengineering.* Maintenance of a system within the framework of CASE technology is characterized by maintenance of the project, not the program codes. Reengineering and reverse engineering tools allow creating a system model from its codes and integrating the obtained models into the project, automatically updating documentation when codes are changed, automatically changing specifications when codes are edited, etc.

Not all CASE tools support all the above capabilities. Therefore, CASE tools are usually relate any program product, automating that or other totality processes software life cycle and has the following main characteristics:

- the presence of powerful graphic tools for describing and documenting the information system, providing a convenient interface with the developer and developing his creative capabilities;

- integration separate components CASE tools, providing controllability process development of information systems;

- use special way organized storages design metadata (repositories).

Lecture 2. Fundamentals of business process modeling. Program life cycle. Life cycle models.

Basics modeling business processes. Adequate description processes Maybe With with help procedures called modeling. The goal modeling is Creation precise, sufficient, laconic, a description of the system that is easy to understand and analyze, as a set of interacting components and the relationships between them

Systems such as an organization's activities or a process have complex internal structure and the complex nature of the interactions between the constituent elements. Very often it is impossible to achieve this by conventional means. give simple description, A hence, And provide understanding such systems makes them design, development and maintenance are labor-intensive and expensive.

Modeling requires the presence of a mandatory set of pictorial (expressive) funds And rules - methods object descriptions. Among the most common ways descriptions can be highlighted:

- Verbal description - description on natural language.

For example, description methods And order works By definition, classifications And identification processes in natural language.

- Mathematical description - description With with help funds And rules Mathematicians.

- Graphic description - description object With with help funds And rules graphic images.

For example, ESKD (Unified System of Design Documentation) is a set of tools and rules for obtaining a graphic description of an object, called a drawing.

The life cycle (LC) of any system is a continuous process that begins with the decision to create it and ends with the complete withdrawal of the system from operation. The structure of the LC of software information systems in accordance with the international standard **ISO/IEC 12207** (ISO – International Organization of Standardization) is based on **three groups** of processes:

- main processes (acquisition, supply, development, exploitation, accompaniment);
- auxiliary processes (documentation, verification, security qualities And etc.);
- organizational processes (control projects, education And etc.). Let us consider the definitions of some of these processes.

Development includes All works By creation IS V in accordance With given requirements. The development consists of 4 stages:

1. Formation And analysis requirements To system (V result is being compiled specification systems);
2. Conceptual design (creation of an information model of the system without reference to the type of computer and system software);
3. Design implementations (choice computing systems, systemic software funds, data structure design);
4. Physical implementation (development applied programs, bases data, their debugging And testing, writing documentation).

IS modernization is the process of replacing individual components of the system; it is carried out in connection with changes in the subject area, to improve the quality and reliability of the IS, for compatibility with other IS.

Every process The life cycle is characterized by certain tasks And methods their solutions, original data and results. Often the results of later processes change the initial data of earlier stages, therefore the IS life cycle is **iterative**.

Models vital cycle

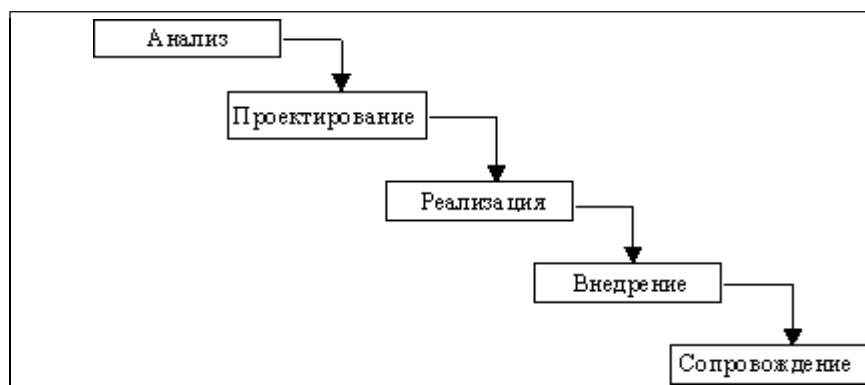
TO real time the greatest spreading received next two main models Life Cycle:

- cascading model (70-85 g.g.);
- spiral model (86-90 (g.g.).

In the initially existing homogeneous IS, each application was a single whole. For the development such type applications was used cascading way. His basic characteristic is dividing the entire development into stages, with the transition from one stage to the next occurring only after the work at the current one has been fully completed (Fig. 1). Each stage is completed with the release of a complete set of documentation, sufficient for the development to be continued by another team of developers.

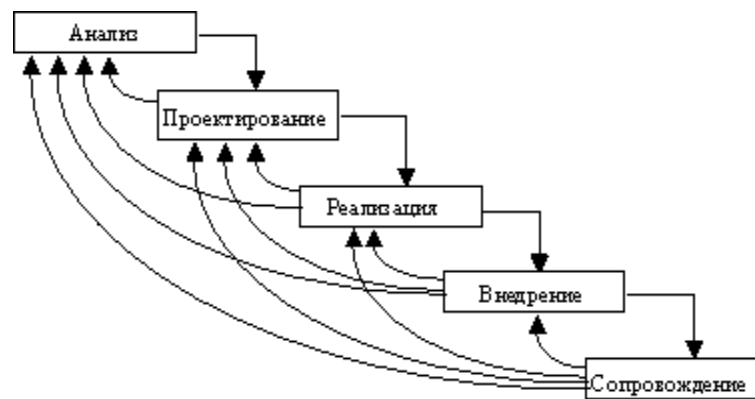
Positive sides applications cascade approaches are concluded V next:

- at each stage, a complete set of design documentation is formed that meets the criteria of completeness and consistency;
- Stages of work performed in a logical sequence allow planning the completion dates of all work and the corresponding costs.



Rice. 1- Cascade model Life Cycle

However, V in the process use this approaches showed up row his disadvantages, caused before total the fact that the actual process of software development never fully fit into such a rigid scheme. In the process of software development, there was a constant need to return to previous stages and clarify or revise previously adopted decisions. As a result, the actual process of software development took the following form:

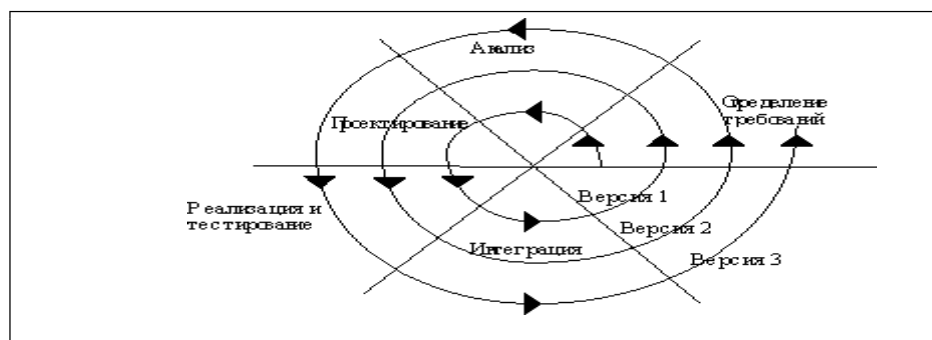


Rice. 2-Return model Life Cycle

The main disadvantage of the cascade approach is a significant delay in obtaining results. The results are agreed upon with users only at points planned after the completion of each stage of work, the requirements for the IS are "frozen" in the form of a technical assignment for the entire period of its creation. Thus, users can make their comments only after the work on the system is completely completed. In the case of inaccurate presentations requirements or their changes V flow long lasting period creations BY, users receive a system that does not meet their needs. Models (both functional and informational) of the automated object may become obsolete at the same time as they are approved.

For overcoming listed problems were proposed spiral model Life Cycle (Fig. 3), focusing on the initial stages Life Cycle: analysis And design. On these stages feasibility technical solutions is being checked by creating prototypes. Each turn of the spiral corresponds to the creation of a fragment or version of the software, on it the goals and characteristics of the project are specified, its quality is determined and the work of the next turn of the spiral is planned. In this way, the details of the project are deepened and consistently specified and as a result, a justified option is selected, which is brought to implementation.

Iterative development reflects the objectively existing spiral cycle of system creation. Incomplete completion works on each stage allows to cross over on next stage, Not waiting full completion work on the current one. With an iterative development method, the missing work can be completed at the next iteration. The main task is to show the users of the system a working product as quickly as possible, thereby activating the process of clarifying and supplementing the requirements.



Rice. 3 -Spiral model

Lecture 3. General requirements for methodology and technologies. Stages of system design. Requirements for the content of documents. Technical specifications.

Methodologies, technologies and design tools (CASE tools) form the basis of any system project. The methodology is implemented through specific technologies and supporting standards, methods and tools that ensure the implementation of life cycle processes.

Technology design is determined How totality three components:

- step by step procedures, defining subsequence technological operations design (rice. 4);
- criteria And rules, used For ratings results execution technological operations;
- notations (graphics) And text funds), used For descriptions projected systems.



Rice 4-Sequence operations

Technological instructions, which constitute the main content of the technology, should consist of a description of the sequence technological operations, conditions, V dependencies from which is being carried out ta or other operation, and descriptions of the operations themselves.

The technology for designing, developing and maintaining the system must meet the following general requirements:

- technology should support full Life Cycle BY;
- technology should provide guaranteed achievement goals developments IS With given quality And at the appointed time;
- technology should provide opportunity execution large projects V view subsystems;
- technology should provide opportunity management works By design separate subsystems in small groups (3-7 people).
- technology should provide minimal time receipt workable systems.
- technology should provide for opportunity management configuration project, management versions the project and its components, the possibility
 - automatic release project documentation And synchronization her versions With versions project;
 - technology should be supported a set of agreed CASE tools, ensuring automation of processes performed at all stages of the life cycle.

Requirements To content documents

Standard design project documentation must install:

- completeness, compound And structure documentation on each stages design;
 - requirements To her design (including requirements To content sections, subsections, points, tables And etc.),
- rules of preparation, consideration, approval and approval of documentation indicating deadlines for each stage;
- requirements for setting up a publishing system used as an integrated documentation preparation tool;
 - requirements for setting up CASE tools to ensure the preparation of documentation in accordance with established requirements.

Stages design systems

All period the creation of the system consists from several stages. Depending on which degrees are used in design ready-made or known technical solutions and methodology, some stages can be combined. IN others cases, against, separate stages (For example, sketch or technical design) can be supplemented by experimental ones work to explore new solutions, schemes and methods.

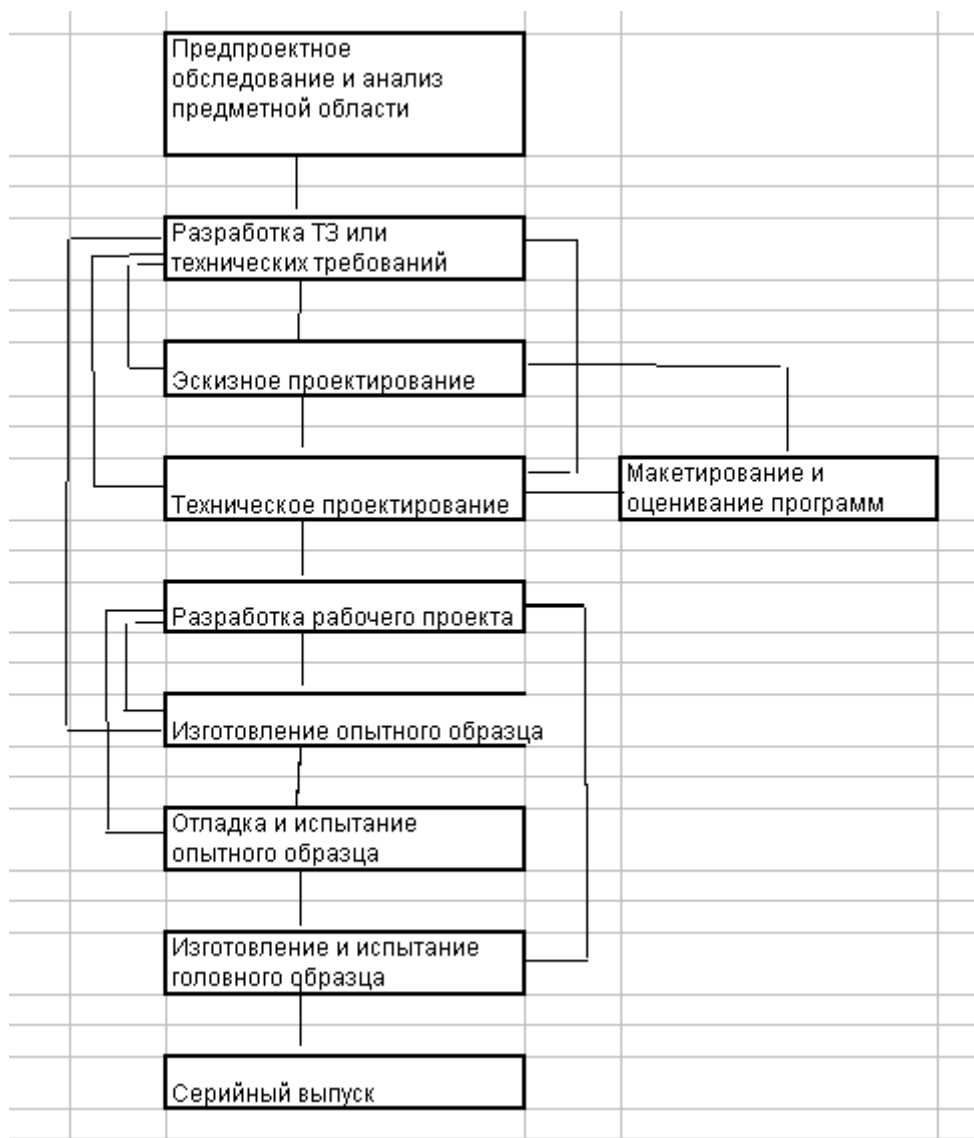


Fig.5- Stages design SYSTEM

1. Pre-project examination And analysis subject areas

It is being held studying And analysis subject areas V which will function system And the general statement of the problem of its creation is formulated.

Subject region - This totality objects real peace or supposed peace, considered within a given context, which is understood as a separate argument, an engineering discipline, a fragment of a scientific theory, or a theory as a whole and is limited within the framework of a given context.

Survey - This studying And diagnostic analysis existing systems processing information.

Target creations systems is formulated How V view specific technical requirements, so and view some general provisions.

2. Stage development technical requirements And technical tasks_(TZ)

Based on the technical proposals reviewed, the customer formulates restrictions for the system being created, consisting from goals systems And forcing connections -factors, limiting choice ways achieving the goal (other in words, customer sets the initial requirements To system, determined by its purpose and the conditions of its creation or use.

The technical specification is the main source document for developers and customers, which contains a systematic description technical requirements To projected system, A Also order her developments And acceptance V exploitation. This is basic original document For developers And customers, which contains systematic description technical requirements To projected system, A Also order her developments And acceptance V operation. Approved TZ is document, which developers should be guided by on all stages of creation systems And design tasks. Changes, contributed V TZ, should to be formalized protocol, being part TZ. Protocol must assert oneself by the customer.

The technical specifications are also developed based on the results of pre-project research and experimental work, analysis and forecasting advanced foreign And domestic scientific and technical achievements.

Agreement between the customer and the contractor on methods for evaluating the system at the initial stage of its creation is a necessary condition for the most complete satisfaction of the customer's requirements. At the same stage, If necessary, proposals that alleviate the problems of creating the system should be agreed upon between the customer and the contractor.

3. Development sketch project

Compound works:

- definition optimal ratios hardware And software ways implementations automated functions of the system;
- clarification decompositions systems on separate complexes;
- study And approbation analogues systems;
- modeling functions And characteristics systems;
- definition of methods and means of organization structures data; processing interfaces (external, user, intercomplex) for data and management;
- clarification requirements systems To computing resources;
- development of refined requirements for the system; drafting an external system specification on functional specification language.
- development of quantitative indicators and methods for quality assessment; development of specific technical specifications for tests for checking, debugging and testing the system and its components, private technical specifications for automated testing equipment.
- development of a model taking into account the entire life cycle; approximate calculations of labor costs and time are carried out And cost developments; is being carried out grade real deadlines developments And existing resources; A consolidated end-to-end development schedule is formed.
- selection and justification of the main concepts of development technology and composition of technological equipment; development of private technical specifications for the components of R&D and programming; creation of cooperation; development of draft guidelines instructions To development; clarification TZ on software means; development private TZ on exercise machines and educational tools; creation of a software project database for automation of management and control of the development process; development of an explanatory note to the draft design.

4. Development technical project. .

Stage technical design characterized by more deep elaboration all main parts systems, and, unlike a draft design, where the existence of several options is required, The technical project defines the only solutions to the main issues. All technical The system's solutions must be technologically coordinated. This requirement determines the level of detail of the project and the degree of design development of the system elements. In some cases, this may require prototyping of individual components. system blocks and their experimental study. As a result of this work, technical conditions (TC) for the delivery of the system are drawn up.

IN composition technical project systems should be next sections:

- description common principles functioning,
- common structures systems With indication subsystems;
- scheme streams information With indication ways transmissions information;
- compound hardware funds;
- technical conditions on supply systems;
- enlarged schedule developments And implementations systems.

The technical project provides for " *Problem statement* " (or "Initial data", which includes: name tasks, her meaningful formulation; data O periodicity solutions tasks; connections given tasks with others tasks And her place is in complex subsystem tasks; description of the method organizations collection of initial data and transferring them to the memory of information processing facilities; description of the algorithm for solving the problem, accuracy of solutions, methods control calculations; calculation reliability; formulation temporary restrictions on issuance solutions tasks; justification of the appropriateness of the proposed version of the task in comparison with other options.

Here same are given (V applications) description forms input documents, forms intermediate documents, information on the presentation of information necessary for communication with other tasks.

Let's give compound main documents technical project V in accordance With state standards:

Technical project of AIS – a technical document (system of technical documents) containing detailed technical solutions for the AIS architecture, all AIS components and an exact description of the AIS functions;

Compound project AIS - chapter (document) technical project, V which indicated location the main design solutions in the relevant sections (documents) of the TP AIS;

Calculation of the economic efficiency of the AIS is a section (document) of the technical project, which specifies: the methodology for determining the economic efficiency of the AIS, calculation of the costs of creating the AIS, calculation of the costs of operating the AIS, calculation of the annual economic effect from using the AIS;

List of documents of TP AIS - a document containing a list of documents included in this TP, including those borrowed from other projects;

General system documentation of the AIS project - documentation intended for a general description and justification of decisions made in the project - explanatory note, general description, calculation of economic efficiency, etc.;

Description of information support (IS) is a section (document) of the TP that contains a description of: principles of organizing IS, classification and coding of data, organization of databases.

TP documents are transferred to the holders of the original technical documentation – the organizations in which they are stored, on a legal basis, originals of all technical documentation for AIS.

The developed technical design of the system is accepted by a commission appointed by the customer. Commission decision With offers And comments it is stated by the customer And is basis For worker design.

5. Development worker project .

Consists of the release of working documentation, according to which the system is manufactured, its debugging, testing and commissioning are carried out. Working programs and instructions for their use and modification are developed. In progress solutions, accepted on stages technical design. Practically this stage consists of two sections:

- *preparation of working drawings* (working documentation) which include: machine algorithms and problem solving programs, instructions for their operation; instructions By preparation initial data to solve problems and use the results obtained; job descriptions; working documentation for placement, installation and installation of hardware; operating instructions. After conducting preliminary tests And elimination identified disadvantages is being carried out adjustment working documentation. Working documentation is practically created in throughout the development of the working project.

- **Creation experienced sample systems** (usually on stand, on which is being carried out complex docking and debugging, it is checked here correspondence systems given in the technical specifications).

The prototype is sent for testing, which are being carried out departmental commission in accordance with the methodology (program) agreed upon and approved by the contractor and the customer.

Lecture 4-5. CASE-technology of systems design. CASE-systems of structural and object-oriented types. Practical recommendations on mastering and implementing CASE-tools, criteria for their selection and comparative analysis

In the 70s and 80s, structural methodology was widely used to automate the design and development of systems, which meant the use of formalized methods for describing the system being developed.

In this case, graphic means of describing various models of systems using diagrams and charts were used. In manual development of systems, such graphic models are labor-intensive to develop and use. These circumstances served as one of the reasons for the emergence of software and technological tools, called CASE tools and implementing CASE technology for creating and maintaining systems as a whole.

CASE technology design systems

CASE - is an acronym Computer-Aided System / Software Engineering, unifying methods automation the most process design informational systems. CASE means support or automate works on all stages vital cycle systems.

Most modern CASE systems use methodologies structural analysis and design based on visual diagrams, using graphs, diagrams, to describe the model of the system being designed tables And schemes. Such diagrams provide strict And visual description projected system, which begins with a general overview and then goes into detail.

Despite the fact that structural methodologies originated as tools for software analysis and design, the scope of their application is currently wide.

Therefore, CASE technologies are successfully used for modeling systems (business processes), but they occupy a stable position in the following areas:

- Business analysis (in fact, business activity models are built using CSA methods and supporting CASE tools).
- Systemic analysis And design.

CASE technologies developed With purpose overcoming restrictions at use structural design methodologies of the 60-70s due to the complexity of understanding, high labor intensity, etc.

Thus, CASE technologies cannot be considered independent methodologies; they only develop structural methodologies and make their application more efficient through automation. Therefore, one of their key features is support for structural system analysis methodologies.

The goal CASE technologies is acceleration process creations systems And increase their quality.

CASE tools Not Necessarily give instant effect, He Maybe be received only later some time. The actual costs of implementation usually far exceed the costs of their acquisition.

Benefits at successful implementation:

- 1) High level technological support processes developments And accompaniment BY;
- 2) Positive impact on such factors How: performance, quality products, compliance with standards, documentation.
- 3) Level recoil from investments V CASE- means.

Advantages CASE tools By comparison With traditional BY:

- improvement qualities developments applications for check funds automatic control And generations;
- opportunity repetitive use components developments;

- decrease time creations systems;
- freeing developers from routine work on documenting the project, since it uses a built-in documenter;
- opportunity final developments IS V mode real time.

Program, How finite product design, contains Not just the ideas of the programmer, but as well as ideas systemic analytics And requirements users. All This Maybe be used V new system. It should be taken into account that old programs were written without observing standards, and the information extracted in such cases is formulated in physical terms (for example, in variable addresses).

Architecture CASE tools:



The repository (encyclopedia, data dictionary) is the core of the system and is a specialized base data, on basis which synchronization of all descriptions (diagrams) and coordination of efforts is performed all developers. Repository contains information about objects projected IS And interrelations between them:

- designers And their rights access To various components systems;
- organizational structures;
- diagrams, reflective organizational, functional, informational, technological And other aspects of the system;
- components diagrams;
- connections between diagrams;
- structures data;
- software modules;
- procedures;
- libraries modules And etc.

Graphic editor diagrams allows create various descriptions systems V view drawings And can perform the following operations:

- create elements diagrams And interrelations between them;
- ask descriptions elements diagrams;
- ask descriptions connections between elements diagrams;
- edit elements diagrams, their interrelations And descriptions.

Verifier diagrams controls right diagrams And synchronizes diagrams between by yourself.

Documenter project documents project V view various reports. Reports can to be built by several criteria, such as time, author, diagram elements, diagram, or the project as a whole.

Administrator project automates execution following administrative functions:

- initialization project;
- tasks initial parameters project;
- appointments And changes right access To elements project;
- monitoring execution project.

Service performs functions archiving data, recovery data And creations new repository.

The models used at the design stage form a metaphor design or approach To design.

CASE systems structural And object-oriented types.

There are 2 main approaches To development systems, fundamental difference between which is determined by different methods of decomposition systems.

The first approach is called functionally modular or structural. It is based on the principle of functional decomposition, in which the structure of the system is described in the hierarchy of its functions and the transfer of information between individual functional elements.

Second, object-oriented approach performs object decomposition.

1 Functionally oriented approach To development systems.

The essence of the structural approach to systems development lies in decomposition (breakdown) into automated functions: system breaks down on functional subsystems, which, V my queue be divided into subfunctions, those into tasks, and so on to specific procedures. At the same time, the automated system maintains a holistic view in which all constituent components are interconnected.

IN structural approach are being performed V basically two groups funds, describing functional structure of the system And relationship between data. Each one group funds correspond certain types models (diagrams), the most common of which are:

- DFD (Data Flow Diagrams) - diagrams streams data;
 - SADT (Stastured Analysis and Design Technigue) - (method structural analysis And design) - models and corresponding functional diagrams;
 - ERD - diagram "entity- connection".
- On stages formations requirements To According to the SADT model DFD are used For constructions models “AS-IS” and the “TO-BE” model, reflecting the existing and proposed structure of the organization’s business processes.

At the design stage, DFDs are used to describe the structure of the software system being designed. ERDs describe the representation of data at a logical level.

2 The object-oriented approach uses object decomposition, while the static structure of the system is described V terms objects And connections between them, A behavior systems is described V terms exchange of messages between objects. Each object of the system has its own behavior, simulating the behavior of a real-world object.

True object-oriented modeling languages began to emerge between the mid-1970s and the late 1980s.

An important link in the development of object technologies was the formation of the ObjectManagementGroup (OMG) consortium in 1989.

The result of the OMG's efforts to this end was the adoption in September 1997 of a language standard called the Unified Modeling Language (UML).

Development UML started V at the end 1994 years, When Butch And Rambo from Rational Software Corporation started their work on combining Booch and OMT (ObjectModelingTechnique) methods.

Unified language modeling complex systems - language UML:

1. Allows modeling both software of complex systems and broad classes of systems and business applications themselves, using object-oriented concepts and methods.
 2. Provides relationship between basic concepts models conceptual, software and physical levels.
 3. Understood systemic analysts And programmers.
- Supported special instrumental software means, realized on various computer platforms.

Practical recommendations By development And implementation CASE—means, criteria their choice And comparative analysis .

Exists some approaches To development strategies implementations CASE- funds.

1 The top-down approach to strategy development recognizes the importance of examining all types of CASE tools and documenting the software development and maintenance processes in a given organization before defining requirements for CASE tools. In this case, an overall analysis of the software development and maintenance process in the organization is performed. This approach often entails a general reorganization of the software development and maintenance processes as they relate to CASE tools. The result of such a reorganization is a large-scale strategy for automating the software development and maintenance processes.

The advantage of the top-down approach is that it covers all processes of software creation and maintenance, ensuring their maximum possible automation. Another advantage is the acquisition of an integrated (or integrable) set of tools, since each individual delivery is subject to common strategies. Descending approach Also Maybe be easily integrated V general strategy development of the process of creating and maintaining software, in which the implementation of CASE tools is only one aspect.

Flaws of this approaches are concluded V next:

- descending approach requires For his implementations significant human And financial resources;
- In general, a large-scale approach of this kind does not allow users to start using the tools in practice quickly enough;
- A top-down approach can result in relatively major changes to existing processes within an organization. It is more difficult to manage and carries a higher risk of failure, leading to CASE tools being "shelved".

Descending approach recommended For relatively mature organizations With established process creations and software maintainers who strive to invest all necessary resources into a fully completed work. To increase the likelihood of success, serious commitment is required from both management and potential users.

2 The bottom-up approach begins with identifying some tool or type of tool that could potentially help the organization improve the performance of its current work. The organization can then evaluate the potential impact of the tool on the software development and maintenance process.

Advantages of this approaches are concluded V next:

- small automation Maybe be completed at minimal costs;
- automation Maybe be completed for short interval time, allowing fast eliminate known deficiencies in existing processes;

- The small scale of a bottom-up strategy allows for greater focus and control over the impact on existing processes.

Flaws of this approaches are concluded V next:

- the tools acquired as a result of individual applications of this approach may not integrate well with each other. This may lead to the need to perform a large amount of manual work;
- While specific, relatively small problems are solved fairly quickly, fundamental problems associated with a wide range of software development processes are usually not addressed.

Ascending approach recommended For organizations With narrowly specific needs V automation, not requiring general process improvement.

The most efficient strategy may combine characteristics of both approaches. For example, top-down methods may be used to define the organization's quality standards, resource needs, and expected results, while bottom-up methods may be used to evaluate and select specific CASE tools, develop implementation plans, and monitor their results.

IN quality main criteria choice CASE tools are accepted next criteria:

1. Support full vital cycle IS With provision evolutionary her development
2. Security integrity project And control for his condition
3. Independence from software and hardware platforms And DBMS
4. Support simultaneous works groups developers
5. Opportunity developments applications "client-server" required configurations
6. Open architecture And possibilities export/import

Lecture 6. Structural approach to software design. SADT functional modeling methodology. Functional model of work organization - AS IS. TO BE model and its functional cost analysis.

SADT is one of the most famous methodologies for systems analysis and design, introduced by Douglas Ross. This methodology supported Ministry defense USA, which was initiator developments IDEF0 standard as a subset of SADT.

The main workers element at modeling is diagram. The SADT model combines And organizes diagrams V hierarchical tree-like structures. IN compound diagrams enter blocks, depicting the system being modeled and the arcs that link the blocks together and depict the interactions and relationships between the blocks.

SADT requires to V diagram was 3-6 blocks: V these within diagrams And models comfortable For readings. Instead of one cumbersome models are used some small interconnected models.

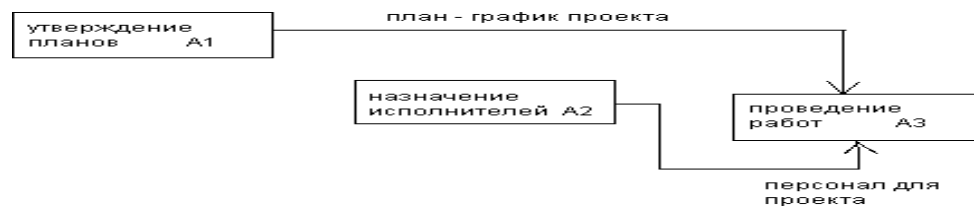
Blocks in diagrams are represented by rectangles and are accompanied by natural language text describing activities. IN SADT each side block has certain special meaning : left side blocks are intended for Inputs, the upper one is for Control, the right one is for Outputs, and the lower one is for Executors. This designation reflects certain principles: Inputs are transformed into Outputs, Control prescribes the conditions for execution, and Executors describe how the transformations are performed.

Arcs V SADT represent sets items And are marked texts on natural language. Items can consist of four possible relationships: Input, Output, Control, Executor. Input arcs are transformed by activities. Control arcs depict information that controls the actions of activities. Output arcs depict objects into which inputs are transformed. Executor arcs reflect the implementation of activities;

Example SADT - block:



SADT requires only five types of relationships between blocks to describe their relationships: Control, Control Feedback, Input Feedback, Output – Performer. The Control and Input relationships are the simplest – they reflect direct influences. The Control relationship arises when the Output of one block directly influences on another block With smaller dominance. Attitude Login arises, When Exit of one block becomes an Input for a block with less dominance. Control Feedback occurs when the output some block influences on block With big dominance, A attitude Input Reverse connections has a place where the Output of one block becomes the Input of another block with greater dominance. Output – Performer relationships (rare) reflect a situation in which the Output of one activity becomes a means to achieve the goal of another activity. An example of an Output – Performer relationship:



An example of a SADT diagram modeling the activities of a company engaged in the distribution of goods by orders:



SADT methodologies design it is advisable use on early stages Life Cycle: For understanding system before its implementation.

Often, model development is used to document a situation that has arisen at a certain point in time. (models "How There is" - "as is"). Later They are applied at creation new models functioning of the system (models of "how it should be" – "to be").

Business process reengineering is defined as a fundamental rethinking and radical redesign of business processes to achieve fundamental improvements in an organization's performance.

The purpose of business process reengineering is the systematic reorganization of material, financial and information flows aimed at simplifying the organizational structure, redistribution and minimization of use various resources, maintaining deadlines realizing customer needs and improving the quality of their service.

- *Models "How There is" - "as is"*, represent by yourself "snapshot" provisions affairs on enterprise (organizational structure, interactions between departments, adopted technologies, etc.) at the time of the survey, and allowing one to understand what the given enterprise does and how it functions from the standpoint of systems analysis.

- *Models "How should be" - "to be"*, integrate promising offers And allow to form a vision of new rational technologies for the organization's work.

Stages developments projects.

The main ones goals developments projects are:

- Performance activities organizations And accepted V German technologies V view hierarchies diagrams;
- Formation on basis analysis offers By reorganization organizational and management structure;
- Organizing informational streams inside organizations;
- Production recommendations By construction rational technologies works divisions organizations

Stage 1. Main tasks: preliminary studying tasks, analysis primary business requirements, preliminary economic assessment of the project, development of a work schedule;

Preliminary studying tasks, gradually answer on questions:

- IN how flaws existing systems?
- Which improvements possible?
- On whom will provide influence new system?

On this stage it is advisable build overview diagram streams data For ratings the existing situation to identify shortcomings.

Stage 2. Conducting examinations activities organizations. On this stage is carried out:

- Preliminary identification requirements, presented To future system;
- Definition organizational staff And itopological structures organizations
- Definition list functions organizations
- Analysis distributions functions By divisions And employees
- Definition list applied on organizations funds automation.

At this stage, the functional activities of each division and the functional interactions between them, objects external to the enterprise and external information interactions are identified.

Stage 3. Building models activities organizations. It is being implemented survey processing and model building activities organizations next 2-my types: Models "How There is" - "as is", models "How should be" – "to be".

Systemic project under construction on basis models "How should be" And results examinations organizations.

Stage 4. Development offers By automation.

Conducting examinations.

IN quality original information at conducting examinations serve:

- Data By organizational and staffing structure organizations;
- Information O accepted technologies activities;
- Strategic goals And Prospects development;
- Results interviewing employees;
- Normative and reference documentation;
- Data By existing V organizations means And systems automation;
- Experience systemic analysts.

When conducting a survey, it is advisable to use the following methods: questionnaires, document collection, interviews. Exemplary option questionnaires:

- Main functions divisions;
- Which information is being formed V division;
- WITH what external organizations (banks, tax service And etc.) interact divisions And what information is

exchanged;

- Documents from And For manuals;
- Used software products. Please attach:

1. Position O division.
2. Used forms, forms And other (For example, report floor form No., task order, consignment note, etc.).

Target interviewing, firstly, identification all external objects, With which is being modeled the organization interacts, the technology of interaction on the part of the organization, as well as information flows that ensure these interactions. Secondly, identifying the real technologies of the organization's work. Thirdly, determining the real functions of the departments and their interrelationships and interdependencies.

Lecture 7. Methodology of functional modeling IDEFO. CASE-tool of structural type BPWin.

The IDEFO functional modeling methodology is a technology for describing a system as a set of interdependent actions. IDEFO is most often used as a technology for researching and designing systems at the logical level. For this reason, it is used in the early stages of project development. The results of IDEFO analysis can be used when designing using IDEFO models and data flow diagrams.

Syntax And semantics models IDEFO. Models IDEFO.

IDEFO combines a graphical potency (it contains only two symbols: blocks and arrows) with strict and specific recommendations, which together are intended to build a high-quality and understandable model of the system.

The first step in building an IDEFO model is to define the purpose of the model. - a set of questions that the model must answer.

The boundaries of modeling are intended to indicate the breadth of coverage of the subject area and the depth of detail and are a logical continuation of the already defined purpose of the model.

The next step is to specify the intended target audience for whose needs the model is being created. From the choice of the target audience depends on the level of detail with which the model should be created. Before building a model, it is necessary to have an idea of what information about the subject of modeling is already known, what additional materials and technical documentation are necessary to understand the model target audience.

The point of view refers to the perspective from which the system was observed when the model was constructed. Point of view is selected like this in this way, to take into account already designated borders modeling And appointments models. Other models can be created to display the data if necessary. system from other points of view. Examples of points of view when building models: client, supplier, owner, editor.

Actions.

An action, called a function in **IDEFO**, processes input parameters on weekends. Since **IDEFO models** represent a system as a set of hierarchical functions, the function that describes the system as a whole must first be defined - context function. Functions are displayed on diagrams as named rectangles or functional blocks. Important pick up names like this in this way, to reflected the system so that names could be viewed from the perspective chosen for modeling.

Borders And connections

A block description must include a description of the objects that the block transforms ("input") and the objects that the block creates as a result of its work ("output").

IDEFO also models control and execution mechanisms. Control refers to objects that affect the way a block transforms input to output. Execution mechanisms refer to objects that actually perform the transformation of input to output.

For displays categories information, present on diagrams **IDEFO**, exists abbreviation

ICOM, displaying 4 possible arrow types :

I (Input) - entrance- What is consumed V in the course execution process;

C (Control) - control - restrictions And instructions , influencing on move execution process;

O (Output) - exit - result execution process;

M (Mechanism) - performing mechanism - What is used For execution process.

4 possible type shooter in IDEFO:



Arrows can represent people, places, things, ideas, events. As with function blocks, naming all the arrows in a diagram is a must.

Input arrows. Input represents the raw material or information consumed or transformed by the functional block to produce the output. Input arrows always point to the left side of the rectangle. The presence of input arrows in the diagram is not mandatory, since it is possible that some blocks they don't transform anything And do not change. An example of a block is, can not have an entrance serves "management decision making", Where For acceptance solutions is signaled some factors, But no one from them Not is transformed and is not consumed as a result of making any decision.

Control Arrows: Control arrows are responsible for regulating how and when a function is executed. block, And If He is being carried out, Which entrance it turns out V result his execution. So How control controls the behavior of a function block to produce the desired output, - then each functional

block must have at least one control arrow. Control arrows always enter the function block from the top.

Management exists in the form of rules, regulations, laws, policies, sets of required procedures or standards. Influencing work block, it is not directly consumed and does not transform V result. It may turn out that the purpose of the functional block is a change of a rule, standard, etc. In this case, the arrow containing the corresponding information should be considered not as a control, but as an input of a functional block.

Control Can consider How specific view entrance. IN cases, When it's unclear, to refer li arrow to the input or to the control, it is preferable to attribute it to the control until the ambiguity is resolved.

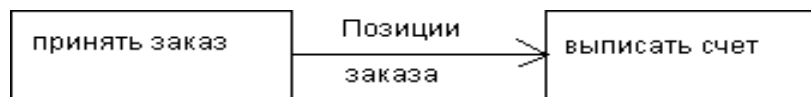
Output arrows. An output is the product or information that results from the operation of a function block. Each block must have at least one output. An action that does not produce any output should not be modeled at all.

When modeling non-production subject areas, the outputs are the data processed by the functional block. In this case, it is important that the names of the input and output arrows are distinguishable. For example, the block "Patient Admission" may have an arrow "Patient Data" at both the input and output. In what situation is the input arrow Can name "Preliminary data O patient", A outgoing - "Confirmed data about the patient."

Arrows mechanism execution. Mechanisms are resource, which performs simulated action. Key personnel, machinery and equipment can be modeled using execution mechanisms. Execution mechanism arrows may be absent if they are found to be unnecessary to achieve the goal modeling.

Combination arrows. There are five main types of combination arrows in IDEF0 : output-input, input-control, output-execution mechanism, output-control feedback, output-feedback connection to the input.

1. Combination shooter exit-entry formation bills should precede reception order:



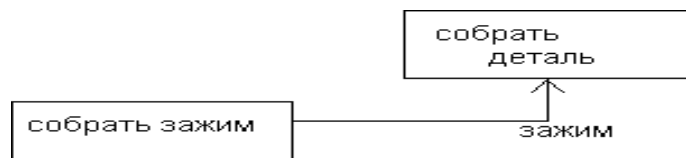
2. Arrow exit - control reflects the situation of dominance of one block over another, when one block controls work other.

Arrow combination Exit-management: principles of investment portfolio formation control the behavior of brokers on the stock exchange:



3. The exit-mechanism arrow is less common and reflects situation. When the output of one functional block is used as equipment for the operation of another block.

combination of arrows Output-execution mechanism:

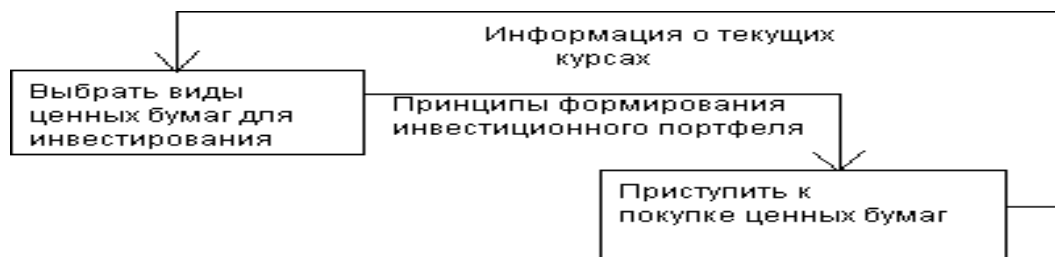


- clamp, must be collected For Togo, to to execute assembly details.

Reverse connections on entrance And on control are applied V those cases, When dependent blocks form feedback loops for the units that control them.

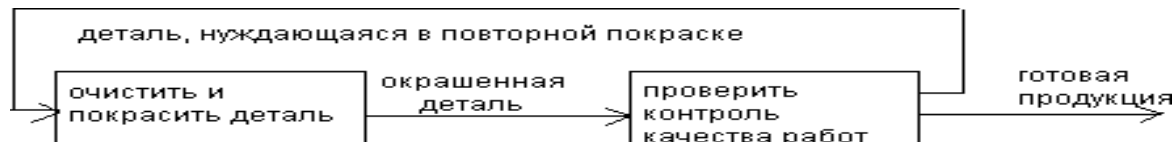
4. Combination arrow output-reverse connection on control.

Information received from brokers about current stock exchange rates is used to adjust the strategy of playing on the stock exchange:



5. Arrow exit - reverse connection at the entrance usually applies For descriptions cycles of repetition processing something. Except Togo this connection applies V case, If defective products Maybe again be used V as raw materials.

Combination arrow output-reverse connection on entrance:



Splitting and connecting arrows. The output of a function block can be used in several other blocks. **IDEFO** provides for How partition, So And compound shooter on diagram. Broken on some The parts of an arrow may have names that differ from the name of the original arrow. The original and the broken (or merged) arrows together are called linked. This technique is usually used to reflect the use in a process of only part of the raw material or information represented by the original arrow. A similar approach is used for merged arrows.

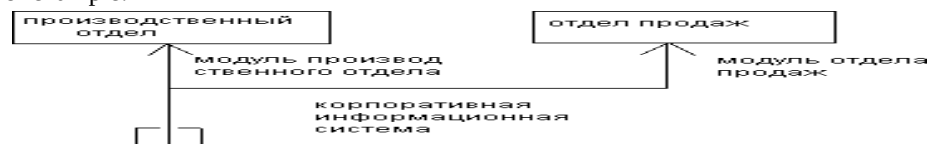
Broken on 2 parts And renamed arrow:



Tunnels

Concept related arrows are used For management level details diagrams. If one from the diagram arrow is absent from the parent diagram (for example, due to its irrelevance for the parent level) and is not connected to other arrows of the same diagram, the insertion of the entry of this arrow to or from the diagram is indicated by a tunnel.

For example:



Arrow "Corporation I.S"- important mechanism execution For given diagrams, But, Maybe, she more is not used anywhere in the model. The tunnel is used in this case as an alternative to cluttering up parent diagrams by placing arrows on them that are not essential for their level.

Additionally, tunnels are used to reflect the situation when an arrow present in the parent diagram is missing from the corresponding block decomposition diagram:



The tunnel at the arrow "production department module" means that the department's production breakdown diagram does not have a control mechanism arrow with the corresponding name.

BPwin supports three methodologies structural analysis And modeling systems - IDEF0, IDEF3 And DFD. In the process of creation models business process on any branches models you can switch to any from methodologies and create a mixed model.

Construction models IDEF0 . Diagrams.

Service information V typical **IDEFO** diagram consists of from allocated top And lower headers and footers (header and footer).

USED AT:	AUTOR: Semenov Oleg Olegovich DATE 15.09.04 PROJECT: Department accounting And reporting REV 17.02.04 NOTES: 1234567890	WORKING	READER DATE	CONTEXT: AO
		DRAFT		
		RECOMMENDED		
		PUBLICATION		
DIAGRAM IDEFO				
MODE: AO	TITLE: Department accounting And reporting			NUMBER

Header elements are used to track the process of creating a model. Footer elements display the name of the model to which the diagram belongs and show its location relative to other diagrams in the model.

Elements Header diagrams **IDEFO**:

Field	Purpose
Use at	Used For reflections external links on given diagram (to be filled in) on printed document manually)
Author, date, project	Contains Full name author diagrams, date creations, date last contributions changes , Name project, V within the framework whom it was created.
Notes 1...10	At manual editing diagrams users can cross out figure every once, When They contribute another one correction.
Status	Status reflects state developments or statement given diagrams. This field is used For implementations formal process publications With in steps revision and statements.
Working	New diagram, global changes lilo new author For existing diagrams.
Draft	Diagram reached some acceptable For readers level And ready For performances on statement.
Recommend ed	Diagram approved And approved. Any changes Not are expected.
Publication	Diagram ready For final seals And publications.
Reader	Full name reader.
Date	Date acquaintance reader With diagram.
Contex	Sketch locations functional blocks on parental diagram, on which is illuminated decomposable given diagram block. For diagrams the very top level (contextual diagrams) V field fits context TOP
Mode	Number diagrams, coinciding With number parental functional block.
Title	Name parental functional block
Number (also called C-Number)	The identifier of this version of this diagram. Thus, each new version of this diagram will have a new value in this field. The C-Number consists of the author's initials and a sequential unique identifier, such as SDO005. At publications these numbers can be replaced standard page numbers. If a chart replaces another chart, the number of the chart being replaced may be concluded V brackets - SDO005 (SDO0004)/ This provides history storage changes to all model diagrams.

Lecture 8-9. Display logical And physical level models V ERWin. CASE–tool of the ERWin structure type

ERwin has two level *performances* models — logical And physical.

The logical level is an abstract view of the data, where the data is presented as it appears in real life. world, And can be called So, How They are called V real world, For example "Constant client", "Department" or "Surname employee". Objects models, represented on logical level,

are called *entities* And *attributes* . *Logical model data* Maybe be built on basis another logical model, For example on based on the model processes. *Logical model data* is universal And no way is associated with a specific implementation of the DBMS.

The *physical data model* , on the contrary, depends on a specific DBMS, being in fact a reflection of the system catalog. The physical model contains information about all DB objects. Since there are no standards for DB objects (for example, there is no standard for data types), the physical model depends on a specific DBMS implementation. Consequently, several different physical models may correspond to the same logical model. If in the logical model it does not matter what specific data type *the attribute has* , then in the physical model models important describe all information O specific physical objects — tables, columns, *indexes* , procedures, etc.

Documentation models

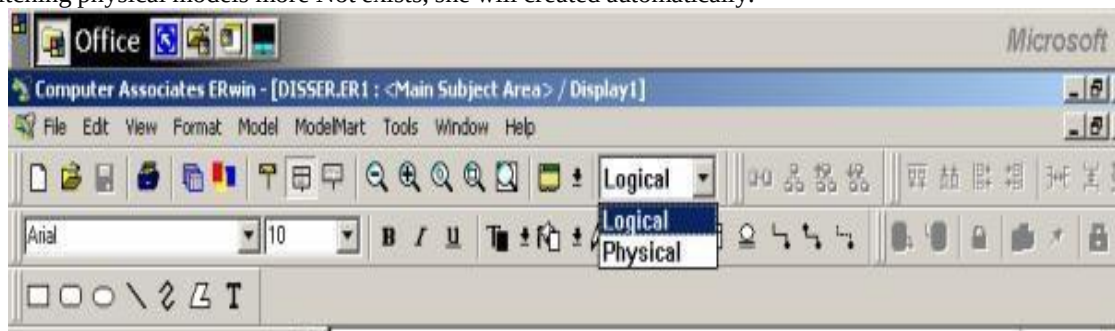
Many DBMS have limitation on naming objects (For example, limitation on length name tables or prohibition of using special characters - spaces, etc.). Often, IS developers deal with non-localized versions of DBMS. This means that DB objects can be named with short words, only Latin characters and without using special characters (i.e., you cannot name a table using a sentence - it can only be named with one word). In addition, DB designers often abuse "technical" names, as a result, the table and columns receive names like RTD_324 or CUST_A12 And etc. Received V result structure can understand only specialists (A more often total — only authors of the model), her impossible discuss With experts subject areas. Separation models on logical And physical allows you to solve this problem. At the physical level, DB objects can be named as required by the DBMS restrictions. At the logical level, these objects can be given synonyms - names more understandable to non-specialists, V volume number on Cyrillic And With using special characters. For example, the **CUST_A12 table** might correspond to the **Regular Customer entity** . This correspondence allows for better documentation of the model and enables discussion of the data structure with subject matter experts.

Scaling

Creation *models data* , How rule, it begins With developments logical models. After descriptions logical model, the designer can select the required DBMS, and ERwin will automatically create the corresponding physical model. Based on the physical model, ERwin can generate a system catalog of the DBMS or a corresponding SQL script. This process is called *forward engineering* . This achieves scalability - having created one *logical data model* , you can generate physical models for any DBMS supported by ERwin. On the other hand, ERwin is capable of creating a system catalog or SQL script recreate physical And *logical model data* (Reverse Engineering). On Based on the obtained *logical data model* , it is possible to generate a physical model for another DBMS and then create its system catalog. Therefore, ERwin allows you to solve the problem of transferring the data structure from one server to another. For example, Can postpone structure data With Oracle on Informix (or vice versa) or postpone structure of dbf files into a relational DBMS, thereby facilitating the transition from a file-server to a client-server IS. However, formally transferring the structure of "flat" tables to a relational DBMS is usually ineffective. In order to benefit from the transition to client-server technology, the data structure should be modified.

For switching between logical And *physical model data* serves list choice V central parts of the ERwin toolbar (Fig. 6).

If at switching physical models more Not exists, she will created automatically.



Rice. 6- Switching between logical And physical model ERwin

Interface. Model Display Levels

Interface completed V style Windows applications, enough simple And intuitively understandable. Let's consider Briefly describes the main functions of ERwin for model display.

To everyone level displays models corresponds own palette tools. On **logical At this level**, the tool palette has the following buttons:

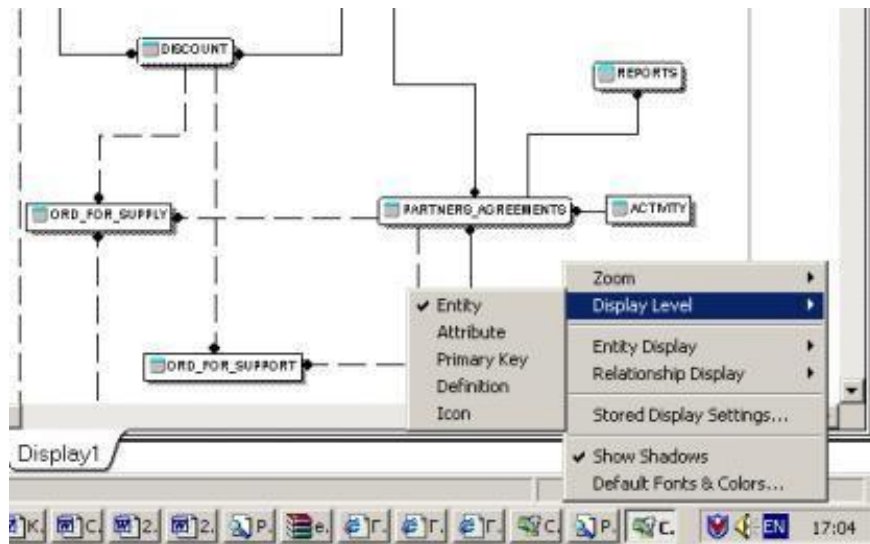
- button pointer (mode mice) — V this mode Can install focus on any object models;
- button contributions *entities* ;
- button categories (category, or categorical *connection* , — special type *connections* between *entities* , which will be discussed below);
- button contributions text block;

- button transfers *attributes* inside *entities* And between them;
- buttons creations *connections* : identifying, "many-to-many" And non-identifying. At **the physical level**, the tool palette has:
- instead of buttons categories — button contributions *representations* (view);
- instead of buttons *connections* "many-to-many" — button *connections of representations* .

For creations *models data* V ERwin Can use two Notations: IDEFIX And IE (Information Engineering).

IN further will be considered notation IDEFIX.

ERwin has several levels of diagram display: *entity level* , *attribute level* , definition level, *primary key level* and icon level. You can switch between the first three levels using the toolbar buttons. You can switch to other display levels using the context menu, which appears, If "click" By to anyone place diagrams, Not busy objects models. In the context menu, select the Display Level item (Fig. 7) and then the required display level.



data model

Levels logical models

Distinguish three level logical models, different By depth *performances* information O data:

- *diagram entity-relationship* (Entity Relationship Diagram, ERD);
- *model data , founded on keys* (Key Based model, KB);
- *full attributive model* (Fully Attributed model, FA).

An entity-relationship diagram is a top-level *data model* . It includes *entities* and relationships that reflect the main business rules of the subject area. Such a diagram is not very detailed, it includes the main *entities* and *relationships* between them that satisfy the main requirements for the IS. *An entity-relationship diagram* can include many-to-many *relationships* and not include a description of keys. As a rule, ERD is used for presentations and discussions of the data structure with subject area experts.

key-based data model is a more detailed *representation* of data. It includes a description of all *entities* and *primary keys* and is designed to *represent* the structure of data and keys that correspond to the subject area.

The full attribute model is the most detailed *representation* of the data structure: it represents the data in the third normal form and includes all *entities* , *attributes* , and *relationships* .

Entities And attributes

The main components of an ERwin diagram are *entities* , *attributes* , and *relationships* . Each *entity* is a set of similar individual objects called instances . Each instance is unique and must differ from all the rest copies. *Attribute* expresses certain property object. WITH dots From the point of view of the database (physical model), *an entity* corresponds to a table, an instance of *an entity* corresponds to a row in a table, and *an attribute* corresponds to a column of a table.

Building a *data model* involves defining *entities* and *attributes* , i.e. it is necessary to determine what information will be stored in a specific *entity* or *attribute* . *An entity* can be defined **as an object, event or concept, information about which should be stored** . *Entities* should have a name with a clear semantic meaning, be called a noun in the singular, not have "technical" names and be important enough to be modeled. Naming *an entity* in the singular makes it easier to read the model later. In fact, the name of *an entity* is given by the name of its instance. An example could be *the entity* Customer (but not Customers!) with *the attributes* Customer Number, Customer Last Name and Customer Address. At the level of the physical model, it could correspond to the Customer table with the columns Customer_number,

Customer_name and Customer_address. Each *entity* must be fully defined using a text description. User-defined properties (UDP) are used to add additional comments and definitions to the *entity*. The usage of (UDP) is similar to their usage in BPwin.

How was indicated higher, every *attribute* keeps information about **certain property entities**, A each instance *entities* must be unique. *Attribute* or group *attributes*, which identify *entity*, called **the primary key**.

It is very important to give the *attribute* the correct name. *Attributes* must be named in the singular and have a clear semantic meaning. Compliance with this rule allows you to partially solve the problem of data normalization already at the stage of defining *attributes*. For example, creating the Employee Phones *attribute in the Employee entity* contradicts the normalization requirements, since the *attribute* must be atomic, i.e., not contain multiple values. According to the syntax IDEFIX *attribute name* must be unique within the model (not just within the *entity*!). By default, if you try to enter an *attribute name that already exists*, ERwin renames it.

Every *attribute* must be defined, at this follows avoid cyclical definitions, For example, when the term 1 is determined through term 2, term 2 — through term 3, A term 3 V my queue — through term 1. It often happens create derivatives *attributes*, T. e. *attributes*, meaning which Can calculate from other *attributes*. An example of a derived *attribute* is the employee's Age, which can be calculated from *attribute* Date birth employee. Such *attribute* Maybe bring To conflicts; really, If the value of the Employee Age *attribute is not updated in time*, it may contradict the value of the Employee Date of Birth *attribute*. Derived *attributes* are a normalization error, but they are introduced to improve system performance, so as not to perform calculations that may be complex in practice.

Connections

A *connection* is a logical relationship between *entities*. Each *connection* must be named with a verb or verb phrase. phrase. *Name connections* expresses some limitation or business rule And makes it easier reading diagrams. By default *Name connections* on diagram Not is shown. On logical level Can establish an identifying one-to-many *relationship*, a many-to-many *relationship*, and a non-identifying one-to-many *relationship*.

IN IDEFIX distinguish dependent And independent *entities*. *Type entities* is determined her *connection* with other *entities*.

An identifying *relationship* is established between an independent (the parent end of the *relationship*) and a dependent (the child end of the *relationship*) *entity*. When an identifying *relationship is drawn*, ERwin automatically converts the child *entity* into a dependent entity. The dependent *entity* is depicted as a rectangle with rounded corners. An instance of a dependent *entity* is defined only through its relationship to the parent *entity*. When an identifying relationship is established, *connections attributes primary key* parental *entities* automatically are transferred in the composition *primary key subsidiary entities*. This operation additions *attributes subsidiary entities* when creating a *relationship is called attribute migration*. In the child *entity*, new *attributes* are marked as a foreign key - FK.

At establishment non-identifying *connections subsidiary essence* remains independent, but *attributes of the primary key parental entities* migrate V compound non-key components of the parent *entity*. A non-identifying *relationship* is used to link independent *entities*.

Identifying *connection* is shown on diagram solid line With fat point on child end of the *relationship*, non-identifying – dotted (see Fig. 10.6).

Power connections (Cardinality) — serves For designations relationship numbers copies parent *entity* to the number of instances of the child.

Distinguish four *type entities*:

- the general case when one instance of the parent *entity* corresponds to 0, 1, or many instances of the child *entity*; not marked with any symbol;
- is marked with the symbol P happening, when alone instance correspond to the parent *entity* 1 or many instances of the child *entity* (null value excluded);
- symbol Z is marked happening, When alone instance parental *entities* correspond 0 or 1 instance of the child *entity* (multiple values excluded);
- The number marks the case of an exact match, when one instance of the parent *entity* corresponds to a predetermined number of instances of the child *entity*.

Verb Phrase — a phrase that characterizes the relationship between parent and child **entities**. For a one-to-many *relationship*, identifying or non-identifying, it is sufficient to specify the name that characterizes the relationship from parent to child *entity* (Parent-to-Child). For a many-to-many *relationship*, both Parent-to-Child and Child-to-Parent names should be specified.

Types entities And hierarchy inheritance

As stated above, *relationships* determine whether an *entity* is independent or dependent. There are several **types of dependent entities**.

Characteristic - a dependent child *entity* that is associated with only one parent and, by meaning, stores information about the characteristics of the parent *entity* (Fig. 8).



Rice. 8- Example characteristic entities "Hobby"

Associative — *essence* , related With several parental *entities* . Such *essence* contains information about *entity relationships* .

The one who names — private happening associative *entities* , Not having own *attributes* (only *attributes* parental *entities* , migrated V quality external key).

Categorical — subsidiary *essence* V *hierarchies inheritance* .

An inheritance hierarchy (or category hierarchy) is a special type of grouping of *entities* that share common characteristics. For example, an organization has full-time employees (constant employees), And part-time workers. From their common properties Can form generalized *essence* (generic ancestor) Employee (rice. 10.8), to introduce information, general For all types of employees. Specific For each type information Maybe be located in the categorical *entities* (descendants) Permanent Employee and Part-Time Employee.

Usually *hierarchy inheritance* creates, When several *entities* have General by meaning *attributes* , either when *entities* have common semantic *relationships* (for example, if a Permanent Employee and a Part-Time Employee had a similar semantic *relationship* "works in" with the Organization *entity*), or when this is dictated by business rules.

For each category, you can specify a discriminator - *an attribute* of the generic ancestor that shows how to distinguish one categorical *entity* from another (the Type *attribute* in Fig. 9).



Rice. 9 - Hierarchy inheritance. Incomplete category

Hierarchies categories are sharing on two type — full And incomplete. IN full categories alone instance the generic ancestor (*entity* Employee, Fig. 10) necessarily corresponds to an instance in some descendant, i.e. in the example, the employee is necessarily either a part-time worker, or a consultant, or a permanent employee.



Rice. 10- Hierarchy inheritance. Complete category

If a category is not yet fully constructed and there may be instances in the generic ancestor that do not have corresponding instances in the descendants, then such a category will be incomplete. Figure 10.8 shows an incomplete

category — employee Maybe be Not only permanent or part-time worker, But And consultant, but the Consultant *entity* has not yet been entered into the *inheritance hierarchy* .

Keys

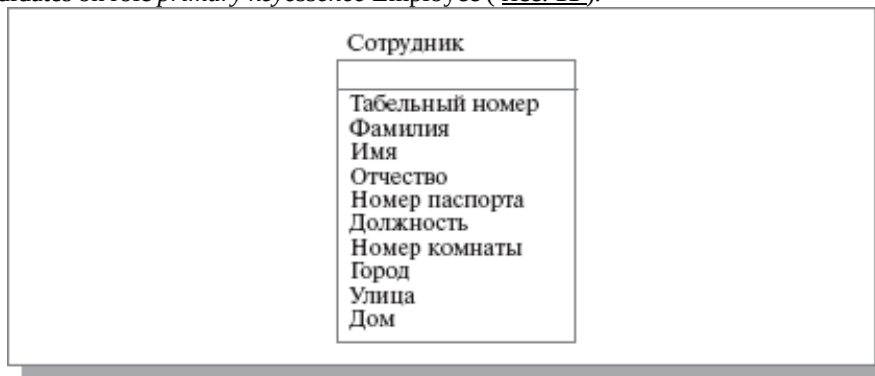
How was said higher, every instance *entities* must be unique And must differ from other *attributes* .

A **primary key** is an *attribute* or group of *attributes* that uniquely identifies an instance. *entities* . *attributes of the primary key* on diagram Not demand special designations — These are the *attributes* that are in the list of *attributes* above the horizontal line (see, for example, Fig. 10.9).

IN one *entities* can turn out to be some *attributes* or sets *attributes* , claimants for the role of the *primary key* . Such candidates are called candidate **keys** .

Keys can be complex, i.e. containing several *attributes* . Complex *primary keys* do not require special notation - they are a list of *attributes* located above the horizontal line.

Let's consider candidates on role *primary key* essence Employee (rice. 11).



Rice. 11- Definition primary key For entities "Employee" The following

potential keys can be identified here :

1. Time card number;
2. Number passports;
3. Last name + Name + Surname.

For Togo to become primary, *potential key* must satisfy row requirements:

Uniqueness . No two instances of a candidate key should have the same value. *candidate key* # 3 (Last name + Name + Patronymic) is a bad candidate, because in the organization can work full namesakes.

Compactness . A complex possible key should not contain any *attribute* , the removal of which would not lead to the loss of uniqueness. To ensure the uniqueness of key #3, we will supplement it with the *attributes* Date of birth and Hair color. If the business rules say that the combination of *attributes* Last name + First name + Middle name + Date of birth is sufficient for the unambiguous identification of an employee, then Hair color turns out to be redundant, i.e. the key Last name + First name + Middle name + Date of birth + Hair color is not compact.

When choosing a *primary key*, preference should be given to simpler keys, i.e. keys containing fewer *attributes* . In the example given, keys #1 and #2 are preferred over key #3.

attributes must not contain null values. The value of key *attributes* must not change during the entire lifetime of the *entity instance* . An employee of the organization can get married and change both her last name and passport. Therefore, keys #2 and #3 are not suitable for the role of the *primary key* .

Every *entity* must have at least one *candidate key* . Many *entities* have only one *candidate key* . This key becomes the primary key . Some *entities* may have more than one candidate key. Then one of them becomes the primary key and the others become *alternate keys* .

Alternative key (Alternate Key) — This *potential key* , Not became primary. Data normalization

Data normalization is the process of examining and reorganizing *entities* and *attributes* to meet requirements. To relational *models data* . Normalization allows be sure, What each *attribute* defined for its *essence* , and also significantly reduce the amount of memory for storing information and eliminate anomalies in the organization of data storage. As a result of normalization, a data structure should be created in which information about each fact is stored in only one place. The normalization process comes down to the consistent reduction of the data structure to normal forms - formalized requirements for data organization. Six normal forms are known.

In practice, it is usually enough to reduce data to the third normal form. For an in-depth study of normalization, the book by K. J. Date "Introduction to Database Systems" (Kiev; Moscow: Dialectics, 1998) is recommended.

ERwin does not contain a full normalization algorithm and cannot perform normalization automatically, but its capabilities make it easier to create a normalized *data model* . Prohibition on assigning non-unique names *attributes* V within the framework models (at corresponding installation options Unique Name) makes it easier compliance rules

"one fact - one place". The names of foreign key *attribute roles* and the *unification of attributes* also facilitate the construction of a normalized model.

Domains

Domain Can define How totality values, from which are taken meanings *attributes*. Each *attribute* can be defined on only one *domain*, but each *domain* can have multiple *attributes defined*. A *domain* includes not only the data type, but also the range of data values. For example, you can define the *domain "Age"* as a positive integer and define the *attribute Employee Age* as belonging to this *domain*.

In ERwin, a *domain* can be defined only once and used in both the logical and physical models.

Domains make it easier for developers to work with data at the design stage and for DB administrators at the system operation stage. At the logical level, *domains* can be described without specific physical properties. At the physical level, they automatically receive specific properties that can be changed manually. Thus, the *domain "Age"* can have the *Number* type at the logical level, and at the physical level, the *domain* columns will be assigned the *INTEGER* type.

Every *domain* Maybe be described, supplied comment or property, certain by user (UDP) Creating a Physical Data Model

The physical model contains all the information necessary to implement a specific DB. There are two levels of the physical model:

- transformational model;
- model DBMS.

The transformation model contains information for the implementation of a separate project, which can be part of a general IS and describe a subset of the subject area. This model allows designers and DB administrators to better imagine which DB objects are stored in the data dictionary and to check how the physical model meets the IS requirements.

The DBMS model is automatically generated from the transformation model and is an exact reflection of the DBMS system catalog.

The physical level of the model *representation* depends on the selected server. ERwin supports more than 20 relational and non-relational databases.

By default ERwin generates names tables And *indices* By template on basis names of the corresponding *entities* and keys of the logical model, which can later be manually adjusted. Names tables And columns will be generated By default on based on the names of *entities* and *attributes* of the logical model.

Rules validations And meanings By default

ERwin supports *rules validations* For columns, A Also meaning, assigned columns By default. *Rule validations* asks list acceptable values For specific columns and/or rules validation checks values. IN list acceptable values Can bring in new values. ERwin allows generate *rules validations* respectively syntax selected DBMS With taking into account borders range or list values.

Default value - the value to be entered into the column if no other value is explicitly specified. way in time input data. WITH each column or *domain* Can tie up meaning By default. The list of values can be edited.

Once you have created a *validation rule* and default value, you can assign them to one or more columns or *domains*.

Indexes

IN BD data is usually stored V volume in order, V which their entered V table. Many relational DBMS have a page organization, at which table Maybe be stored fragmentarily V different areas disk, moreover The table rows are arranged randomly on the pages. This method allows you to quickly enter new data, but makes it difficult to find data.

To solve the search problem, DBMSs use objects called *indexes*. **An index** contains information sorted by a column or several columns and points to the rows that store a specific column value. Since the values in the *index* are stored in a specific order, when searching, you need to look through need to significantly less volume data, What significantly reduces time query execution. It is recommended to create an *index* for those columns that are frequently searched.

When generating a physical database schema, ERwin automatically creates an *index* based on the *primary key* of each table, as well as on all *alternate keys* and foreign keys, since these are the columns most frequently used. For search data. Can refuse from generations *indices* By default And create custom *indexes*. To increase the efficiency of search, the database administrator should analyze frequently executed queries and, based on the analysis, create custom *indexes*.

Triggers And stored procedures

Triggers And stored procedures are named SQL code blocks, which are pre-compiled and are stored on server For Togo, to fast produce processing requests, validation data And other frequently performed functions. Storage And execution code on server allows create code only one once, A Not V

each application that works with the database. This saves time when writing and maintaining programs. It also ensures that data integrity and business rules are maintained regardless of which client application accesses the data. *Triggers* and *stored procedures* do not need to be sent over the network from the client application, which significantly reduces network traffic.

A **stored procedure** is a named set of precompiled SQL commands that can be called from a client application or from another *stored procedure*.

Trigger it's called procedure, which is being carried out automatically How reaction on event. Like this event can be insert, change or removal lines V existing table. *Trigger* reports DBMS, which actions must be performed when executing SQL INSERT, UPDATE, or DELETE commands to provide additional functionality running on the server.

referential integrity trigger is a special type of *trigger* used to maintain integrity between two tables, which are related between yourself. If line in one table is inserted, is modified or deleted, the referential integrity *trigger* tells the DBMS what to do with those rows in other tables whose foreign key value matches the *primary key value* of the inserted row (modified or deleted row).

To generate *triggers*, ERwin uses a template mechanism – special scripts that use macro commands. At generations code *trigger* instead of macro commands are substituted names tables, columns, variables and other code fragments that match the syntax of the selected DBMS. The referential integrity *trigger templates* generated by ERwin by default can be modified.

ERwin provides special editors for creating and editing *stored procedures*, similar to those used to create *triggers*. Unlike a *trigger*, a *stored procedure* is not executed in response to an event, but is called from another program that passes the procedure name to the server. A *stored procedure* is more flexible than a *trigger* because it can call other *stored procedures*. It can be passed parameters, and it can return parameters, values, and messages.

Design storages data

Data warehouses store data that rarely changes. Warehouses are designed to perform analytical queries that provide decision support for executives and managers. When designing data warehouses, the following requirements must be met:

- storage should have understandable For users structure data;
- should be highlighted static data, which are being modified By schedule (daily, weekly, quarterly);
- should be simplified requirements To requests For exceptions requests, requiring multiple SQL statements in traditional relational DBMSs;
- should be provided with support complex requests SQL, requiring processing millions records. Calculating the size of the database

ERwin allows you to calculate the approximate size of the database as a whole, as well as tables, *indexes* and other objects through definite period time after beginning exploitation IS. Calculation under construction on basis following parameters: initial number of lines; maximum number of lines; increase in the number of lines per month. The calculation results are summarized in a report.

ERwin supports not only the design of the database server, but also the automatic generation of the client applications in the MS Visual Basic and Power Builder development environments. The generation technology consists of the fact that at the stage of developing the *physical data model*, each column is assigned extended *attributes* containing information about the properties of the client application objects (including visual ones) that will display the information stored in the corresponding column. This information is recorded in the model file. Based on the information, contained in extended *attributes*, are generated on-screen forms. Received code Maybe be compiled and executed without additional manual coding.

Each one column V models ERwin Can ask preliminary described And named properties:

- *rules validations* (examination values);
- initial values, installed By default;
- style visual object (For example, radio button, field input And etc.);
- format images.

For descriptions each properties ERwin contains relevant editors. Generating Code in

Visual Basic

ERwin supports code generation in Visual Basic versions 4.0 and 5.0. The ERwin model serves as a source of information when generating forms. With ERwin, you can simultaneously describe both the client part (objects displaying data on the screen) and the DB server (procedures and *triggers*), thereby optimally distributing the functionality of the information system between the client and server parts. The ERwin Form Wizard component automatically designs forms with child objects - buttons, lists, fields, radio buttons, etc., using extended *attributes*.

Creation reports

To generate reports, ERwin has a simple and effective tool – Report Browser. By default, Report Browser contains predefined reports that allow you to visually present information about the main objects *models data* - How logical, So And physical. WITH with help special editors

existing reports can be modified or you can create your own report. Each report can be customized individually, the data in it can be sorted and filtered. Browser Report allows you to save the results of running reports, print them and export them to common formats.

Generation dictionaries

For managing large projects, ERwin has a special tool – ERwin Dictionary, which provides collective work over diagrams And allows save And document various versions of *data models* . ERwin Dictionary is a special DB that allows solving the problems of documenting and storing models, but does not fully meet the requirements of multi-user work.

Lecture 10. Model Mart - system management models For group developments

Model Mart - model management system for group development when creating applications for the client-server architecture, data warehouses, Web. Provides multi-user access to models created with ERwin and BPwin. Models are stored on a central server and are available to all team members design, at this is provided opportunity collective creations complex And volumetric models.

Main peculiarities ModelMart:

- supported some versions models, participants team designers have access To the latest version of the model and can see the results of the changes made;
- are used flexible means distinctions access;
- user Maybe get list differences between two versions models And, at necessity, revert to previous version;
- is produced analysis influences contributed V model changes;
- supported opportunity mergers several models V one;
- is provided flexible blocking access To models;
- special means intellectual permissions conflicts automatically identifies any conflicts that arise when the same model is modified simultaneously by multiple users.

ModelMart is built according to the rules of the client-server architecture: it is installed on the server in the DBMS environment, as which V present time can perform Sybase, M.S. SQL Server, Oracle , Informix. IN quality client ERwin/ERX for ModelMart, ERwin/Navigator for ModelMart versions 2.6 and higher, and BPwin versions 2.01 and higher can be used.

Models in ModelMart are stored in user-defined libraries. All models stored in a single library share a set of objects: domains, triggers, templates for them, validation rules, etc. physical properties. When creation in this library new models, she automatically accesses these shared objects. When changes are made to such an object, these changes are reflected in all models of the library, ensuring the consistency of the project.

ModelMart supports simultaneous work With models data several users. Before When you start working with a model, ModelMart saves its initial state, which will then be used to compare changes made by a particular user with changes made by others.

When accessing a model simultaneously, the Intelligent Conflict Resolution component automatically identifies any conflicts that may arise when trying modify the model at the same time.

Supported opportunity keep some versions one And that same models.Early version models You can compare it with a later one, see all the differences and, if necessary, return to the older version.

A model merger is supported. Independent data models can be intelligently merged into a new large model. Designers can interactively view all changes made to the model and assess their potential impact. A special component, the "change management manager", allows you to accept or reject each change. Opportunity produce "rollback" By relation To specific changes allows the designer to explore his model in a "what if" manner. ModelMart also records which user is responsible for a particular change and when they made it.

ModelMart includes a special utility - ModelMart Synchronizer, which allows synchronization of process models (BPwin) and data (ERwin) stored in ModelMart libraries.

Distinction access is carried out on basis special profiles access. To everyone to the user predefined, built-in ModelMart, and custom profiles customized for a specific situation can be assigned. The changes a specific user can make can be limited to the library, diagram, or subject areas V within diagrams. This gives opportunity strictly to delimit responsibility of team members.

ModelMart is a team development environment for large projects that integrates toolkits means systemic analysts And developers BD. ModelMart implemented on architecture client-server. The relational DBMS Sybase, Microsoft SQL Server, Informix and Oracle can be used as a platform for implementing the storage. The client applications are ERwin 3.x and BPwin 2.x. When installing, you should pay attention to the correspondence of the versions of ModelMart, ERwin and BPwin. A certain version of ModelMart works with quite certain versions ERwin And BPwin - compatibility With previous versions Not exists. So, ModelMart version 3.02, which will be discussed below, is compatible with ERwin 3.52 and BPwin 2.5.

Installation ModelMart Maybe call difficulties at unprepared user, That's why at installation, you must strictly follow the instructions set out in the documentation ("Administrator's Guide"). As an example, let's consider the installation of ModelMart on the Oracle DBMS.

Before installation ModelMart necessary make sure, What installed And Right are functioning server and communication parts of the Oracle DBMS. To install ModelMart you need:

- 50 M disk spaces And 30 M RAM on server;
- minimum 15 M disk spaces And 16 M RAM (recommended) 40 M And 32 M) on working stations.

Next, you need to create physical memory objects for ModelMart in Oracle. This work is most conveniently done using ERwin (see Chapter 2.3.6). The following preparatory operations are necessary:

- check the system table space (SYSTEM TABLESPACE). It should be at least 16 M, 32 M is recommended;
- create tablespace for tables And indices ModelMart. Recommended highlight Not less than 50 M for tables and 50 M for indexes. To create a table space, you can use the ORACLE Physical Object Editor dialog (in ERwin, Server/Physical Object menu), Fig. 12;

- create a role. You can create a role and grant it privileges using the SQL*Plus utility by executing the CREATE ROLE MMUSER and GRANT CREATE SESSION TO MMUSER commands.

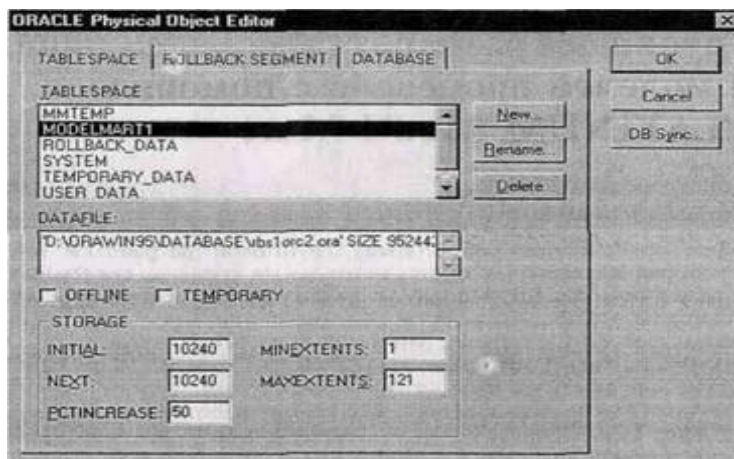


Fig. 12 - Creating a tablespace for Mode/Mart in the ORACLE Physical Object Editor dialog After this Can launch program installations. IN the process installations necessary specify serial number And number licenses ModelMart. After completion installations is starting program initialization ModelMart, which creates the necessary objects in the DBMS system directory. During the initialization process, the ModelMart Connection Manager dialog appears, in which you should specify the user name and password. Then, in the ModelMart Manager dialog, you should specify the table space for tables and indexes (Fig. 13).

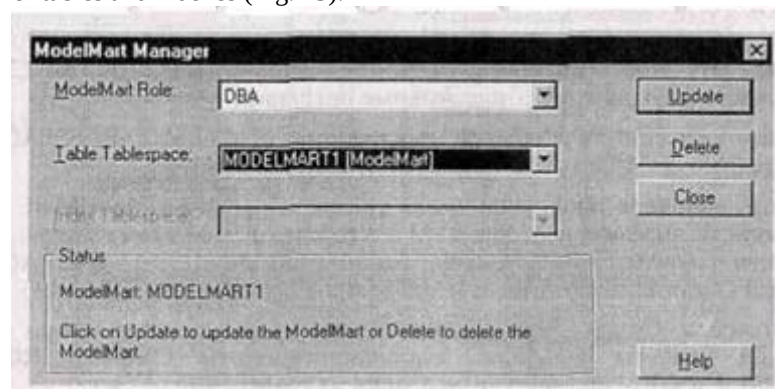


Fig.13- Dialogue ModelMart Manager

After endings process initialization Can to proceed To creation users ModelMart.

Lecture 11. Unified language modeling UML. Main elements object-oriented approach

Methods of object-oriented approach (OOP) and object-oriented automated approach (OOAP) are interconnected Friend With friend through language UML. Diagrams method OOP, compiled from UML language icons describe and document the object codes of software tools of information systems, and the OOAP method diagrams, compiled in the UML language, are used to describe and design the information systems themselves. It can be said that the UML language is a general technological tool for methods OOP And OOAP, With with help whom systemic analysts And programmers create project documentation of information systems and their software.

Conceptual basis **object-oriented approaches** is **object model**. The main elements are:

- *Abstraction (abstraction)*
- *Encapsulation (encapsulation)*
- *Modularity (modularity)*
- *Hierarchy (hierarchy)*

Except main there are more three additional element, Not being strictly mandatory:

- *Typification (typing)*
- *Parallelism (concurrency)*
- *Sustainability (persistence)*

Abstraction is the identification of the essential characteristics of an object that distinguish it from all other types of objects and thus clearly define its conceptual boundaries for further consideration and analysis. Abstraction focuses attention on the external features of an object and allows one to establish the most essential features of its behavior from the details of their implementation. Choosing the Right Set abstractions For given subject areas And represents by yourself home task object-oriented design.

Encapsulation is the process of separating individual elements of an object that determine its structure and behavior. Encapsulation serves to isolate the object's interface, which reflects its external behavior, from the internal implementation of the object. The object approach assumes that the object's own resources, which can only be manipulated by the methods of the class itself, are hidden from the external environment. Abstraction and encapsulation are complementary operations: abstraction focuses attention on the external features of an object, and encapsulation does not allow user objects to distinguish the internal structure of the object.

Modularity is property systems, related With opportunity her decomposition on row internally related, but loosely coupled modules. Encapsulation and modularity create a barrier between abstractions.

Hierarchy - This ranked or ordered system abstractions, location their By levels. The main types of hierarchical structures as applied to complex systems are class structure (hierarchy by nomenclature) and object structure (hierarchy by composition). Examples of class hierarchy are simple and multiple inheritance (one Class uses structural or functional Part one or several classes respectively), and the hierarchy of objects is aggregation.

Typing is a constraint imposed on a class of objects that prevents different classes from being interchangeable. Typing allows defend oneself from use objects of one class instead of another, or at least control such use.

Sustainability - properties object exist in time (outside dependencies from process, gave birth to given object) and/or in space (when moving an object from the address space in which it was created).

The main ones concepts object-oriented approaches are object And Class.

Object - will be determined How tangible reality (tangible entity) -item or phenomenon, having clearly defined behavior.

Class - This set objects, related community structures And behavior. Any object is an instance of a class.

Definition objects And classes - one from the most complex tasks object-oriented design.

Lecture 12. UML diagrams. Description requirements V context models precedents

WITH with help combinations pictograms are being built UML diagrams. Nine main species diagrams listed in the third branch of the UML structural tree.

IN language UML is used four main species graphic designs:

- Icons or pictograms. Icon represents by yourself graphic figure fixed size And forms.
- Graphic symbols on a plane. Such two-dimensional symbols are depicted using some geometric figures and can have different heights and widths in order to accommodate other UML constructs inside these figures.
- Paths, which are sequences of line segments connecting individual graphic symbols.
- Lines text. They serve For performances various species information V some grammatical form.

Description requirements V context models precedents

Idea descriptions functional requirements V view precedents was formulated V 1986 G. Ivar Jacobson (Ivar Jacobson) – one of the developers of UML.

Precedents - This mechanism simplifications stages formulations requirements For all interested persons. Essentially, these are stories about using the system in the process of solving the tasks. Here is an example of a short description of a use case.

Example. Processing sales Buyer fits To at the box office With selected goods. Cashier With with help POS systems register each product. The system displays information about each product name and calculates

total amount. The buyer enters the required information, the system verifies and registers it. The system performs an inventory. The buyer receives a sales receipt and leaves the store with purchases.

Often precedents need to think through much more carefully. But main idea consists of V research and formulating functional requirements by writing stories "from the life of the system." These stories help formulate various tasks And represent by yourself scenarios use systems. On first sight, Describing precedents is not difficult, although it is often difficult to determine what is required from the system and describe it at the required level of detail.

The power of the precedent mechanism lies in the ability to scale the level of complexity and formality of the description depending on real needs.

Options use And active faces

Active faces (actors - Actor)

Current face - This essence, possessing behavior. This role, which user plays in relation to To system. The characters represent by yourself roles, and not specific people or names works. K number active persons can relate and herself under consideration system, If she calls services others systems.

DL are Not only People, But And organizations, cars And programs. Exists three type external in relation to the DL system being developed:

Types active persons (actors):

- **Main current face** (primary actor) -his tasks are being carried out With using systems.
- **Auxiliary acting face (supporting actor)** - serves system, For example, provides information.
- **Backstage executor (offstage actor)** - interested V implementations option usea, But Not is a primary or secondary actor. For example, the tax service.

Active faces can:

- Only supply information system
- Only receive information from systems
- To supply information And receive information from systems.

Usually DL are determined from descriptions tasks or by way negotiations With customers And experts. For detection DL The following group of questions can be used:

- Who interested V certain systemic requirement?
- Which one? role system will fulfill V organizations?
- Who will receive advantages from use systems?
- Who will supply system information, use information And receive information from systems?
- Who will realize support And service systems?
- Performs li any participant systems V several roles?
- Performing li various participants V one roles?
- Will li new system interact with old?

Example . IN system registration courses Can highlight following active persons: Student, Professor, Registrar, Payment system, Course catalog.

Description active persons.

IN model desirable turn on brief description each DL, V which need to specify role DL when interacting with the system. For example,

- Student -This Human, which is registered on courses For visits classes V university
- Professor - Human, which reads lectures V university
- Registrar -Human, control system registration on courses
- Estimated system - - external system, performing functions calculations for courses.
- Catalog courses -external system, containing information O courses

Options use (precedents)

The behavior of a system is its reaction to external events. In the UML, externally observable and testable behavior is captured in the form of precedents or use cases. A use case or precedent is a use case. The behavior of the system being developed (i.e., the functionality provided by the system) is described using a functional model that displays the system precedents or options use (use case) , systemic environment (current persons or actors

- actors) And connections between options use And actors (diagrams Options usage -use case Use Case – Performs a business function that can be observed by an external entity and can be subsequently separately tested V in the process developments. Option use - This mechanism simplification of the stage formulations requirements For all interested persons. By being This stories about use systems in the process of solving assigned tasks.

Options use - This requirements. IN basically, This functional requirements, indicating on what the system should do. Use case descriptions are text documents, not diagrams. Use case modeling is a writing process, not a drawing process.

The main objective of the Use Case model - to be a single tool that allows the customer, end user and developer to discuss the functionality and behavior of the system together. The basic idea is to explore and formulate functional requirements by writing stories "from the life of the system". The development of a use case model begins at the concept stage with the selection of actors and the definition of general principles of the system's operation. At first glance, it is not difficult to describe a use case, although it is often difficult to determine what is required from the system and describe it at the required level of detail. Then on at the development stage model is being supplemented detailed information to existing use cases, and if necessary, new ones are created.

Algorithm discharge options usage:

1. highlight tasks (goals) user
2. Define For each from them separate option use

The use case name should match the task name, for example, the checkout task should have the use case **Checkout Sales**.

Degree formalization

IN dependencies from needs stand out some

Degrees formalization descriptions options usage:

- *Compressed* - annotations V view one paragraph. Usually she describes main successful scenario
- *Free* - informal style descriptions. Description options use takes up some paragraphs and covers various scenarios
- *Expanded* - most detailed style descriptions. At such approach are described All steps And scenario development options, as well as preconditions and results.

Options use describe, What will do yours system. More specifically details are determined in a document called *the Flow of Events*. It typically contains:

Expanded description precedent (options usage):

- Brief description
- Preconditions
- Basic flow events
- Alternative flow events
- Postconditions Let's look

at these main parts **Brief**

description (Explanations).

Introductory elements.

IN description options use Can add various introductory elements. Those from them, which are important for the presentation subsequent script, follows put V start descriptions. For example, V description Can add the following introductory element:

Preconditions And postconditions

Preconditions - This scroll prerequisites, which *Always should to be carried out* to beginning scenario use cases. *Preconditions Not* are being checked at implementations options use. This conditions, which are considered true. Usually, a precondition is a successful outcome of another scenario. **Preconditions** - these are the ones prerequisites, on which variant developer use wants turn special attention.

Results or postconditions

Describes what conditions must be met in order for the scenario to be successful. These results must satisfy the needs of all stakeholders.

Basic flow events or Basic scenario

This section can also be called the core process. It describes the typical sequence of actions that lead to the successful completion of the scenario and meets the needs of all stakeholders.

Basic flow events (Basic scenario)

Described next types actions:

1. Interaction between performers
2. Verification (usually with sides systems)
3. Change states systems

IN section main scenario are described three species actions:

Alternative streams events (Extensions)

Extensions -This branches from main scenario. IN this section are indicated All rest scenarios or branches, leading to success or unsuccessful completion options use. Description The extension consists of two parts: the condition and the method of its processing.

When the alternate flow is finished processing, the default behavior is to return to the main scenario unless the alternate flow provides for a different course of events.

Diagrams options use

A use case diagram is a graphical representation of all or part of the actors, use cases, and their interactions in a system.

Every system has a main use case diagram that shows the boundaries of the system (actors) and the main functional behavior of the system (use cases). Other use case diagrams can be created as needed.

Use case diagrams and their relationships are of secondary importance when working on use cases. Use cases are text documents. Working on use cases means writing text descriptions. In this perspective, the diagram only illustrates the ways in which external users use the system.

Lecture 13. Classes. Class diagram. Modeling interactions between objects. The central place in OOP is occupied by the development of a logical model of the system in the form of a class diagram. Diagram classes V terminology UML it's called diagram, on which shown kit *classes*, *interfaces*, *cooperatives*, A Also connections (relationships) between them.

Diagrams classes contain:

Classes Interfaces

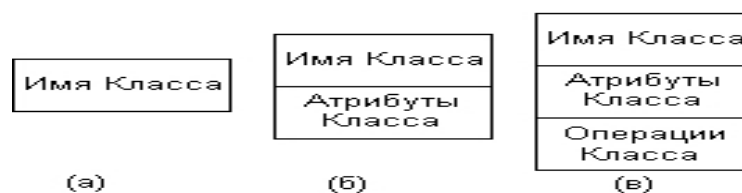
Cooperation

Relationship dependencies, generalizations And associations

Usually diagrams classes are used V following purposes:

1. For modeling dictionary systems. Modeling dictionary systems suggests acceptance solutions O which abstractions are part of the system and which are not. Class diagrams can be used to identify these abstractions and their responsibilities.
2. For modeling simple collaborations. A collaboration is a collection of classes, interfaces, and other elements that work together to provide some cooperative behavior that is greater than the sum of its parts. For example, when modeling transaction semantics, you cannot understand the processes that occur by looking at a single class, because the semantics are provided by several classes working together.
3. To model the logical scheme of the database. The logical scheme can be thought of as a drawing of the conceptual design of the database.

Diagram classes (class diagram) serves For performances static structures models systems. Class diagram Maybe reflect, in in particular, various relationships between separate entities subject areas, , and also describes their internal structure and types of relationships. From this point of view, the class diagram is a further development of the conceptual model of the designed system.



Rice. 14- Graphic image class on diagram classes

A class in UML is used to designate a set of objects that have the same structure, behavior, and relationships with objects from other classes. Graphically, a class is depicted as a rectangle, which can be additionally divided by horizontal lines into sections (Fig. 14). These sections can indicate the class name, attributes (variables), and operations (methods).



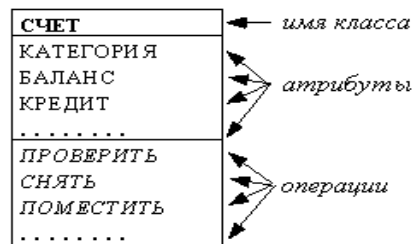
Fig.15- Examples graphic images classes on diagram

Two apples belong to the same class of objects (this is what makes them identical). The color, shape, weight, and taste of an apple are its attributes: the set of attributes and their values (e.g. red, oval, hundred-gram, sweet and sour) characterizes the object.

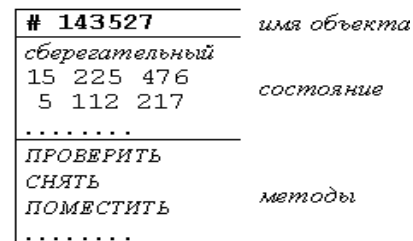
All objects are the same same class are characterized by the same sets of attributes. However, the union of objects V classes is determined Not sets attributes, A semantics. So, For example, objects stable And horse

can have identical attributes: price And age. At this They can treat To alone class, if they are considered in the problem simply as goods, or to different classes, which is more natural.

Пример класса



Объект класса СЧЕТ



Rice. 16- Example class And object this class

Combining objects into classes allows us to introduce abstraction into the problem and consider it in a more general setting. Class has Name (For example horse), which refers to to to all objects this class. Except Togo, V class, the names of attributes that are defined for objects are entered. In this sense, the description of a class is similar to the description of a structure (record) type; in this case, each object has the same meaning as an instance of the structure (variable or constant of the corresponding type). An example of a class and an object of this class is shown in Figure 16.

A mandatory element of class designation is its name. At the initial stages of diagram development, individual classes can be designated simply rectangle with indication only name the corresponding class (Fig. 16, a). As individual components of the diagram are developed, class descriptions are supplemented with attributes (Fig. 16, b) and operations (Fig. 16, c).

Even if the attributes and operations section is empty, it is highlighted with a horizontal line in the class notation to immediately distinguish the class from other UML elements. Graphical representation examples classes on diagram classes are given on rice. 16 . IN first case For class "Rectangle" (rice. 16 , a) only its attributes are indicated - points on the coordinate plane that determine its location. For the "Window" class (Fig. 16, b) only its operations are indicated, the attributes section is left empty. For the class "Account" (Fig. 16, c) additionally shows a fourth section, which indicates an exception - refusal to process an expired credit card.

Name class

The class name is written in the center of the name section in bold and must begin with a capital letter. Examples of class names include nouns such as "Employee", "Company", directly related to the modeled subject area and the functional purpose of the designed system.

A class may not have instances or objects. In this case, it is called an abstract class, and its name is written in italic. The general convention in UML is that any text that refers to an abstract element is written in italic.

Attributes class

Attribute -property class.

In the second above sections of the class rectangle are written its attributes or properties. Each class attribute corresponds to a separate line of text, which consists of the attribute visibility quantifier, the attribute name, its multiplicity, the attribute value type, and possibly its original value:

```
<quantifier visibility><name attribute>[multiplicity]:
<type attribute> = <original value>{string- property}
```

Attributes are contained within a class, so they are hidden from other classes. Because of this, it is sometimes necessary to specify which classes have the right to read and change attributes. This property is called attribute visibility. The visibility quantifier can take one of three possible values and is therefore displayed using special symbols:

- **Public (general, open)** - This meaning visibility suggests, What attribute will visible by all other classes. Any class can view or change the value of the attribute. According to with notation UML The general attribute is preceded by a "+" sign .

- **Private (closed, secret).** Corresponding attribute Not visible none to others classes. For To view and edit such attributes, other classes must ask the class that contains it, which is usually done using public operations. A private attribute is denoted by a "-" sign in accordance with UML notation.

- **Protected (protected)** This attribute is available only to the class itself and its descendants. In UML notation it is denoted by the "#" sign.

Notations UML For attributes:

"- » - Private (closed, secret)

« + » - **Public (general, open)**

« # » - **Protected**

In general, it is recommended to make attributes closed or protected. This allows better control over the attribute itself and the code, as well as to avoid a situation where the attribute value is changed by all classes of the system. The visibility parameters you set affect the generated code.

Quantifier visibility Maybe be omitted. IN this case his absence Just means, What visibility attribute is not specified. This situation differs from the default conventions in traditional programming languages, where the absence of a visibility quantifier is interpreted as public or private. However, instead of the conventional graphic notations, you can write the corresponding keyword: public, protected, private.

An attribute name is a text string that is used as an identifier for the corresponding attribute and must therefore be unique within a given class. The attribute name is the only required element of the attribute's syntactic notation.

The multiplicity of an attribute characterizes the total number of specific attributes of a given type that are part of a separate class. In general, the multiplicity is written as a text string in square brackets after the name of the corresponding attribute:

[lower_bound1 .. upper_bound1, lower_bound2 .. upper_bound2, ..., lower_boundak .. upper_boundak],

where *lower_bound* and *upper_bound* are positive integers, each pair of which serves to denote a separate closed interval of integers whose lower (upper) bound is equal to the value *lower_bound* (*upper_bound*). In general, this conventional notation of multiplicity corresponds to the set-theoretic union of the corresponding intervals. The special symbol "*" can be used as the upper_bound, which denotes an arbitrary positive integer. In other words, this denotes an unlimited value of the multiplicity of the corresponding attribute.

IN quality example let's consider next options tasks multiplicities attributes.

[0..1] means, What multiplicity attribute Maybe accept meaning ABOUT or 1. At this 0 means no value for this attribute.

[0..*] means that attribute multiplicity can take any positive integer value greater than or equal to 0. This multiplicity can be written more briefly as a simple symbol - [*].

[1..*] means that the attribute multiplicity can take any positive integer value greater than or equal to 1.

[1..5] means, What multiplicity attribute Maybe accept any meaning from numbers: 1, 2, 3, 4, 5.

[1..3,5,7] means, What multiplicity attribute Maybe accept any meaning from numbers: 1, 2, 3, 5, 7.

[1..3,7.. 10] means that the attribute multiplicity can take any value from the numbers: 1, 2, 3, 7, 8, 9, 10. [1..3,7..*]

means, What multiplicity attribute Maybe accept any meaning from numbers: 1, 2, 3, A Also any positive whole meaning more or equal 7.

If multiplicity attribute Not indicated, That By default accepted her meaning equal 1..1, T. e. V accuracy 1.

Operation

The third section of the rectangle from the top contains the operations or methods of the class. An operation is a service that each instance of the class provides on a specific request. The collection operations characterizes functional aspect behavior class. Recording operations class V language UML is also standardized and follows certain syntactic rules. Each class operation has a separate string, which consists of the visibility quantifier of the operation, the name of the operation, an expression of the type of the value returned by the operation, and, possibly, a string property of the operation:

<quantifier visibility><name operations>(list parameters):

<expression type returned values>{string- property}

The visibility quantifier, as in the case of class attributes, can take one of three possible values and, accordingly, is displayed using a special symbol.

Symbol "+" means operation With area visibility type public (public).

Symbol "#" means operation With area visibility type protected (protected).

AND, finally, **the symbol "-"** is used For designations operations With area visibility type closed (private). Quantifier visibility For operations Maybe be omitted. IN this case his absence Just means, what is visibility operations Not is indicated. Instead of conditional graphic designations Also Can record corresponding key word: public, protected, private.

The operation name is a text string that is used as an identifier for the corresponding operation and must therefore be unique within the class. The attribute name is the only required element of the syntactic designation of an operation.

List parameters is list separated comma formal parameters, every from which can be represented as follows:

<view parameter> <name parameter>: <expression type> = <value parameter By default>.

Here view parameter - There is one from key words **in**, out or **inout** with meaning in By by default, V case if the parameter type is not specified. The parameter name is the identifier of the corresponding formal parameter. The expression type is dependent from specific language programming specification type returned

values for the corresponding formal parameter. Finally, the default value is generally by yourself expression For meanings formal parameter, syntax whom depends from specific programming language and is subject to the restrictions adopted in it.

IN quality examples records operations Can bring next designations separate operations:

+create() - Maybe designate abstract operation By creation separate object class, which is public and does not contain formal parameters. This operation does not return any value after its execution.

+draw() shape: Polygon = rectangle, fill_color: Color = (0, 0, 255)) - can denote an operation to draw a rectangular area of blue color on the monitor screen, if no other values are specified as arguments to this operation.

request_customer_account() account_number: Integer): Currency - denotes an operation to establish the availability of funds on current account client jar. At this argument given operations is number bills client, which is written as an integer (e.g., "123456"). The result of this operation is a certain number written in the accepted monetary format (e.g., \$1,500.00).

issue_message(): {"Division by zero error"} - the meaning of this operation does not require explanation, since it is contained in the operation property string. This message may appear on the monitor screen in the event of an attempt to divide a number by zero, which is unacceptable.

In addition to the internal structure or class structure, the corresponding diagram shows various relationships between classes. The basic relationships or connections in the UML language are:

Attitude dependencies (dependencyrelationship) Association

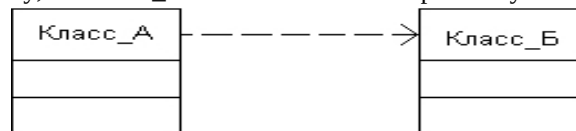
relationship Generalization relationship (generalization

relationship) Realization relationship

Every from these relations has own graphic performance on diagram, which reflects the relationships between objects of the corresponding classes.

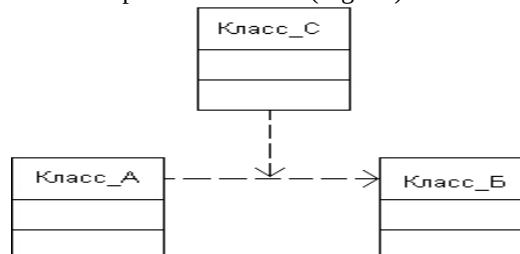
Attitude dependencies

Attitude dependencies graphically is depicted dotted line between corresponding elements with an arrow at one of its ends ("->" or "<-"). In a class diagram, this relationship links individual classes between by yourself, at this arrow directed from client class dependencies To independent class or source class (Fig. 17 This figure shows two classes: Class_A and Class_B, where Class_B is the source of some dependency, and Class_A is the client of this dependency.



Rice. 17- Graphic image relationship dependencies on diagram classes

IN quality client class And source class dependencies can perform whole sets elements models. In this case, a single line with an arrowhead coming from the dependency source splits at some point into several separate lines, each of which has a separate arrowhead for the client class. For example, if the operation of Class_C depends from features implementations of Class_A and Class B, then this dependence can be depicted as follows (Fig. 18).



Rice. 18- Graphic performance dependencies between client class (Class_C) And source classes (Class_Ai Class_B)

An arrow can be marked with an optional but standard keyword in quotation marks and an optional individual name. There are predefined keywords for the dependency relationship that denote some special kinds of dependencies. These keywords (stereotypes) are written in quotation marks next to the arrow, which corresponds given dependencies. Examples stereotypes For relationship dependencies are presented below:

"access" - serves to indicate the availability of public attributes and operations of the source class for client classes;

"bind" - client class Maybe use some sample For his subsequent parameterization;

"derive" - attributes client class can be calculated By attributes source class ;

"import" - open attributes and the operations of the source class become part client class as if if they were declared directly in it;

"refine" - indicates that the client class serves as a refinement of the source class for historical reasons when additional information becomes available during the course of the project.

Attitude associations

Most simple happening of this relationship - binary association. She connects V accuracy two class And, as an exception, Maybe tie up Class With ourselves by yourself. For binary associations on diagram Maybe be specified order of succession classes With using triangle V form arrows near With name given associations. The direction of this arrow indicates the order of the classes, one of which is first (from the side of the triangle), and the other is second (from the side of the triangle's apex). The absence of this arrow next to the association name means that the order of the classes in the relationship in question is not defined.

For of this relationship defined order following classes, first from which is Class "Employee", a second - Class "Company". Separate example or instance of this relationship Maybe to be pair meanings (Meder, "MEGA"). This means that employee Meder works for the company "MEGA".



Rice. 18. Graphic image relationship binary associations between classes

Ternary association and associations of higher arity are generally called N-ary association (read - "en-ary association"). Such an association connects 3 or more classes with some relation, and one class can participate in the association more than once. The association class has a certain role in the corresponding relation, which can be explicitly indicated in the diagram. Each instance of N-ary association is an N-ary tuple of object values from the corresponding classes. Binary association is a special case of N-ary association, when the value of N=2, and has its own designation.

An N-ary association is graphically represented by a rhombus, from which lines lead to the symbols of the classes of the given association. In this case, the rhombus is connected to the symbols of the corresponding classes by solid lines. Usually the lines are being carried out from peaks rhombus or from middle his sides. Name N-ary associations is being recorded near With diamond of the corresponding association.



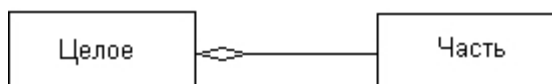
Rice. 19 Graphic image ternary associations between three classes

Attitude aggregations

Attitude aggregations has place between several classes V volume case, If one from classes represents a certain entity that includes other entities as constituent parts.

This relationship is of fundamental importance for describing the structure of complex systems, since it is used to represent systemic relationships of the "part-whole" type. By revealing the internal structure of the system, attitude aggregations shows, from what kind components consists of system And How They connected between by yourself. This is the attitude By his essence describes decomposition or partition complex systems on more simple composite parts that can also be decomposed if this becomes necessary later.

Graphically, the aggregation relationship is represented by a solid line, one end of which is an unshaded line. inside a rhombus. This the diamond indicates that from the classes that represent by yourself "whole". The rest of the classes are its "parts"



Rice. 20 Graphic image relationship aggregations V language UML

Another example of the aggregation relationship is the division of personal information, which is known to every reader. computer on composite parts: systemic block, monitor, keyboard And mouse. Using UML language notations, the component composition of the PC can be represented in the form of a corresponding class diagram, which in this case illustrates the aggregation relationship.

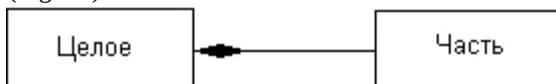


Rice. 21- Diagram classes For illustrations relationship aggregations on example PC

Attitude compositions.

The composition relation, as mentioned earlier, is a special case of the aggregation relation. This relation serves to highlight a special form of the "part-whole" relation, in which the constituent parts in some meaning are located inside whole. Specificity interrelations between them consists of V volume, What parts Not can act in isolation from the whole, i.e. with the destruction of the whole, all its constituent parts are also destroyed.

This rhombus indicates the class that represents by yourself class-composition or "whole". The remaining classes are its "parts" (Fig. 21).

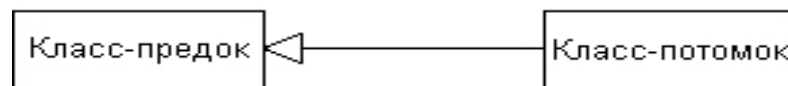


Rice. 21- Graphic image relationship compositions V language UML

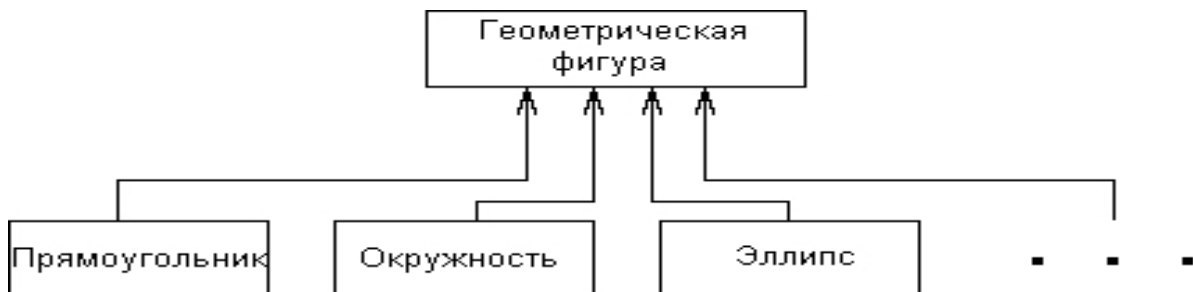
Attitude generalizations

A generalization relationship is a common taxonomic relationship between a more general element (a parent or ancestor) and a more specific or specialized element (a child or descendant). This relationship can be used to represent relationships between packages, classes, use cases, and other UML elements.

Arrow indicates on more general Class (ancestor class or super class), A her absence - on more special class (descendant class or subclass).



Rice. 22- Graphic image relationship generalizations V language UML



Rice. 23. Example graphic images relationship generalizations classes

Stereotypes classes.

- Main Stereotypes: *Borderline classes (boundaryclasses)*, *Entity classes (entityclasses)*, *Control classes (controlclasses)*..

Stereotypes classes

Stereotype - mechanism, used For creations new element modeling (For creations new class types). The class stereotype is indicated under its name and enclosed in double brackets "stereotype".

Main stereotypes: Boundary classes , entity classes , control classes.

These three types correspond to the model-view-control concept and help the analyst separate the view, domain, and control of the system. Because the analysis and design process in the system is iterative, the list of classes changes over time and the initial selection of classes is likely to be different from the final selection.

Entity classes

Entity class is used For modeling data And behavior With long vital cycle. This class type can represent real-world entities or internal elements of a system. Such classes are usually independent from environment, those. They insensitive To interactions surrounding Wednesday With system. Hence, they are application independent and can be used in various applications. Entity classes are usually determined

under development . They are often called domain classes because they represent abstractions of the real world.

Borderline classes

Boundary classes provide interaction between the environment and the internal elements of the system.

Such classes provide an interface for the user or another system (i.e., an actor). They constitute the external dependent part of the system and are used to model the system interfaces.

In order to determine the boundary classes, it is necessary to examine the use case diagrams. Each interaction between in force face And option use must correspond By at least one boundary class. It is this class that allows the actor to interact with the system.

Managers classes

Managers classes serve For modeling consistent behavior one or several actors and the coordination of events that implement the behavior embedded in them.

Control classes can be thought of as classes that "execute" a use case and define its dynamics. They are usually application-specific.

Early on, manager classes are added for each Actor-Use Case pair. These classes define the flow of events in a use case. A manager class is responsible for coordination, but does not itself have any functionality - other classes do not send it many messages. Instead, it sends many messages itself. A manager class simply delegates responsibility to other classes, which is why it is often called a manager class. For example, the TransactionManager class coordinates messages related to database transactions.

Modeling interactions between objects systems

In the process of analyzing a specific use case, a sequence diagram is first constructed. (one or more), describing basic flow events And his subordinates streams. For each alternative flow of events is constructed separate diagram (for reasons of simplicity of visual perception). It is not advisable to describe trivial flows of events (for example, only one object participates in the flow).

System behavior is a description of what actions the system performs, without defining the mechanism for their implementation. One part of such a description is interaction diagrams.

Interaction diagram - describes interactions consisting of a set of objects and the relationships between them, including the messages they exchange.

Interaction diagrams are divided into two types: sequence diagrams and collaboration diagrams. They are interchangeable. The first type of diagrams are organized by time and are used at the requirement analysis stage, and collaboration diagrams are used at the system design stage.

A sequence diagram is a diagram of interactions that emphasizes the temporal ordering of messages, graphically represented by a table with objects arranged along the x-axis and messages in ascending order along the y-axis.

A collaboration diagram is an interaction diagram that focuses on structural organizations objects, hosts And sending messages. Graphically such The diagram is a graph of edges and vertices.

How rule, diagrams interactions contain:

- Objects;
- Connections;
- Messages.

Models interactions covers questions interactions between objects, necessary For implementation of the Use Case. They are created at more advanced stages of requirements analysis, when the class model becomes known.

Use cases define how performers interact with the system. During this interaction, performer are generated events, transmitted system, which represent by yourself requests to perform some operation. For example, a Professor, by entering his last name in the Course Catalog, thereby instructs the Course Catalog system to record this data. That is, this event initiates the execution of some operation.

Sequence diagrams illustrate the interaction of an executor with a system and the operations that are initiated.

Interaction is a set of messages inherent to the behavior of a certain system, which are exchanged between **objects** in accordance with **the connections established between them** .

A sequence diagram is a diagram that, for a given use case scenario, shows events generated by external executors, their order, and events generated within the system itself. It is represented as a two-dimensional graph. Objects are arranged horizontally. In UML, an object in a sequence diagram is indicated by a rectangle containing the underlined name of the object. The name can consist of the object name only, the name of the object and its class, or the name of the class only. Each object has its own timeline, depicted as a dotted line under the object. The message sequences are arranged vertically from top to bottom. Each vertical line is **life line** .

Messages, transmitted between objects, are depicted arrows, directed from the client sending the message to the provider receiving the message. Each message is indicated with at least its name. Show *return on diagram* control from the recipient object to the sender object is not necessary, since the arrow pointing to the recipient object implies automatic return of control to the sender.

Lecture 14-15. Application UML diagrams each type For modeling behavior systems.
Design systems on language UML V Wednesday Rational Rose, Aris, M.S. Office Visio.

Activity diagrams can be used to model the dynamic aspects of a system. They are typically used to model the sequential steps of a computational process. While interaction diagrams focus on the flow of control from one object to another, activity diagrams describe the transitions from one activity to another.

Diagram activities (Activity diagram)- shows flow transition from one activities To another.

Activity -This continued in time non-atomic step calculations V automatic.

Activities, V final account, lead To execution some actions, compiled from atomic computations performed, each of which either changes the state of the system or returns some value.

Diagram activities consists of from:

States activities And states Transitions actions

Objects

States actions can be subjected to decompositions, They atomic. In progress V condition the action work cannot be interrupted, while it is assumed that the duration of one state of action takes an imperceptibly small time.

States activities can be subjected to further decompositions, They Not are atomic and may be interrupted. It is assumed that they require a significant amount of time to complete.

Transitions are used to describe the flow of control . - showing path from one states actions /activities V other. Such transitions are called non-trigger because control is immediately transferred to the next state after completion of work in the initial state.

Flow management somewhere must start off And somewhere end. Can ask start And end flow.

The model can include *branching* , which describes different execution paths depending on some conditions. In point branches Maybe enter one transition, A comes out -two And more. For each The outgoing transition is specified by an expression that is evaluated once upon entering the transition point.

Simple and branching flows are the most common in activity diagrams. But you can also find parallel flows performed streams, What characteristically For modeling business processes. IN UML For organizations such threads use **synchronization**. The merge point is a mechanism for synchronizing several parallel threads of execution - each of them waits until all the others reach this point, after which execution continues within the framework of a single process. One or more transitions enter this point, and one exits.

Diagrams states

On diagram states display vital cycle one object, starting With moment his creations And ending in destruction.

State diagram (stateshchart diagram) - shows the position of a single object, event, or message that causes a transition from one state to another, and the actions that result from the state change.

Creation diagrams states:

1. Call contextual menu class V browser
2. Choose command New—Stateshchart Diagram
3. Enter Name diagrams

A state is a certain position in the life of an object in which it satisfies a certain condition, performs a certain action, or waits for an event.

A state diagram includes all the messages that an object receives and sends. A scenario is a single pass through the state diagram. The interval between two messages sent by an object typically provides the state. Thus, to obtain the state, you need to examine the sequence diagram .

Transitions between states

Transitions (state transitions) represent a change from the initial state to the subsequent one (which may be the same as the initial one).

Transition Maybe be accompanied certain action. E

Eat two methods exit from states: auto And non-automatic. Automatic change state occurs when the action of the original state is completed - there are no events associated with the transition.

Non-automatic transition between states is called determined event (from other object or from

external environment). It is believed that both type of transition are being carried out for zero time And Not can be interrupted. The transition between states is depicted as an arrow pointing from the initial state to the next one.

Creation transitions:

1. Click on button StateTransition on panels tools
2. Click By original state diagrams
3. Conduct line transition To subsequent condition.
4. Enter Name transition

Special states

There are two special states on diagram states - This initial (Startstate) and final (Stopstate).

Each one diagram Maybe have one And only one initial state, because object Maybe be in a complete state immediately after its creation.

Object Maybe have some final states. UML notations:

Initial state

Final state



Parameters transitions

WITH transition between states Maybe be connected **condition (guard condition)** and/or action (**action**). **A transition can also trigger an event.**

Action - This behavior, manifesting at emergence transition.

An event is message, sent to another object systems.

Condition - boolean expression values attributes, which ass drops transition, If it true.

AND action, And examination conditions represent by yourself behavior object And usually are being implemented V view operations.

Notations UML For parameters transition:

Name events [borderline condition]/action Class

name.Send message

Parameters states

Actions, accompanying possible transitions V certain state, Can consider How **entry actions** for this state.

AND vice versa, actions, accompanying transitions from of this states, are For him **exit (exitaction)**.

Behavior, emerging inside states is called **activity (activity)**

Activity it begins at at the entrance V state And is ending or interrupted at transition from him. A behavior can be a simple action or an event sent to another object.

Diagram components (componentdiagram) A

component is a physical module of code.

Components can be either source code libraries or executable files. For example, in C++, a class body file (.CPP) and a header file (.H) would be separate components.

After creations components from they place on diagram classes. The only one type connections between components are connections.

Diagram components (Component diagram) - it's called diagram, on which shown system components and connections between them.

On such diagram Can see original code And executable components systems.

Types components

There are two main type components: libraries original code And executable files. Different icons are used to represent each of them.

Diagrams placements (deploymentview)

Diagram placements displays All nodes networks, connections between them And processes, in progress on each node

. Creation diagrams placements V RationalRose:

1. Twice click mouse on DeploymentView (Performance Placements) V browser

Processors

Processor (processor) - it's called any car, having computing power, those. capable produce processing data.

IN this one category will get there servers, workers stations And other devices, containing physical processors.

Generation code

- Examination models
- Creation components

- Display classes on components
- Installation properties generations software code
- Choice class, component or package
- Generation software code.

IN different languages All these stages are optional. For example, V C++ Not Necessarily Creation packages. Create program code without checking the models, although this sometimes leads to errors during generation.

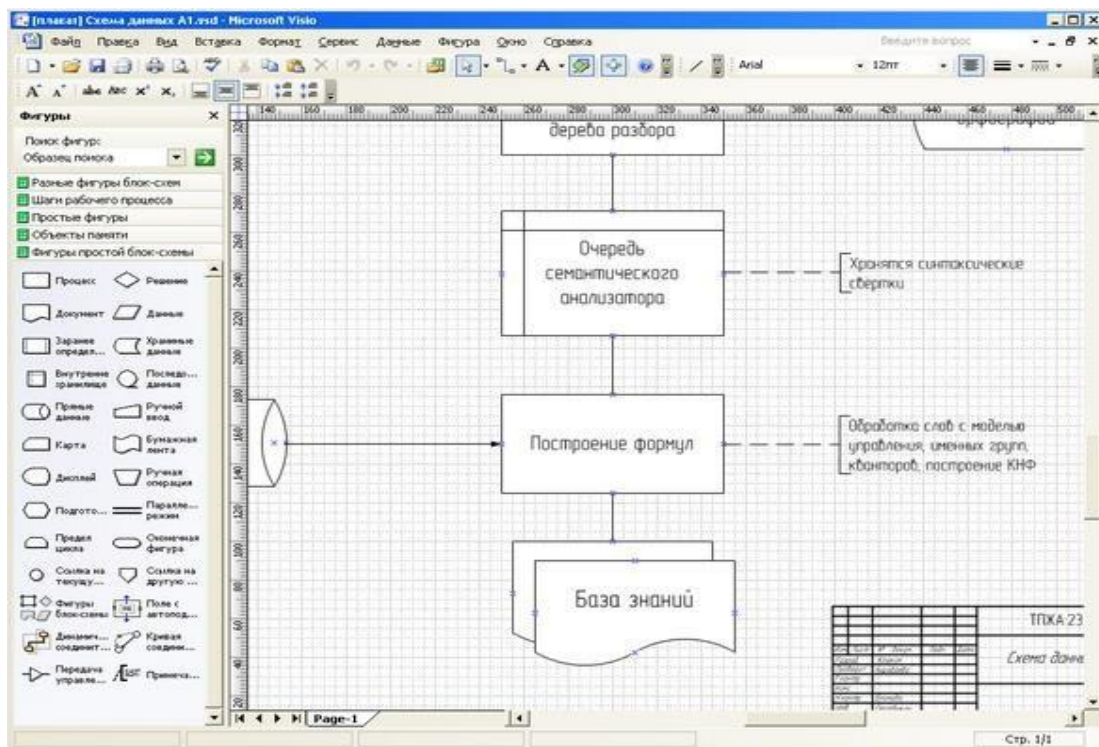
While not all steps are required, the first five are recommended to be completed before starting the generation process. models will help reveal inaccuracies And flaws, unwanted With dots of vision subsequent program generation. Stages, on which are being considered components - This way display logical scheme systems on its physical implementation, and at these stages a large amount of useful information is collected. If you skip these stages, That For creations Rational components Rose will use the package structure logical presentations.

Design systems on language UML V Rational environment Rose, Aris, M.S. Office Visio

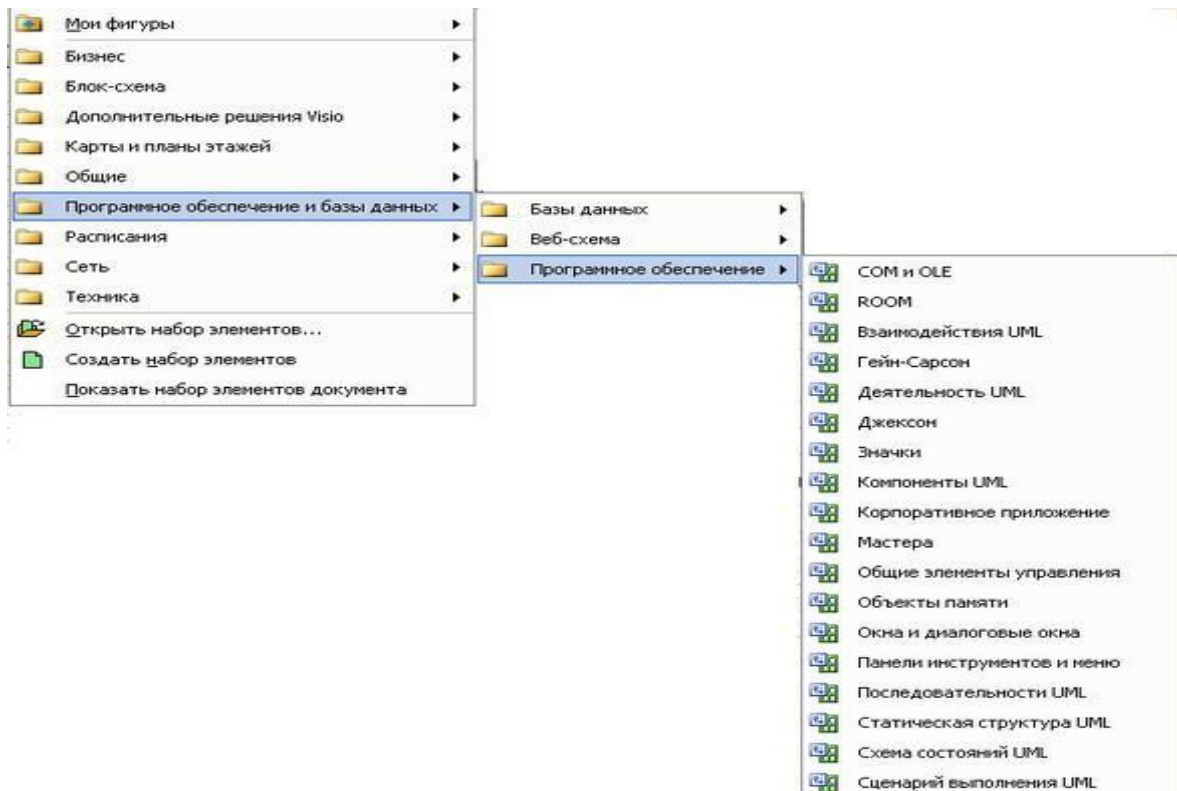
Microsoft Office Visio

IN difference from powerful highly specialized packages Visio has simple user-defined interface and an exhaustive set of graphic primitives. Secondly, all graphic primitives are divided into clear groups according to existing standards for constructing diagrams, according to the subject of the diagrams. Here are present both elements for constructing schemes according to clear standards (for example, IDEF0), So And more complex primitives designed to please eye and draw attention to your design.

Microsoft has been developing Visio for quite a long time. The package gained particular popularity after the release of Visio 2000, which was an independent product. Subsequent versions were integrated with the MS Office package, which was missing the last tool to be complete - a convenient means of visualizing professional-level diagrams and charts.



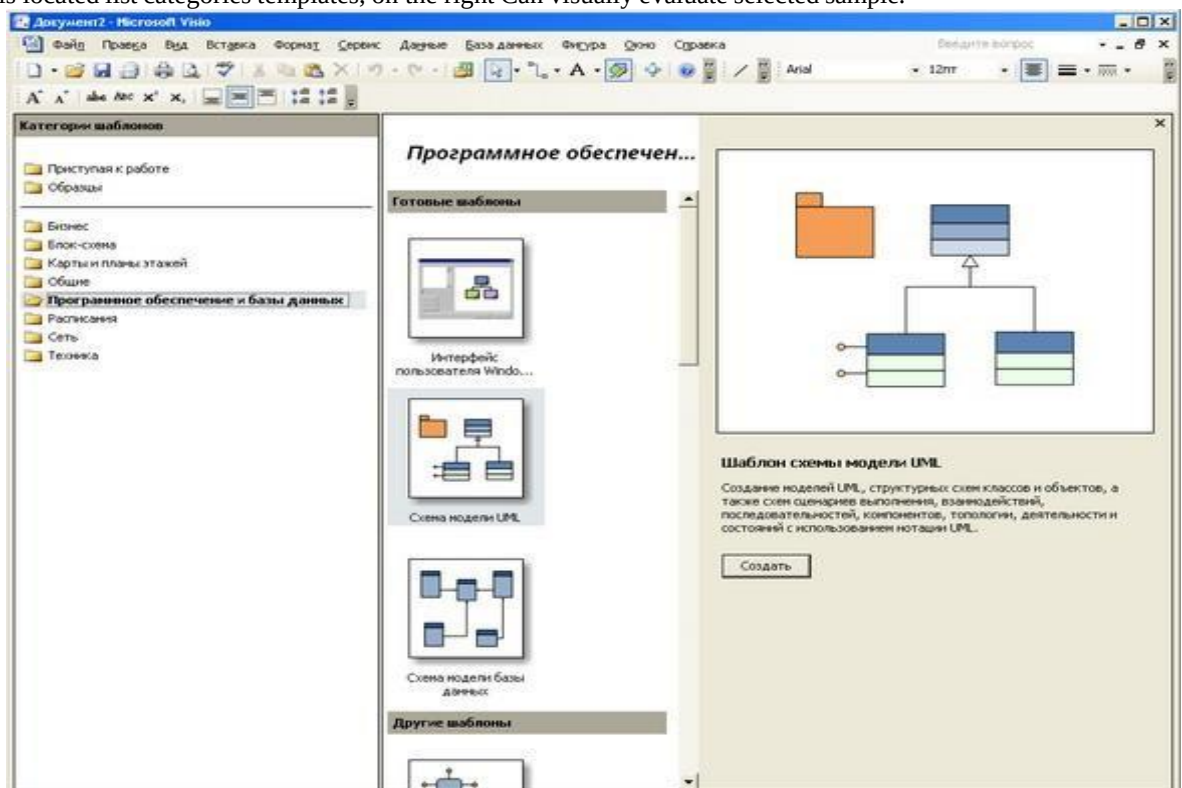
Interface classical, on Russian language.



The number of basic graphic primitives for constructing diagrams and charts is pleasing - all are divided into many categories. Therefore, you can easily find the necessary figures.

When starting work, you can choose one of the many templates that best suits your visualization needs. After opening the required template, a new project is created, and the required graphic primitives are located on the right sidebar. To build a diagram, simply drag the figures onto the project workspace, arrange and connect them as needed.

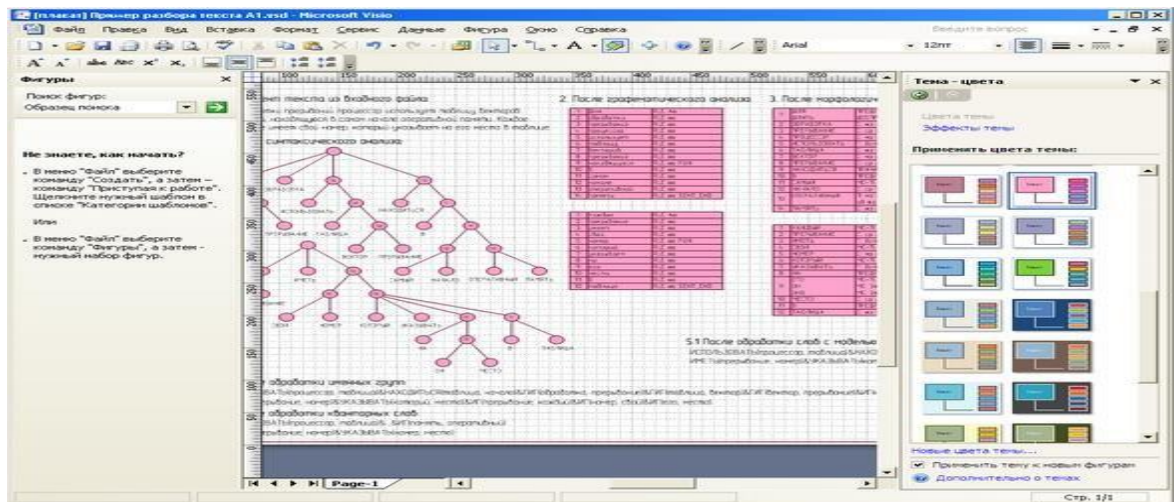
Left is located list categories templates, on the right Can visually evaluate selected sample.



In external view working areas, V interface V in general Not discovered Nothing fundamentally new - Microsoft adheres to a strict classical style here. What's new in terms of functionality?

Microsoft Office Visio 2007 | Functional innovations

The new version of Visio now includes the ability to use color themes and special effects to make your drawings more informative and visually appealing.



Among effects, which Can apply To document, There is shading, rounding contours And connecting lines, fill.

Very a useful innovation became function autoconnections. If she active, That at hovering cursor on any from primitives around last appear small translucent blue arrows With each sides. When you hover over any of these arrows, a primitive with which the connection will occur is highlighted in red (usually the closest primitive in the selected direction). Then, when you click on this arrow, the figures will be connected.

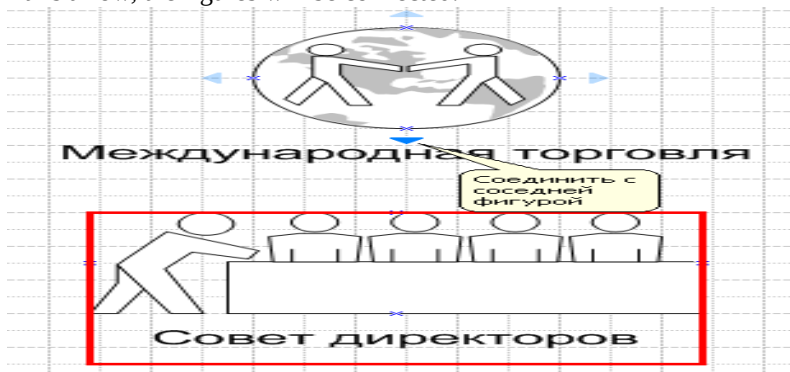
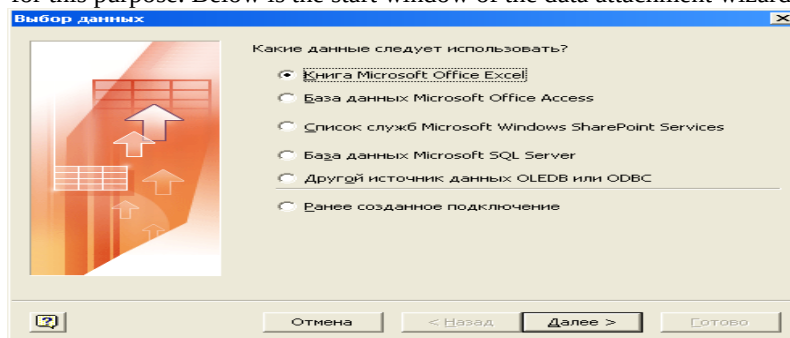


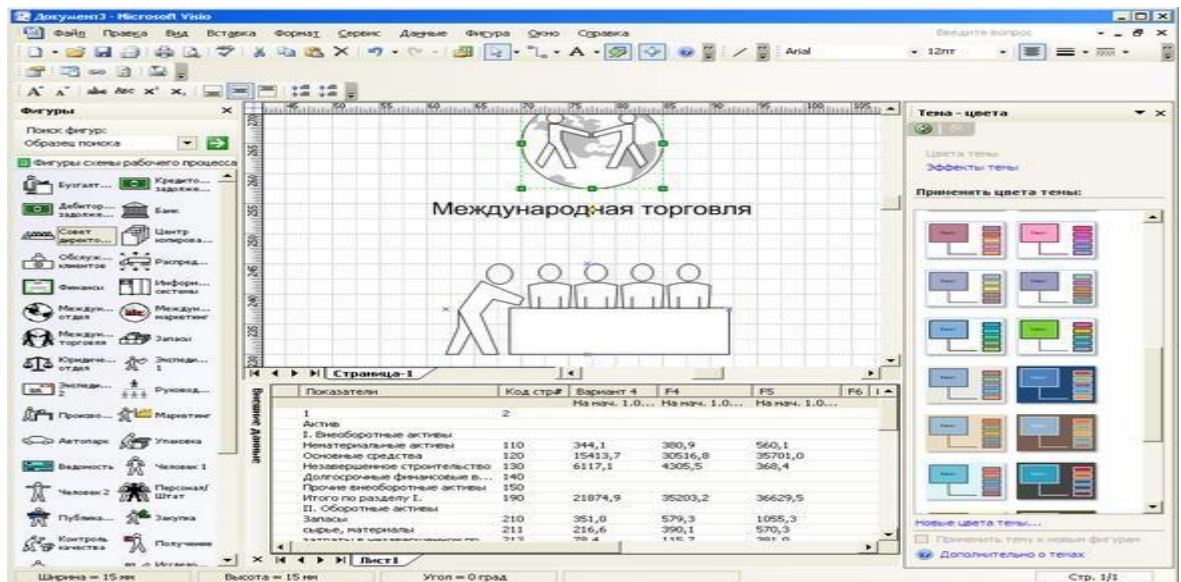
Illustration functions autoconnections.

The professional version now has the ability to include external data in documents. There is even a separate menu item "Data" for this purpose. Below is the start window of the data attachment wizard.



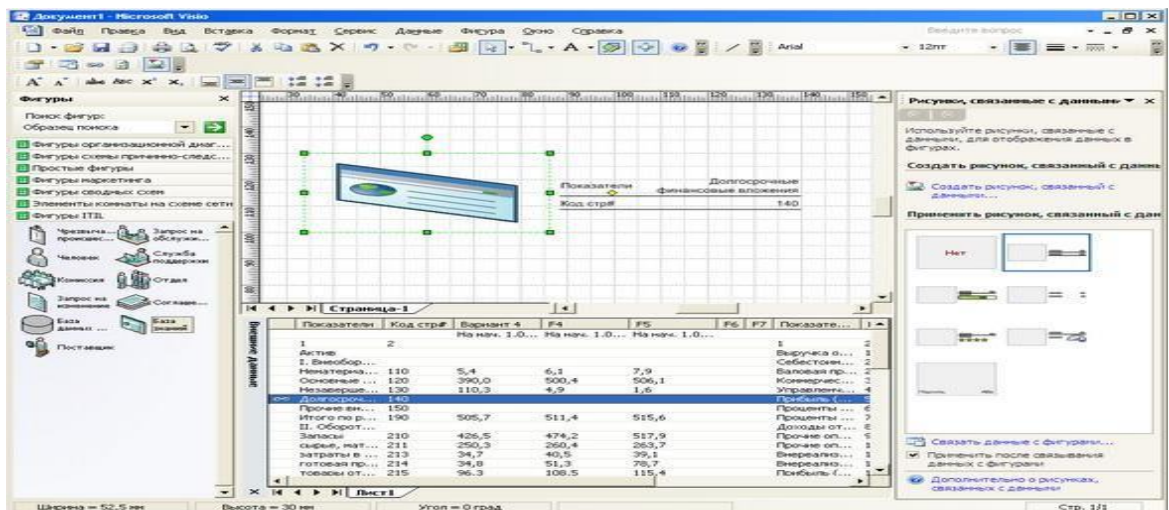
Master choice data.

For example, we linked a table created in Excel to the document. After that, the data was displayed in a separate window at the bottom of the application's workspace. Of course, you can't edit them, but this window fulfills its informative function to the fullest extent.



After adding external data in the form of a table created in Microsoft Office Excel, a separate information window appeared at the bottom of the application's workspace.

Next Can Very Just tie up separate lines tables With figures V document. It is enough to select the necessary primitive on the left, and then drag the necessary lines to the document work area. As a result, the selected primitive with the assigned data will be displayed on the document.



Object And belonging to him data. Click on picture For increases

On this Job With data Not is limited. Every primitive, connected With certain indicator, can reflect its value in the form of certain symbols (scales and other indicators). In this case, it is also possible to automatically change the color of the figures at certain values of the indicators associated with them. To update the data values, use the "Data/Refresh data" item.



In the professional version of the product, it is also possible to build a new type of diagrams - summary diagrams. These diagrams are primarily intended for analyzing data in a hierarchy. A specialized template is provided for their creation.

We have listed the main innovations of the new version of the product. Perhaps there are not so many of them in quantitative terms, but pay attention to how qualitative they are. The ability to link elements of diagrams and schemes with real data can be very useful.

System modeling business processes ARIS

The ARIS tool environment represents a certain approach to formalizing information about the enterprise's activities and presenting it in the form of graphical models that are convenient for understanding and analysis. The models created with this tool reflect the existing situation at the enterprise with a certain degree of approximation. The degree of detail of the description depends on the goals of the project being implemented. Models built using this methodology can be used to analyze and develop various types of solutions for enterprise reorganization, implementation of an information system for supporting enterprise activities.

The system implements the principles of structural analysis, when the enterprise is represented as a complex system consisting of various components that have various types of interrelations with each other. Thus, the considered tools allow us to define and reflect in models the main components of the enterprise, ongoing processes, and the information used, as well as to present the interrelations between these constituent components.

The models created in the ARIS environment represent a documented set of knowledge about the management system at the enterprise – the organizational structure of the enterprise, interactions between the enterprise and other market entities, the composition and structure of documents, sequences of process steps, job descriptions departments And their employees. IN difference from other similar funds, ARIS keeps all information V a single repository, which ensures the integrity and consistency of the modeling and analysis process. In the Gartner Group rating, it is the manufacturer ARIS (IDS Prof. Scheer) that occupies a leading position in the market of business process modeling and analysis tools.

The methodology of the product under consideration – ARIS Toolset – is based on the theory of “Construction of Integrated Information Systems” developed by Professor Sher, which defines the principles of visual display of all aspects of the functioning of the analyzed companies.

Purpose systems

The ARIS system is designed for the visual representation of the principles and conditions of operation of various types of companies, as well as for the analysis of their activities according to various indicators in order to determine the ideal characteristics of the company's activities, reorganization of its organizational structure, goals and functions, business processes, and the data used.

IN within the framework methodologies ARIS there is Also opportunity define requirements To automated control system and carry out its design.

The range of companies using the integrated tool environment ARIS is quite wide. These can be companies engaged in consulting activities, using ARIS as a means of analyzing the activities of client firms. Also, this product can be used by software development companies to ensure the process of designing information systems. Moreover, the ARIS system Maybe successfully apply various genus companies, engaged How production products, So and provision services, For analysis And reorganization own activities And maintenance internal processes of the company in accordance with changing market conditions, operational goals and other conditions. That is, the company always has at its disposal a visual display of all aspects of its activities, as well as means of their analysis (cost, dynamic, compliance of the organizational structure with the functions performed, etc.). The range of enterprises analyzed is practically unlimited.

The focus of this product on quality management according to the ISO9000 standard will ensure the enterprise's activities at a level that meets generally accepted requirements for the organization of processes. In addition, this system will provide support for the process of preparation and direct certification according to this standard. For this purpose, the system provides for the creation of special reports that meet the requirements of ISO9000, as well as exist models, reflective full methodology process certifications By standard ISO 9000 – analysis of the existing management system at the enterprise, reorganization of business processes, personnel training and the certification itself. Correct implementation of all necessary measures will allow the enterprise to successfully enter the European market and be recognized by the majority of European companies.

Review main modules ARIS

The ARIS integrated tool environment consists of a set of tools. ARIS tools can be divided into three main categories – functional modules, prototype models, interfaces to other software tools.

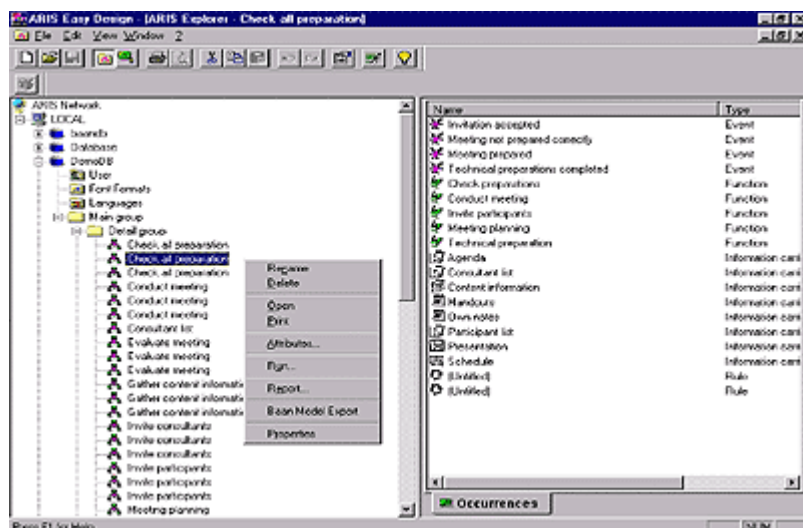
TO main functional modules relate next components: ARIS Toolset (basic Wednesday modeling and analysis), ARIS Easy Design (simplified modeling environment), ARIS for R/3 (system implementation toolkit R/3 management from SAP).

There are a number of additional functional modules: ARIS ABC (a tool for functional and cost analysis of enterprise processes), ARIS Simulation (dynamic modeling of processes).

ARIS tools can be used as a single-user environment, and also support collective development in a client-server environment. The possibility of collective work is provided by the following tools: the procedure for merging databases (Merge component), ARIS Server (a tool for joint access of several users to a single data storage – ARIS repository), ARIS Web Link (a component that allows organizing the publication of models and their joint use by several users via the Internet and intranet).

Main instrumental Wednesday ARIS Toolset

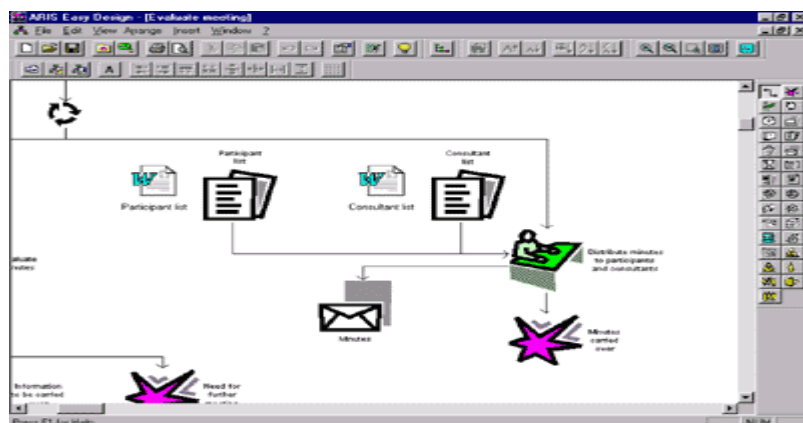
The main module of analysis and design is the basic component of ARIS Toolset, which includes tools for working with enterprise activity analysis models (analysis of organizational structure, goals and functions, data, business processes). Includes tools for modeling, analysis, navigation, access control, reports and others necessary possibilities For full-fledged works By analysis And modeling. All functional modules



appropriate to his role in the project is provided.

Each object or connection has a set of attributes, with the help of which you can set the quantitative and qualitative characteristics of the elements being modeled. In particular, for each function you can set its name, unique code in the project, author, time and date creation, detailed description, example of implementation, time and money costs for the implementation of this functions and so on further. All the specified characteristics objects and connections are formalized and used when conducting an analysis or compiling a report. The values of the characteristics are entered into the corresponding tables. Tables also play an important role in conducting functional cost analysis and analyzing the results of dynamic modeling.

The presence of wizards makes it easier for the user to go through multi-step operations in the ARIS environment. The interactive mode increases the visibility and ease of use of the main functional capabilities of ARIS. Wizards provide explanations of options or alternatives, and allow you to go back a step. Wizards are used For such operations, How Creation bases data And models, generation reports And tables



In addition to information, incoming structure models (objects, connections And their characteristics) users can complement models text headlines, standard graphic objects (circles,

ARIS consists of four main components:

Explorer, Designer, Tables and Wizards.

The Explorer, whose ideology is similar to the Microsoft Windows Explorer, serves as a means of organizing projects, models, and objects, and also provides a number of administrative functions for managing projects, models, objects, and other information.

The designer provides the process of modeling, that is, documentation of knowledge O enterprise With with help graphic objects and the relationships between them.

Unlike other graphical tools, AIRS stores information about objects and the relationships between them in a database, making it possible to use this information V others models, at conducting the project manager to develop a methodology user Only the amount of ARIS methodology that is

functional cost analysis, conducting semantic checks models and so on.

Creating models in the ARIS environment does not require any special knowledge. Process removal objects on the model field and making connections between objects is quite simple. Convenient means of aligning objects and implementing automatic the placement of objects on the model significantly reduces the creation of capacious and easy-to-perceive graphic representations. The ability to control the process of automatic placement of objects on the model increase flexibility And convenience

rectangles), and also embed external documents and applications into models. External documents and applications can be related With specific objects models or same taken out directly on field modeling, both as an image and as an icon. Thus, directly from the ARIS environment, the user can access additional information commenting on the model he is studying.

An important difference between ARIS and other tools is the ability to model using several languages. All information in the database can be presented in several languages, which is extremely important in the work of multilingual teams of specialists.

Developed by models can be analyzed By certain rules. Rules analysis are being asked in specially developed algorithms based on a language similar to Visual Basic. Some analysis algorithms already exist in the ARIS environment:

Classification functions - analysis And classification functions V dependencies from values their attributes; this algorithm built like this in this way, What He Maybe be configured For various needs (For example, identifying the number of manual functions or the number of functions that add value to the final product).

Necessary organizational changes in processes. This type of analysis provides recommendations for changing the responsibility of performers for the implementation of individual process operations.

Analysis movements information inside process - analysis informational breaks inside simulated processes.

Analysis of the use of elements of an automated system in processes – determination of the composition of functions supported by the automated system and performed in the traditional way.

The analysis results are presented in the form of tables that can be viewed both by built-in ARIS tools and by other applications (RTF editors, Microsoft Word, Excel, etc.).

Using the corresponding wizard, users have the opportunity to create their own analysis algorithms that will be used in subsequent projects.

The operation of creating a report according to specified algorithms allows you to present the contents of the database (graphics interrelations objects And meanings their attributes) V text view. Structure And compound report is determined by the algorithm used, which can also be developed by the user himself.

There are special reporting algorithms that meet certain certification standards. Here are some from them: DIN EN ISO 900X, EN 45011, EN 45013, VDA 6, Certification under the Medical Products Act MPG, EN 460001, EN 460002, GMP, EU - Ecological Audit Ordinance, and DIN ISO 14001 Environmental Management. For certification By ISO900x V Wednesday ARIS exist relevant elements qualities, which can be are specified for each model created.

Built-in component Merge allows realize copying models And objects from one bases data to another by the Drag&Drop method ("drag and drop"). This option becomes especially important when it is necessary to combine the results of several independent teams, as well as when organizing the accumulation process And use prototypes design solutions. Component Merge carries out processing conflicts of matching model and object names during the copying process. The rules for resolving such conflicts are defined by the user.

The ARIS Toolset basic modeling environment provides essential database administration functions projects V Wednesday ARIS. Functions administration are being carried out through Conductor. TO main The following are administrative elements: servers, databases, users and user groups, font formats, database languages, methodological filters, model groups, models, objects, tables.

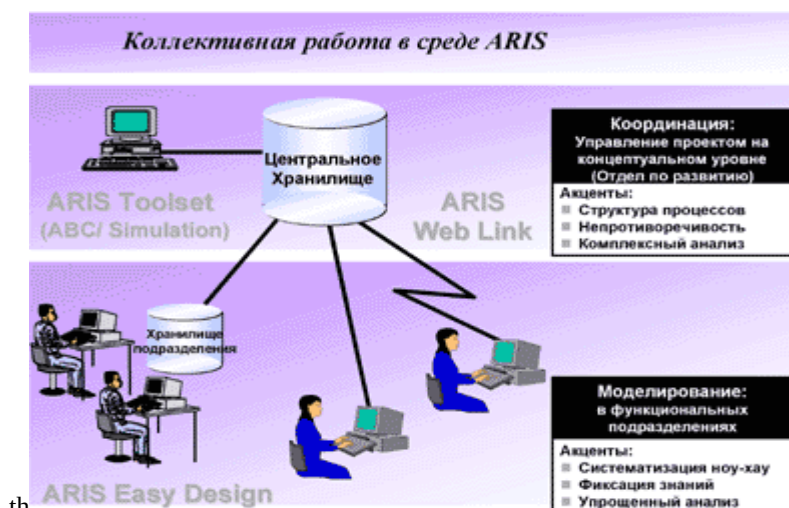
For each created bases data is determined compound users, having right access To this database. Among the database users, there is one or more database administrators who have the appropriate administration rights. An important element of administration is the definition of a hierarchy of groups within each database - an analogue of the file structure in operating systems. For each group, access privileges to it for the corresponding users are defined, which ensures the confidentiality of the information stored in the database. Each group can store models, objects, tables and other information. The administrative function of database reorganization allows you to delete unused database elements, as well as reorganize the internal structure of the database.

Like this in this way, ARIS Toolset is powerful means modeling, analysis And documentation knowledge about the company's activities, providing a variety of convenient and effective functional capabilities.

ARIS Easy Design

ARIS Easy Design is a simplified modeling and analysis tool, limited in functionality compared to ARIS Toolset. In terms of methodology, ARIS Easy Design supports all 83 modeling methods used in ARIS Toolset. In this respect, there is full compatibility between them. Models created in one of these tools, can be without losses used V friend. More Togo, both these means can function V single environment collective developments. At this workers places ARIS Easy Design are used V relevant

divisions enterprises For collection information And conducting general analysis, A system ARIS Toolset - For



identifying deeper contradictions and optimizing the developed models.

The ARIS Easy Design environment lacks such features as combining database contents using the Merge component, administering methodological filters, developing custom analysis algorithms and report structures, consolidating objects (combining two or more objects created by different users but having the same meaning), generating business rules, and some others.

In addition, ARIS Easy Design does not allow the use of such additional functionality modules, How ARIS ABC And ARIS Simulation.

Functional cost analysis in the ARIS ABC module

specify various cost items for performing functions in

th However, to conduct a more accurate analysis, it is necessary to use appropriate methods. For implementation full-fledged analysis cost processes (chains processes) exists module ARIS ABC, consisting of module, connectable To basic Wednesday developments, And set diagrams, necessary For descriptions initial data for cost analysis (description of resources, cost centers, description of the relationship of cost centers and resources used with business process chains).

As a result, without leaving the main environment, you can switch to the cost analysis mode to calculate various economic indicators.

Usage method functional-cost analysis justified disadvantages traditional methods of determining costs, as well as changes in the cost structure at enterprises (the volume of overhead costs increases). This connected V the first queue With increase numbers operations By preparation, planning, control And tracking the progress production. IN my queue, efforts managers production cost are being reduced And Not can be a reliable basis for determining overhead costs for the production of products (goods or services).

Under the traditional calculation system, overhead costs for the production of individual goods and services were allocated using approved coefficients, which did not always accurately reflect reality. According to the ABC method, overhead costs are allocated between individual products based on the processes required for their production. Processes imply the entire sequence of actions for planning, production and sale of products to the end consumer. In other words, overhead costs are redistributed from the centers of their origin (functional units) to the processes aimed at producing products.

TO advantages use method ABC Can to attribute the following:

- transparency structures overhead expenses;
- warp For "more precise" methods definitions cost price products;
- identification ineffective used resources;
- opportunity conducting analysis "What, If?", That There is definition Togo, What will happen With expenses, If processes will be changed;
- warp For periodic control processes.

Conducting functional cost analysis in the ARIS ABC environment is carried out for processes developed by means of the ARIS family (ARIS Toolset, ARIS Easy Design, ARIS for R/3). A single integrated modeling and analysis environment significantly increases the efficiency of such projects. The possibility of collective work ensures a rational distribution of responsibilities between specialists - modeling of individual elements takes place in each functional unit, and a comprehensive analysis of all business processes is carried out in the development department.

**MINISTRY OF SCIENCE AND HIGHER EDUCATION OF
THE REPUBLIC OF KAZAKHSTAN
NAU «L. N. GUMILYOV EURASIAN NATIONAL UNIVERSITY»
Department of Computer and Software Engineering**

**METHODICAL INSTRUCTIONS FOR PERFORMING LABORATORY WORKS
IN THE DISCIPLINE "PROGRAM DEVELOPMENT TOOLS"**

**for students of the educational program
6B06104 - Computer Engineering and Software**

Astana, 2026

Guidelines for completing laboratory work on the subject "Software Development Tools" for students in the educational program 6B06104 - Computer Engineering and Software

Developer:

**Senkovskaya A.A.,
candidate of technical science, senior lecturer**

Reviewed and recommended for publication at the meeting of the Department of Computer and Software Engineering, protocol No. 10 dated May 04, 2026.

Head of the Department _____ Dyussekeyev K.A.

Approved at the meeting of the Educational and Methodological Commission, protocol No. 9 dated May 14, 2026.

Chairman of the Editorial Board _____ Davletova A.Kh.

CONTENTS

INTRODUCTION	4
ORGANIZATIONAL GUIDELINES	5
LABORATORY CLASSES PLAN.....	6
LABORATORY WORK #1-2.....	6
LABORATORY WORK #3.....	8
LABORATORY WORK #4	11
LABORATORY WORK #5	13
LABORATORY WORK #6.....	15
LABORATORY WORK #7	17
LABORATORY WORK #8-9.....	19
LABORATORY WORK #10.....	22
LABORATORY WORK #11-12.....	24
LABORATORY WORK #13	27
LABORATORY WORK #14-15.....	29
EVALUATION CRITERIA FOR THE DISCIPLINE.....	32
LIST OF RECOMMENDED REFERENCES.....	34

INTRODUCTION

The course "Software Development Tools" is designed to develop students' practical skills in the use of modern software tools used at all stages of software development. This course covers the design, development, testing, debugging, and maintenance of software products using specialized tools and development environments.

The course's guidelines are designed to help students organize and complete laboratory work and provide recommendations for preparing, completing, and formatting practical assignments. Completing laboratory work allows students to consolidate their theoretical knowledge, acquire skills in working with software development tools, and master methods for solving practical problems in programming and software development.

During laboratory work, students explore the capabilities of modern integrated development environments, version control systems, program debugging and testing tools, and development automation tools. Particular attention is paid to developing skills in problem analysis, designing the structure of a software product, and writing and testing program code. The guidelines include a description of the goals and objectives of the labs, a list of required software, assignment procedures, quizzes, and a list of recommended literature. The material is designed to develop practical competencies required by future IT and software engineering professionals.

By completing the labs, students should acquire the skills to effectively use software development tools, learn how to apply modern programming technologies, and ensure the quality of the software they create.

ORGANIZATIONAL GUIDELINES

These guidelines define the procedure for completing, formatting, and defending laboratory assignments in the course "Software Development Tools," and establish requirements for organizing student independent work.

Laboratory assignments are completed in accordance with the curriculum and schedule approved by the instructor. Before each assignment, students are required to familiarize themselves with the theoretical material on the relevant topic, study the assignment, and prepare answers to the test questions.

Laboratory assignments are completed individually, unless otherwise specified in the assignment. During the assignment, students must independently develop an algorithm for solving the assigned problem, implement it using the selected development environment, and analyze the results.

It is recommended that modern software development environments (e.g., Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect) be used for laboratory assignments.

The defense of the laboratory assignment is conducted in the form of an interview, during which the student must demonstrate understanding of the completed assignment, explain the operating principle of the developed algorithm, and answer additional questions.

In the event of late submission or unsatisfactory defense, the lab work must be revised and resubmitted. Independent completion is mandatory; unattributed plagiarism may result in the disqualification of the work.

Compliance with these organizational guidelines ensures effective learning of the course, the development of practical skills, and the development of professional competencies in the field of algorithms and data structures.

LABORATORY CLASSES PLAN

Week no.	Module no.	Name of topics of laboratory classes	Number of hours
1	1	BPwin 4.0 tools. Creating a functional model using BPWin 4.0.	2
2	1		2
3	1	Using Erwin to develop a data model. Logical model. Entities and relationships.	2
4	1	Features of the IBM Rational Rose program user interface.	2
5	1	Building a Use Case Diagram (Diagrams of Use Cases).	2
6	1	Formulation of technical specifications. Writing system requirements.	2
7	1	Activity diagram - Construction of activity diagrams.	2
8	2	Modeling interactions between objects. Interaction diagrams: Sequence diagrams. Collaboration diagrams.	2
9	2		2
10	2	State diagrams.	2
11	2	Classes. Class diagram	2
12	2		2
13	2	Developing apps using no-code platforms. AppMaster platform.	2
14	2	Developing an application with database connectivity on the AppMaster platform	2
15	2		2
TOTAL			30

LABORATORY WORK #1-2

BPwin 4.0 tools. Creating a functional model using BPWin 4.0.

Objective: Explore the capabilities of the BPwin 4.0 software tool and develop skills in constructing functional business process models using the IDEF0 methodology.

Work Plan: Creating a Functional Model.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

BPwin 4.0 is a business process modeling tool designed for building functional models of an enterprise or information system. The program is based on the IDEF0 methodology, which allows a system to be represented as interconnected functions and data flows.

The main elements of an IDEF0 model are:

- Functional block (activity, process);
- Inputs;
- Outputs;
- Control;
- Mechanisms.

Process Flow

1. Launching BPwin 4.0

1. Launch BPwin 4.0.
2. Select File → New Model.
3. In the window that opens, specify:
 - Model name;
 - Author name;
 - Methodology type – IDEF0.

4. Click OK.

2. Creating a context diagram

1. An A-0 context diagram is automatically created on the workspace.
2. Rename the main functional block to match the theme of the process being modeled.

For example:

- "Organization of the educational process";
- "Order processing";
- "Library operations".

3. Add arrows:

- Left - input data;
- Right - output data;
- Top - control actions;
- Bottom - process execution mechanisms.

3. Adding functional elements

1. Select the arrow creation tool.
2. Create the necessary links between the model elements.
3. Specify a name for each arrow.
4. Check the correct flow direction.

4. Decomposing the functional block

1. Select the main block.
2. Select Diagram → Decompose.
3. Specify the number of child processes.
4. Create a decomposition diagram.

For example:

- A1 - Data reception;
- A2 - Information processing;
- A3 — Result Generation;
- A4 — Execution Monitoring.

5. Configuring Process Links

1. Connect child processes with arrows.
2. Configure input and output flows.
3. Add control elements and mechanisms.
4. Check the logical correctness of the model.

6. Model Design

1. Configure the block layout.
2. Resize objects if necessary.
3. Check the diagram's readability.
4. Add text comments.

7. Model Validation

1. Analyze the model for errors.
2. Ensure that all required links are present.
3. Check the correctness of element naming.

8. Saving the Project

1. Select File → Save As.
2. Specify the file name and save directory.

3. Save the project.

Individual Assignments

Option 1. Build a functional model of the "Library Operation" process. Option 2. Develop a model of the "University Student Admissions" process.

Option 3. Build a functional model of the "Online Store Operations" process.

Option 4. Develop a model of the "Customer Service at a Bank" process.

Option 5. Build a functional model of the "Car Service Operations" process.

Option 6. Develop a model of the "Hotel Service Organization" process.

Option 7. Build a functional model of the "Polyclinic Operations" process.

Option 8. Develop a model of the "Goods Delivery Organization" process.

Option 9. Build a functional model of the "College Educational Process" process.

Option 10. Develop a model of the "Warehouse Operations" process.

Conclusion. During this lab, the participants explored the key capabilities of BPwin 4.0 for creating functional models of business processes. They also acquired skills in constructing context diagrams and decomposition diagrams using the IDEF0 methodology, as well as practical skills in information process modeling.

Test Questions

1. What is the purpose of BPwin 4.0?
2. What is the IDEF0 methodology?
3. What elements are used in an IDEF0 functional model?
4. What is the difference between inputs and control actions?
5. What is a mechanism in the IDEF0 model?
6. What is the purpose of process decomposition?
7. What is a context diagram?
8. What are the benefits of functional modeling?
9. How is model validation performed in BPwin?
10. In what areas is BPwin 4.0 used?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.

2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)

3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #3

Using Erwin to develop a data model. Logical model. Entities and relationships

Objective: Explore the capabilities of the Erwin CASE tool for building logical data models and master the principles of creating entities, attributes, and relationships between objects.

Work plan: Design a logical model, build entities and relationships.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

Erwin Data Modeler is a database design tool designed for creating logical and physical data models.

A logical model describes the structure of data independently of a specific database management system and includes entities, attributes, and the relationships between them.

The main elements of a logical model are:

- Entities;
- Attributes;
- Relationships;
- Primary Keys;
- Foreign Keys.

A logical model allows you to define the structure of the future database, the relationships between objects, and the rules for storing information.

Process Flow

1. Launching Erwin

1. Launch Erwin Data Modeler.
2. Select File → New.
3. Create a new logical data model.
4. Specify the model type — Logical Model.

2. Creating entities

1. Select the Entity tool on the toolbar.
2. Place the required entities on the workspace.
3. Assign a name for each entity.

For example:

- Student;
- Group;
- Teacher;
- Course.

3. Adding Attributes

1. Double-click to open the entity's properties.
2. Go to the Attributes tab.
3. Add entity attributes.

Example for the "Student" entity:

- Student_ID;
- Last_Name;
- First_Name;
- Date_of_Birth;
- Address.

4. Assign a Primary Key.

4. Creating Relationships between Entities

1. Select the Relationship tool.
2. Connect the entities according to their subject area.
3. Specify the relationship type:

- One-to-One;
- One-to-Many;
- Many-to-Many.

Example:

- One group contains many students;
- One instructor teaches several courses.

5. Configuring Relationship Properties

1. Open the relationship properties.
2. Set the relationship to mandatory.
3. Check for automatic foreign key creation.
4. Specify role names if necessary.

6. Validating the Model

1. Perform a model validation.
2. Ensure there are no connectivity errors.
3. Check for the presence of primary keys in all entities.

7. Model Design

1. Organize the arrangement of entities.
2. Configure the display of attributes.
3. Add comments to model objects.
4. Check the readability of the diagram.

8. Saving the Project

1. Select File → Save As.
2. Specify a project name.
3. Save the model file.

Individual Assignments

Option 1. Develop a logical model of the "Library" database.

Option 2. Develop a data model for the "Educational Process."

Option 3. Develop a database model for the "Online Store."

Option 4. Develop a data model for the "Car Service Center."

Option 5. Develop a database model for the "Hospital."

Option 6. Develop a data model for the "Hotel."

Option 7. Develop a database model for the "Movie Theater."

Option 8. Develop a data model for the "Sports Club."

Option 9. Develop a database model for the "Cafe."

Option 10. Develop a data model for the "Warehouse."

Conclusion

This lab explored the basic capabilities of Erwin Data Modeler for creating logical data models. We learned methods for constructing entities, defining attributes, forming relationships between objects, and designing a database structure.

Questions

1. What is the purpose of Erwin?
2. What is a logical data model?
3. What is an entity in a data model?
4. What are attributes used for?
5. What is a primary key?
6. How is a primary key different from a foreign key?
7. What types of relationships exist between entities?
8. Why is data normalization performed?
9. What are the benefits of using CASE tools?
10. How does a logical model differ from a physical data model?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.

2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)

3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #4

Features of the IBM Rational Rose program user interface.

Objective: To study the basic interface elements of the IBM Rational Rose program, master the principles of working with UML modeling projects, and acquire practical diagramming skills.

Work Plan: Interface Design.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

IBM Rational Rose is a CASE tool for object-oriented analysis and design of software systems, designed for modeling information systems using UML (Unified Modeling Language).

The program allows you to:

- Develop UML diagrams;
- Model system structure and behavior;
- Analyze software projects;
- Document software architecture.

The main elements of the IBM Rational Rose interface are:

- Main menu;
- Toolbar;
- Project browser;
- Diagram workspace;
- Object properties window;
- Status bar.

Process Flow

1. Launching IBM Rational Rose

1. Launch IBM Rational Rose.
2. Familiarize yourself with the main program window.
3. Study the layout of the main interface elements.

2. Creating a new project

1. Select File → New.
2. Create a new UML project.
3. Save the project in the working directory.

3. Exploring the Interface Structure

1. Review the main program panels:
 - o Command menu;
 - o Standard toolbar;
 - o Model browser;
 - o Diagram plot area.
2. Familiarize yourself with the purpose of interface elements.
3. Learn how to switch between diagrams.

4. Working with the Model Browser

1. Open the Browser window.
2. Examine the project structure:
 - o Use Case View;
 - o Logical View;
 - o Component View;
 - o Deployment View.
3. Create a new package.

5. Creating a UML diagram

1. Select the Logical View section.
2. Create a new class diagram:

- o New → Class Diagram.

3. Specify a diagram name.

6. Adding objects to the diagram

1. Select the Class tool.
2. Place several classes on the workspace.
3. For each class, specify:
 - o Name;
 - o Attributes;
 - o Methods.

Example:

- Student;
- Teacher;
- Course.

7. Creating Relationships Between Classes

1. Select a relationship tool.
2. Connect the classes:
 - o Association;
 - o Inheritance;
 - o Dependency.
3. Configure relationship properties.

8. Working with Object Properties

1. Double-click to open class properties.
2. Edit object parameters.
3. Configure the display of attributes and operations.

9. Saving and Organizing the Project

1. Check the diagram for correctness.
2. Organize the elements.
3. Save the project using File → Save.

Individual Assignments

Option 1. Create a class diagram for the "Library" system.

Option 2. Develop a UML model of the "Online Store" system.

Option 3. Create a class diagram for the "Polyclinic" system.

Option 4. Develop a UML model of the "Bank" system.

Option 5. Create a class diagram for the "University" system.

Option 6. Develop a UML model of the "Hotel" system.

Option 7. Create a class diagram for the "Car Service" system.

Option 8. Develop a UML model of the "Movie Theater" system.

Option 9. Create a class diagram for the "Warehouse" system.

Option 10. Develop a UML model of the "Sports Club" system.

Conclusion. During this lab, you learned the basic elements of the IBM Rational Rose interface and the principles of working with UML models. You learned how to create projects, construct class diagrams, and configure object and relationship properties.

Questions

1. What is IBM Rational Rose for?
2. What is UML?
3. What are the main elements of the IBM Rational Rose interface?
4. What is the Project Browser used for?
5. What types of UML diagrams does Rational Rose support?
6. What is a class diagram?
7. What types of relationships exist between classes?

8. What are class attributes and methods used for?
9. What are the benefits of CASE development tools?
10. In what areas is IBM Rational Rose used?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #5

Building a Use Case Diagram (Diagrams of Use Cases).

Objective: To study the purpose and features of UML use case diagrams and to develop skills in constructing use case diagrams to model user interactions with an information system.

Work plan: Working with primary and alternative event flows in use case diagrams.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

A use case diagram is one of the main types of UML diagrams, designed to describe the functionality of a system and user interactions with the system.

The diagram allows you to:

- define system requirements;
- identify system participants;
- describe the main functions of the system;
- represent the interactions between users and the software.

The main elements of a use case diagram are:

- actor;
- use case;
- association;
- include;
- extend;
- generalization.

Process Flow

1. Launching the CASE Tool

1. Launch IBM Rational Rose or another UML modeling tool.
2. Create a new project.
3. Save the project in the working directory.

2. Creating a Use Case Diagram

1. In the project browser, select the Use Case View.
2. Run the command:
 - o New → Use Case Diagram.
3. Specify a name for the diagram.

3. Define system actors

1. Select the Actor tool.
2. Place actors on the diagram.

3. Specify a name for each actor.

Example:

- User;
- Administrator;
- Client;
- Operator.

4. Create use cases

1. Select the Use Case tool.
2. Add use cases.
3. Specify the names of system functions.

Example:

- User registration;
- Authorization;
- Report generation;
- Order placement.

5. Build relationships

1. Connect actors and use cases with associations.
2. Configure relationship types:
 - o Include;
 - o Extend;
 - o Generalization.
3. Verify that the relationships are correct.

6. Configure element properties

1. Open use case properties.
2. Add a brief description of the scenarios.
3. Specify the conditions for executing the functions.

7. Diagram Design

1. Arrange the diagram elements.
2. Resize objects if necessary.
3. Check the diagram's readability.

8. Model Validation

1. Check the correctness of the element names.
2. Ensure there are no unnecessary relationships.
3. Analyze the interaction logic.

9. Saving the Project

1. Select File → Save.
2. Save the UML project.

Individual Assignments

- Option 1. Build a use case diagram for the Library system.
- Option 2. Develop a use case diagram for the Online Store system.
- Option 3. Build a use case diagram for the Polyclinic system.
- Option 4. Develop a use case diagram for the Bank system.
- Option 5. Build a use case diagram for the "University" system.
- Option 6. Develop a use case diagram for the "Hotel" system.
- Option 7. Build a use case diagram for the "Car Service" system.
- Option 8. Develop a use case diagram for the "Movie Theater" system.
- Option 9. Build a use case diagram for the "Warehouse" system.
- Option 10. Develop a use case diagram for the "Sports Club" system.

Conclusion. During this lab, the basic principles of constructing UML use case diagrams were explored. Methods for defining actors, use cases, and the relationships between them were mastered, and skills for modeling the functional requirements of an information system were acquired.

Questions

1. What is the purpose of a use case diagram?
2. What is UML?
3. What is an actor in a use case diagram?
4. What is a use case?
5. What types of relationships are used in use case diagrams?
6. What is the difference between Include and Extend?
7. What is a use case diagram used for?
8. What are the benefits of UML modeling?
9. What CASE tools are used to create UML diagrams?
10. At what stage of development is a use case diagram used?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #6

Formulation of technical specifications. Writing system requirements.

Objective: To study the structure of technical specifications for software system development and to develop skills in formulating functional and non-functional software requirements.

Work plan: Developing detailed system requirements. Writing technical specifications for application development.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

A technical specification (TS) is a document defining the goals, objectives, requirements, and key characteristics of an information system or software product being developed. The TS forms the basis for the design, development, testing, and implementation of software.

The main sections of the TS are:

- General information;
- System purpose;
- Functional requirements;
- Non-functional requirements;
- Interface requirements;
- Reliability and security requirements;
- Software and hardware requirements.

A well-written TS minimizes development errors and ensures a common understanding of the project by all participants.

Process Flow

1. Familiarization with the subject area

1. Study the description of the given information system.
2. Determine the purpose of the software product.
3. Identify the main functions of the system.
4. Determine user categories.

2. Define the goals and objectives of the system

1. Formulate the main goal of the development.
2. Define the tasks solved by the system.
3. Establish the scope of the software product.

Example:

- automated data accounting;
- information storage;
- report generation;
- processing user requests.

3. Forming functional requirements

1. Describe the system functions.
2. Define user actions.
3. Specify the input and output data.

Example functional requirements:

- user registration;
- information retrieval;
- data editing;
- report generation.

4. Forming non-functional requirements

1. Define requirements for:
 - o performance;
 - o security;
 - o reliability;
 - o interface;
 - o compatibility.
2. Specify system limitations.

Example:

- response time no more than 3 seconds;
- access rights differentiation;
- multi-user support.

5. Describing the system structure

1. Identify the main system modules.
2. Describe the interactions between components.
3. Identify the main data processing processes.

6. Preparing the Technical Specifications

1. Create the technical specifications document.
2. Format the sections according to the established structure.
3. Check the logicity and completeness of the requirements description.

7. Requirements Analysis

1. Check the requirements for:
 - o Completeness;
 - o Consistency;
 - o Correctness;
 - o Feasibility.
2. Correct any identified deficiencies.

8. Saving the Document

1. Save the technical specifications file.
2. Prepare the document for presentation to the instructor.

Individual Assignments

Option 1. Develop technical specifications for the "Library" system.

Option 2. Generate requirements for the "Online Store" system.

Option 3. Develop technical specifications for the "Polyclinic" system.

- Option 4. Generate requirements for the "Bank" system.
- Option 5. Develop technical specifications for the "University" system.
- Option 6. Generate requirements for the "Hotel" system.
- Option 7. Develop technical specifications for the "Car Service" system.
- Option 8. Generate requirements for the "Movie Theater" system.
- Option 9. Develop technical specifications for the "Goods Warehouse" system.
- Option 10. Generate requirements for the "Sports Club" system.

Conclusion. During this lab, we learned the principles of creating technical specifications and methods for describing software system requirements. We mastered the skills of defining functional and non-functional requirements, analyzing the subject area, and structuring project documentation.

Questions

1. What is a technical specification?
2. Why is a technical specification necessary?
3. What sections does a technical specification include?
4. What are functional requirements?
5. What are non-functional requirements?
6. What are the quality requirements for a technical specification?
7. Why is requirements analysis performed?
8. What is a subject area?
9. What errors can occur when formulating requirements?
10. At what stage of development are technical specifications created?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #7

Activity diagram - Construction of activity diagrams.

Objective: To study the purpose and features of UML activity diagrams and to develop skills in modeling business processes and system operation algorithms using Activity Diagrams.

Work Plan: Activity Diagram - Building Activity Diagrams. Building a Statechart.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

An Activity Diagram is a type of UML diagram used to describe the sequence of actions, business processes, and system functioning algorithms.

Activity diagrams allow you to:

- model the sequence of operations;
- describe system operation algorithms;
- display transition conditions and parallel processes;
- analyze task execution logic.

The main elements of an Activity Diagram are:

- initial state;
- activity;
- transition;
- decision;
- parallel execution (fork/join);
- final state.

Process Flow

1. Launching a CASE modeling tool

1. Launch IBM Rational Rose or another UML tool.
2. Create a new UML project.
3. Save the project in the working directory.

2. Creating an Activity Diagram

1. In the Project Browser, select the required package.
2. Run the command:
 - o New → Activity Diagram.
3. Specify a name for the diagram.

3. Adding an Initial State

1. Select the Initial State element.
2. Place it on the workspace.
3. Define the start of the process.

4. Creating Activities

1. Select the Activity tool.
2. Add activities corresponding to the process stages.
 - Example:
 - data entry;
 - information processing;
 - result verification;
 - data saving.
3. Specify activity names.

5. Building Transitions

1. Select the Transition tool.
2. Connect activities in a logical sequence.
3. Check the correctness of the transitions.

6. Adding Conditions and Branching

1. Select the Decision element.
2. Add process conditions.
3. Specify transition options under various conditions.
 - Example:
 - "data is correct";
 - "input error".

7. Creating Parallel Processes

1. Use the Fork and Join elements.
2. Model parallel execution of operations.
3. Check process synchronization.

8. Adding a Final State

1. Select the Final State element.
2. Place it on the diagram.
3. Complete the algorithm execution.

9. Checking and configuring the diagram

1. Check the logical sequence of actions.
2. Arrange the diagram elements.

3. Customize the display of objects.
4. Save the project.

Individual Assignments

- Option 1. Build an Activity Diagram for the "Library Book Checkout" process.
- Option 2. Develop an Activity Diagram for the "Online Store" system.
- Option 3. Build an Activity Diagram for the "Patient Appointment at the Clinic" process.
- Option 4. Develop an activity diagram for the "Banking Services" system.
- Option 5. Build an activity diagram for the "Student Registration" process.
- Option 6. Develop an activity diagram for the "Hotel Reservation" system.
- Option 7. Build an activity diagram for the "Car Repair" process.
- Option 8. Develop an activity diagram for the "Movie Ticket Sales" system.
- Option 9. Build an activity diagram for the "Warehouse Inventory" process.
- Option 10. Develop an activity diagram for the "Sports Club Operations" system.

Conclusion. During this lab, the principles of constructing UML activity diagrams were explored. Methods for modeling algorithms and business processes were mastered, and skills for constructing sequences of actions, conditions, and parallel processes were acquired.

Questions

1. What is the purpose of an activity diagram?
2. What is UML?
3. What elements are used in an Activity Diagram?
4. What is an Activity?
5. What are Decision elements used for?
6. What are Fork and Join?
7. How is an Activity Diagram different from a flowchart?
8. What processes can be modeled using an Activity Diagram?
9. What CASE tools are used to create UML diagrams?
10. At what stage of development are activity diagrams used?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #8-9

Modeling interactions between objects. Interaction diagrams: Sequence diagrams. Collaboration diagrams.

Objective: To study the principles of modeling object interactions in UML and to develop skills in constructing sequence diagrams and collaboration diagrams.

Work plan: Analysis of a specific use case. Sequence diagrams - constructing sequence diagrams. Collaboration diagrams - defining a variety of systems, from computer programs to business processes, breaking a software system into structural components and relationships (dependencies) between components.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

UML interaction diagrams are designed to describe the exchange of messages between system objects during the execution of a specific scenario.

Interaction diagrams include:

- Sequence Diagrams;
- Collaboration Diagrams.

Sequence Diagram

Shows the interaction of objects over time and shows the order in which messages are passed between them.

Key elements:

- Object;
- Lifeline;
- Message;
- Activation.

Collaboration Diagram

Shows the interaction of objects, emphasizing the structural relationships between them.

Key elements:

- Objects;
- Relationships;
- Messages;
- Interaction numbering.

Interaction diagrams are used in designing the behavior of software systems and analyzing user scenarios.

Process Flow

1. Launching a CASE modeling tool

1. Launch IBM Rational Rose or another UML tool.
2. Create a new UML project.
3. Save the project in the working directory.

2. Creating a sequence diagram

1. In the project browser, select:
 - o New → Sequence Diagram.
2. Specify a diagram name.
3. Open the diagram workspace.

3. Adding objects

1. Select the Object tool.
2. Place objects on the diagram.
3. Specify object names.

Example:

- User;
- System;
- Database.

4. Creating lifelines

1. A lifeline is automatically created for each object.
2. Check that the objects are positioned correctly.
3. Configure the display of elements.

5. Adding messages

1. Select the Message tool.
2. Connect the objects with messages.
3. Specify operation names.

Example:

- data entry;
- user validation;
- saving information;
- output results.

4. Configure message flow.

6. Creating a Collaboration Diagram

1. Run the command:
 - o New → Collaboration Diagram.
2. Add system objects.
3. Create links between objects.
4. Configure message exchange.

7. Message numbering

1. Specify the interaction order for the collaboration diagram.
2. Check the sequence of operations.
3. Ensure the interaction logic is correct.

8. Checking and formatting diagrams

1. Check the links are correct.
2. Arrange the object layout.
3. Configure message display.
4. Save the project.

9. Analyzing the interaction scenario

1. Analyze the message exchange logic.
2. Determine the sequence of operations.
3. Identify potential interaction errors.

Individual Assignments

- Option 1. Build interaction diagrams for the "Library" system.
- Option 2. Develop a Sequence and Collaboration Diagram for the "Online Store" system.
- Option 3. Build interaction diagrams for the "Polyclinic" system.
- Option 4. Develop interaction diagrams for the "Bank" system.
- Option 5. Build interaction diagrams for the "University" system.
- Option 6. Develop interaction diagrams for the "Hotel" system.
- Option 7. Build interaction diagrams for the "Car Service" system.
- Option 8. Develop interaction diagrams for the "Movie Theater" system.
- Option 9. Build interaction diagrams for the "Warehouse" system.
- Option 10. Develop interaction diagrams for the "Sports Club" system.

Conclusion. During this lab, we explored the principles of modeling object interactions using UML diagrams. You have mastered the methods for constructing sequence and collaboration diagrams, and acquired skills in analyzing the exchange of messages between system objects.

Questions

1. What is the purpose of UML interaction diagrams?
2. What is a sequence diagram?
3. What is a collaboration diagram?
4. What elements are used in a sequence diagram?
5. What does an object's lifeline represent?
6. What are messages between objects used for?
7. How does a sequence diagram differ from a collaboration diagram?
8. What are the advantages of interaction diagrams?
9. When are sequence diagrams used?
10. What CASE tools are used to construct UML diagrams?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #10

State diagrams.

Objective: To study the purpose and features of UML state diagrams and to develop skills in modeling the behavior of system objects based on events and state changes.

Work Plan: Analysis of system object states, object behavior during transitions from one state to another, and identification of guard conditions for correct transitions between states.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

A Statechart Diagram is a type of UML diagram used to describe the behavior of an object in various states and display transitions between them under the influence of events.

Statecharts are used to:

- model the life cycle of an object;
- describe changes in system states;
- analyze the system's response to events;
- design the logic of object behavior.

The main elements of a statechart diagram are:

- State;
- Initial State;
- Final State;
- Transition;
- Event;
- Transition condition.

Statecharts are widely used in the design of information systems, automated processes, and real-time software.

Process Flow

1. Launch the CASE modeling tool

1. Launch IBM Rational Rose or another UML tool.
2. Create a new UML project.
3. Save the project in the working directory.

2. Creating a Statechart Diagram

1. In the Project Browser, select:
 - o New → Statechart Diagram.
2. Specify a name for the diagram.
3. Open the modeling workspace.

3. Adding an Initial State

1. Select the Initial State element.
2. Place it on the diagram.

3. Define the initial state of the object.

4. Creating Object States

1. Select the State tool.
2. Add object states.
3. Specify state names.

Example:

- "Waiting";
- "Processing";
- "Executing";
- "Completed".

5. Building Transitions between States

1. Select the Transition tool.
2. Connect states with transitions.
3. Specify the events that trigger the transition.

Example:

- Receiving data;
- Operation confirmation;
- Execution error.

6. Adding Transition Conditions

1. Specify the conditions for executing transitions.
2. Configure alternative behavior scenarios.
3. Check the correctness of the transition logic.

7. Creating the Final State

1. Select the Final State element.
2. Place it on the diagram.
3. Complete the object's lifecycle.

8. Checking and formatting the diagram

1. Check the correctness of the transitions.
2. Arrange the diagram elements.
3. Configure the display of objects.
4. Save the project.

9. Analyzing Object Behavior

1. Analyze the sequence of states.
2. Check the system's response to events.
3. Identify possible logic errors.

Individual Assignments

Option 1. Build a statechart diagram for the "Library" system.

Option 2. Develop a statechart diagram for the "Online Store" system.

Option 3. Build a statechart diagram for the "Polyclinic" system. Option 4. Develop a statechart diagram for the "Bank" system.

Option 5. Build a statechart diagram for the "University" system.

Option 6. Develop a statechart diagram for the "Hotel" system.

Option 7. Build a statechart diagram for the "Car Service" system.

Option 8. Develop a statechart diagram for the "Movie Theater" system.

Option 9. Build a statechart diagram for the "Warehouse" system.

Option 10. Develop a statechart diagram for the "Sports Club" system.

Conclusion. During this lab, the principles of constructing UML statecharts were explored. Methods for modeling object behavior, describing states and transitions between them were mastered, and skills for analyzing the life cycle of system objects were acquired.

Questions

1. What is the purpose of a statechart diagram?
2. What is an object state?
3. What elements are used in a statechart diagram?
4. What is a state transition?
5. What are events used for in a statechart diagram?
6. What is the difference between an initial state and a final state?
7. What processes are conveniently modeled using statecharts?
8. What are the advantages of UML statecharts?
9. Which CASE tools can be used to create a statechart diagram?
10. How does a statechart diagram differ from an activity diagram?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #11-12

Classes. Class diagram

Objective: To study the basic principles of object-oriented modeling and develop skills in constructing UML class diagrams to describe the structure of a software system.

Work Plan: Construct a static structure diagram describing the system structure, demonstrating the system's classes, their attributes, methods, and dependencies between classes. Relationships in class diagrams. Construct association and generalization relationships in class diagrams.

Equipment: This lab is performed on a personal computer in the chosen development environment (Microsoft Visual Studio, MS Project, Adalo, Sparx Enterprise Architect).

Theoretical Information

A class diagram is one of the main types of UML diagrams, designed to model the static structure of a system. A class diagram displays classes, their attributes, methods, and the relationships between objects.

Class diagrams are used:

- in software system design;
- to describe database structure;
- in object-oriented analysis;
- to document software architecture.

The main elements of a class diagram are:

- class;
- attributes;
- operations;
- association;
- inheritance;
- aggregation;
- composition;
- dependency.

Procedure

1. Launch the CASE modeling tool

1. Launch IBM Rational Rose or another UML tool.
2. Create a new UML project.
3. Save the project in the working directory.

2. Create a class diagram

1. In the project browser, select:
 - o Logical View → New → Class Diagram.
2. Specify a name for the diagram.
3. Open the modeling workspace.

3. Create classes

1. Select the Class tool.
2. Place classes on the diagram.
3. Specify class names.

Example:

- Student;
- Teacher;
- Discipline;
- Group.

4. Add class attributes

1. Open class properties.
2. Go to the attributes tab.
3. Add class attributes.

Example:

- StudentID;
- LastName;
- DateOfBirth;
- Address.

4. Specify attribute data types.

5. Add class methods

1. Go to the Operations tab.
2. Add class methods.

Example:

- add();
- delete();
- edit();
- save().

3. Specify parameters and return types.

6. Building relationships between classes

1. Select a relationship tool.
2. Create relationships between classes:
 - o association;
 - o inheritance;
 - o aggregation;
 - o composition.
3. Configure the relationship cardinality.

Example:

- one instructor teaches several courses;
- one group contains many students.

7. Configuring class properties

1. Change class display options.

2. Configure the visibility of attributes and methods:
 - o public;
 - o private;
 - o protected.
3. Check the correctness of the model structure.

8. Checking and formatting the diagram

1. Organize the arrangement of objects.
2. Check the correctness of the relationships.
3. Configure the readability of the diagram.
4. Save the project.

9. Model Analysis

1. Analyze the system structure.
2. Check the logical relationships between classes.
3. Determine the completeness of object descriptions.

Individual Assignments

- Option 1. Build a class diagram for the "Library" system.
Option 2. Develop a class diagram for the "Online Store" system.
Option 3. Build a class diagram for the "Polyclinic" system.
Option 4. Develop a class diagram for the "Bank" system.
Option 5. Build a class diagram for the "University" system.
Option 6. Develop a class diagram for the "Hotel" system.
Option 7. Build a class diagram for the "Car Service" system.
Option 8. Develop a class diagram for the "Movie Theater" system.
Option 9. Build a class diagram for the "Warehouse" system.
Option 10. Develop a class diagram for the "Sports Club" system.

Conclusion. During this lab, we explored the principles of object-oriented modeling and methods for constructing UML class diagrams. We mastered the skills of creating classes, defining attributes and methods, and building relationships between system objects.

Questions

1. What is the purpose of a class diagram?
2. What is a class in UML?
3. What elements does a class contain?
4. What are class attributes?
5. What are class methods used for?
6. What types of relationships exist between classes?
7. What is inheritance?
8. How does aggregation differ from composition?
9. What are the advantages of object-oriented modeling?
10. Which CASE tools are used to create class diagrams?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #13

Developing apps using no-code platforms. AppMaster platform.

Objective: Explore the capabilities of the AppMaster no-code platform and develop skills in designing applications without writing code.

Work Plan: Design business processes on the AppMaster platform. Identify patterns and requirements for maintaining business process logic.

Equipment: This lab is performed on a personal computer in the chosen development environment (MS Project, Adalo, AppMaster platform).

Theoretical Information

No-code platforms are designed for developing software applications without traditional programming. These platforms enable the creation of web applications, mobile applications, and business systems using visual design tools.

The AppMaster platform provides the following capabilities:

- visual interface design;
- business logic creation;
- database modeling;
- automatic application generation;
- integration with external services.

The main components of the platform are:

- business process editor;
- interface designer;
- data management system;
- application publishing tools.

Process Flow

1. Registration and Login

1. Open the AppMaster platform.
2. Register as a user.
3. Log in to the system.
4. Familiarize yourself with the platform's interface.

2. Creating a New Project

1. Select the Create New Project command.
2. Specify:
 - o project name;
 - o application description;
 - o type of application being created.
3. Create a new project.

3. Designing the data structure

1. Go to the Database section.
2. Create database entities.
3. Add fields and data types.

Example:

 - User;
 - Product;
 - Order;
 - Category.
4. Configure relationships between entities.

4. Creating business logic

1. Open the business process editor.
2. Create a data processing process.

3. Configure actions:
 - o Add records;
 - o Edit data;
 - o Delete information;
 - o Search data.
4. Verify the correctness of the logic.

5. Designing the user interface

1. Go to the interface creation section.
2. Add elements:
 - o Buttons;
 - o Tables;
 - o Forms;
 - o Text fields.
3. Configure the interface's interaction with the database.

6. Configuring application navigation

1. Create application pages.
2. Configure window transitions.
3. Check navigation ease.

7. Testing the application

1. Launch the built-in preview mode.
2. Check:
 - o Interface functionality;
 - o Data processing accuracy;
 - o Business processes.
3. Fix any errors.

8. Publishing the project

1. Build the application.
2. Configure publishing parameters.
3. Generate the application.

9. Saving and Exporting the Project

1. Save the project on the platform.
2. Export data if necessary.
3. Prepare demonstration materials.

Individual Assignments

- Option 1. Develop a "Library" application.
- Option 2. Create an "Online Store" application.
- Option 3. Develop a "Clinic" application.
- Option 4. Create a "Bank" application.
- Option 5. Develop a "University" application.
- Option 6. Create a "Hotel" app.
- Option 7. Develop a "Car Service" app.
- Option 8. Create a "Movie Theater" app.
- Option 9. Develop a "Warehouse" app.
- Option 10. Create a "Sports Club" app.

Conclusion. During this lab, we explored the capabilities of the AppMaster no-code platform for designing software applications without writing code. We mastered the skills of creating data structures, configuring business logic, designing interfaces, and testing applications.

Questions

1. What is a no-code platform?
2. What is the purpose of the AppMaster platform?
3. What are the advantages of no-code technologies?

4. What components does AppMaster include?
5. What is application business logic?
6. What is the purpose of the visual interface editor?
7. What types of applications can be created in AppMaster?
8. How does no-code differ from traditional development?
9. What stages does app design involve?
10. What are the limitations of no-code platforms?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

LABORATORY WORK #14-15

Developing an application with database connectivity on the AppMaster platform

Objective: Explore the capabilities of the AppMaster platform for creating applications with database connectivity and develop skills in designing, configuring, and testing applications with data storage and processing.

Work Plan: Design business processes on the AppMaster platform. Develop an application on a selected topic.

Equipment: This lab is performed on a personal computer in the chosen development environment (MS Project, Adalo, AppMaster platform).

Theoretical Information

The AppMaster platform is a modern no-code application development tool that allows you to create web and mobile applications without writing code. The platform supports database structure creation, business process configuration, user interface development, and integration with external services.

Database connection provides:

- information storage;
- user data processing;
- search and filter operations;
- report generation;
- multi-user support.

The main elements of the application are:

- database tables;
- data entry forms;
- business processes;
- user interface;
- database interaction mechanisms.

Process Flow

1. Login to the AppMaster platform

1. Open the AppMaster platform.
2. Log in.
3. Explore the project workspace.

2. Create a new project

1. Select the Create New Project command.
2. Specify:
 - o application name;
 - o project description;
 - o Application type.
3. Create a project.

3. Database design

1. Go to the Database section.
2. Create database tables.
Example:
 - Users;
 - Products;
 - Orders;
 - Categories.
3. Add table fields:
 - o ID;
 - o Name;
 - o Date;
 - o Quantity;
 - o Cost.
4. Specify field data types.
5. Configure relationships between tables.

4. Configure data operations

1. Create operations:
 - o Add records;
 - o Edit data;
 - o Delete records;
 - o Search for information.
2. Configure database query processing.
3. Verify correct interaction with the database.

5. Creating business processes

1. Go to the business logic editor.
2. Create data processing processes.
3. Configure automatic actions:
 - o Save data;
 - o Validation;
 - o Displaying user messages.

6. Designing the User Interface

1. Create application pages.
2. Add interface elements:
 - o Input forms;
 - o Tables;
 - o Buttons;
 - o Search fields.
3. Link the interface elements to the database.

7. Configuring Navigation

1. Create the application menu.
2. Configure page transitions.
3. Test user experience.

8. Testing the Application

1. Launch Preview Mode.
2. Test:

- o Adding data;
- o Displaying records;
- o Editing information;
- o Deleting data.

3. Correct any errors.

9. Publishing the Project

1. Build the application.
2. Configure publication settings.
3. Generate the finished application.

Individual Assignments

- Option 1. Develop an application with the "Library" database.
- Option 2. Create an online store application with order storage.
- Option 3. Develop a clinic application with patient management.
- Option 4. Create a bank application with customer information storage.
- Option 5. Develop a university application with a student database.
- Option 6. Create a hotel application with a booking system.
- Option 7. Develop a car service application with repair services.
- Option 8. Create a movie theater application with ticket sales.
- Option 9. Develop a warehouse application with product management.
- Option 10. Create a sports club application with a customer database.

Conclusion. During this lab, we explored the capabilities of the AppMaster platform for developing applications with database connections. We mastered the skills of designing a database structure, configuring business processes, creating interfaces, and testing applications.

Questions

1. What is the purpose of the AppMaster platform?
2. What is no-code development?
3. What is the purpose of a database in an application?
4. What elements does a database structure include?
5. What is a business process?
6. What operations can be performed with data?
7. What are data entry forms used for?
8. What are the benefits of using no-code platforms?
9. What stages does application development involve?
10. What limitations do applications created on no-code platforms have?

Recommended literature

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. - 366с. - ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань: электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.

EVALUATION CRITERIA FOR THE DISCIPLINE

Letter grade	Numerical equivalent of points	% content	Evaluation according to the traditional system	Scoring criteria
A	4,0	95-100	Excellent	An A grade in the subject “Tools of program development” is awarded when the student: understands the software life cycle and can clearly distinguish between waterfall, iterative, spiral, and iterative models; confidently builds AS IS / TO BE models, performs functional and cost analysis; freely uses CASE tools (BPWin, ERWin, Rational Rose, Ms Office Visio, Aris) for modeling structural and object-oriented systems; creates correct UML diagrams (use case, class, sequence, activity, etc.), taking into account the requirements and interaction of components; demonstrates excellent command of IDEF0 and SADT: builds complete functional models; competently prepares documentation in accordance with GOST/IEEE requirements.
A-	3,67	90-94		An A- grade is given when the student: understands the software life cycle and can clearly distinguish between waterfall, iterative, spiral, and iterative models; confidently builds AS IS / TO BE models, performs functional and cost analysis; freely uses CASE tools (BPWin, ERWin, Rational Rose, Ms Office Visio, Aris) for modeling structural and object-oriented systems, but allows minor errors; models (UML, IDEF0, ER models) are logically correct, but may not take into account rare scenarios; demonstrates excellent command of IDEF0 and SADT: builds complete functional models.
B+	3,33	85-89	Good	A B+ grade is given when the student knows the key stages of design, applies SADT and UML methodologies at a good level; minor details or interrelationships may be omitted in UML and IDEF0 models; able to work in CASE environments, but with prompts; documentation is complete, but not always structured.
B	3,0	80-84		A grade of B is given when the student: understands the principles of the life cycle and modeling, but sometimes confuses stages or designations; UML diagrams are drawn correctly, but superficially; has a basic command of CASE tools, prefers templates.
B-	2,67	75-79		A B- grade is given when the student: partially understands the difference between structural and object-oriented approaches; has difficulty constructing coherent models; diagrams are incomplete; uses only basic CASE tool functions; project tasks are completed formally, models require further refinement.
C+	2,33	70-74		A C+ grade is given when a student: has limited knowledge of basic concepts (life cycle, UML diagrams, SADT); uses CASE tools uncertainly, often fails to follow notation; the project contains errors in the logic or structure of models; documentation is incomplete or formal.
C	2,0	65-69	Fair	A C grade is given when the student: demonstrates a general understanding of the discipline, but cannot apply it in practice; creates diagrams with errors, does not distinguish between types of UML diagrams; uses CASE tools as a drawing tool, without understanding the logic of models.
C-	1,67	60-64		A C- grade is given when the student: knows the terms but does not know how to apply them: IDEF0, SADT, UML,

				CASE; Errors even in basic diagrams (e.g., in use case or class diagrams); Does not know how to use CASE tools; Lab work completed with outside help; Documentation poorly formatted, no structure.
D+	1,33	55-59		A D+ grade is given when the student: demonstrates weak knowledge: can recall topics but cannot apply them in practice; diagrams and models do not correspond to methodology, logic is lacking; CASE tools are rarely used.
D	1,0	50-54		A D grade is given when the student: understands only individual concepts, but does not master the basics of modeling; does not use CASE tools at all or works with errors; Laboratory assignments are not completed or defended; the project does not meet the requirements.
FX	0,5	25-49		The grade “poor” corresponds to the letters FX and F , which have a numerical equivalent of 0 and a percentage content of 0-49. This grade is given if the student has gaps in their knowledge of the basic material covered by the program, has not mastered more than half of the module (discipline) program, has made fundamental errors in their answers, has not completed individual assignments provided for in the forms of current, intermediate, and final control, or has not studied all the basic literature provided for in the program.
F	0	0-24	Poor	

LIST OF RECOMMENDED REFERENCES

Main literature:

1. Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. Приемы объектно-ориентированного проектирования. – С.Пб.: Питер, 2020. – 366с. – ISBN 978-5-4461-1213-5.
2. Вейцман, В. М. Проектирование информационных систем: учебное пособие для вузов. – 2-е изд. – С.Пб.: Лань, 2022. – 316 с. – ISBN 978-5-8114-9982-3. – Текст: электронный // Лань : электронно-библиотечная система. – URL: <https://e.lanbook.com/book/208946> (дата обращения: 30.05.2025)
3. Белов В.В. Проектирование информационных систем: учебник для студентов, обучающихся по направлению «Прикладная информатика» и другим экономическим специальностям. - 2-е изд., стер. – М.: Академия, 2015. – 351с. - ISBN 978-5-4468-2440-3.
4. Брусникин Г.Н., Соколова Н.Ю. Разработка UML-моделей при проектировании информационных систем: учебное пособие. – М.: МИЭТ, 2023. – 52 с. – ISBN 978-5-7256-1016-1.

Additional literature:

1. Грекул В.И., Денищенко Г.Н., Коровникова Н.Л. Проектирование информационных систем. – М.: Национальный открытый Университет «ИНТУИТ», 2016. – 571с.
2. Шевчук Е.В. Интеллектуальные информационные системы: учебно-методическое пособие. - Алматы: CyberSmith, 2017. – 137с. - ISBN 978-601-310-319-8.
3. Цуканова О.А. Методология и инструментарий моделирования бизнес-процессов: учебное пособие. - Министерство образования и науки Российской Федерации, Университет ИТМО. - Санкт-Петербург: Университет ИТМО, 2015. - 101 с.

Electronic and Internet resources:

1. Унифицированный язык моделирования UML <https://www.lucidchart.com/pages/ru/uml>
2. Полный курс UML <https://blog.skillfactory.ru/glossary/uml/>
3. <https://www.youtube.com/user/ariscommunity>
4. <https://www.ariscommunity.com>

IWS				
Week no.	Module no.	Name of topics of IWS. Deadlines for submitting IWS	Number of hours	Types and methods of training
1	1	Preliminary survey and analysis of the subject area. Justification and selection of automated tasks (task setting). Preparation of an individual task with a verbal description of the algorithm.	8	summary
2	1	Preliminary survey and analysis of the subject area. Justification and selection of automated tasks (task setting). Preparation of an individual task with a verbal description of the algorithm.	8	presentation
3	1	Creating a business process model.	7	report
4	1	Creating a business analysis model.	7	abstract
5	1	Creating a business analysis model.	7	summary
6	1	Functional modeling and construction of a model of the selected subject area using Platinum BPWin. Individual.	7	presentation
7	1	Functional modeling and construction of a model of the selected subject area using Platinum BPWin. Individual.	7	report
8	2	Functional modeling and database structure design for a selected subject area using Platinum ERWin. Individual.	7	abstract
9	2	Functional modeling and database structure design for a selected subject area using Platinum ERWin. Individual.	7	summary
10	2	Functional modeling and database structure design for a selected subject area using Platinum ERWin. Individual.	7	presentation
11	2	Designing a model of the selected subject area in the Rational Rose.	7	report
12	2	Designing a model of the selected subject area in the Rational Rose.	7	summary
13	2	Designing a model of the selected subject area in the Aris.	7	presentation
14	2	Designing a model of the selected subject area in MS Office Visio.	7	report
15	2	Designing a model of the selected subject area in MS Office Visio.	7	abstract
TOTAL			105	

Materials For current, borderline control, intermediate certifications

Questions on first boundary control (colloquium)

1. Classify modern tools and list their main features
2. Vital cycle (LC) any systems This:
3. Administrator - This specialist or group specialists who:
4. Systemic programmers - This specialist or group specialists who:
5. Applied programmers - This specialist or group specialists, which:
6. Conceptual performance data – This:
7. Physical performance data - This:
8. External performance data - This:
9. Classification IS Not is produced By alone from following signs:
10. By regime works IS are divided into:
11. By method distributions computing resources IS are sharing to:
12. By character processing information IS are subdivided to:
13. Describe CASE tools developments programs.
14. List main possibilities CASE tools developments programs.
15. Describe software shell FAR manager.
16. Describe appointment menu And configuration FAR manager.
17. Name it principles constructions And connections dynamic modules V FAR- manager.
18. List archiving programs. Describe concept process archiving files.
19. Describe characteristic archiving, bring examples archiving files
20. Describe technology use archiver WinRAR. Bring examples.
21. Describe technology use archiver WinZip. Bring examples.
22. Describe possibilities And appointment MS programs Visio.
23. Describe graphic forms performances programs And informational connections V MS Visio.
24. Expand content object-oriented language modeling.
25. Explain appointment language UML, bring examples
26. Describe concept And principles object-oriented modeling.
27. Describe principles of visual modeling: assignment of models, model levels .
28. Describe process developments software .
29. Describe functions models process.
30. Describe usage UML diagrams: types diagrams, bring examples.
31. Bring examples options use And active persons in UML.

Questions on second boundary control (colloquium)

1. Unlock the power of tools for rapid application development.
2. Describe content methodologies RAD.
3. Describe the concepts: visual programming, event-driven programming.
4. Describe possibilities funds visual programming.
5. Describe possibilities integrated visual Wednesday developments programs. Relational data model .
6. Modern programming - basic concepts And tools.
7. Modern technologies developments BY.
8. Modern languages programming BY.
9. Specific requirements technical tasks.
10. Comparison opportunities packages programming bases data.
11. Comparison opportunities systems management bases data.
12. Comparison diagrams activities And algorithm works programs.
13. Comparison class diagrams And structures databases .
14. Comparison diagrams objects And diagrams component.
15. Comparisons opportunities object languages programming.
16. Testing And debugging BY.
17. Technical and economic justification BY.
18. Technical requirements To development BY.

19. Requirements, presented To development BY.
20. File - managers - control programs files at development - possibilities and their growth, diversity and characteristics of use.
21. Integrity And protection data. Structures BD.
22. Economic requirements developments BY.
23. Stage development of requirements To program - methods and tools.
24. Prospects instrumental funds
25. Comparison opportunities systems management bases data.

Questions on final control (oral exam)

1. Target And tasks instrumental means. Classification software provision.
2. Classification And main peculiarities modern instrumental means.
3. Basics modeling business processes. Vital cycle programs.
4. Models vital cycle.
5. General requirements To methodologies And technologies. Stages design systems.
6. Requirements To content documents.
7. CASE technology design systems. CASE systems structural And object-oriented types.
8. Practical recommendations By development And implementation CASE–means, criteria their selection and comparative analysis.
9. Structural approach To design BY. Methodology SADT functional modeling.
10. Functional model organizations works - AS IS.
11. Model TO BE m her functional cost analysis.
12. Methodology functional modeling IDEFO.
13. CASE–tool structural type BPWin.
14. Display logical And physical level models V ERWin.
15. CASE–tool structural type ERWin.
16. Model Mart - system management models For group development.
17. Unified language modeling UML.
18. Main elements object-oriented approach.
19. UML diagrams. Description requirements V context models precedents.
20. Classes. C l a s s diagram .
21. Modeling interactions between objects.
22. Application UML diagrams each type For modeling behavior of systems.
23. Design systems on language UML in the environment Rational Rose, Aris, M.S. Office Visio.
24. Specific requirements technical tasks.
25. Comparison opportunities packages programming bases data.
26. Comparison opportunities systems management bases data.
27. Comparison diagrams activities And algorithm works programs.
28. Comparison class diagrams And structures databases .
29. Comparison diagrams objects And diagrams component.
30. Comparisons opportunities object languages programming.
31. Testing And debugging BY.
32. Technical and economic justification BY.
33. Technical requirements To development BY.
34. Requirements, presented To development BY.
35. File - managers - control programs files at development - possibilities and their growth, diversity and characteristics of use.
36. Integrity And protection data. Structures BD.
37. Economic requirements developments BY.
38. Stage development of requirements To program - methods and tools.
39. Prospects instrumental funds
40. Comparison opportunities systems management bases data.
41. Unlock the power of tools for rapid application development.
42. Describe content methodologies RAD.
43. Describe the concepts: visual programming, event-driven programming.

44. Describe possibilities funds visual programming.
45. Describe possibilities integrated visual Wednesday developments programs.
- . Relational data model .
46. Modern programming - basic concepts And tools.
47. Modern technologies developments BY.
48. Modern languages programming BY.
49. Classify modern tools and list their main features
50. Vital cycle (LC) any systems This:
51. Administrator - This specialist or group specialists, which:
52. Systemic programmers - This specialist or group specialists who:
53. Applied programmers - This specialist or group specialists who:
54. Conceptual performance data – This:
55. Physical performance data - This:
56. External performance data - This:
57. Classification IS Not is produced By alone from following signs:
58. By regime works IS are divided into:
59. By method distributions computing resources IS are sharing to:
60. By character processing information IS are subdivided to:
61. Describe CASE tools developments programs.
62. List main possibilities CASE tools developments programs.
63. Describe software shell FAR manager.
64. Describe appointment menu And configuration FAR manager.
65. Name it principles constructions And connections dynamic modules V FAR- manager.
66. List archiving programs. Describe concept process archiving files.
67. Describe characteristic archiving, bring examples archiving files
68. Describe technology use archiver WinRAR. Bring examples.
69. Describe technology use archiver WinZip. Bring examples.
70. Describe possibilities And appointment MS programs Visio.
71. Describe graphic forms performances programs And informational connections V MS Visio.
72. Expand content object-oriented language modeling.
73. Explain appointment language UML, bring examples
74. Describe concept And principles object-oriented modeling.
75. Describe principles of visual modeling: assignment of models, model levels .
76. Describe process developments software .
77. Describe functions models process.
78. Describe usage UML diagrams: types diagrams, bring examples.
79. Bring examples options use And active persons in UML.
80. Story development Case- funds.
81. Main principles And provisions constructions Case- funds.
82. Main components And functional possibilities Case- funds.
83. Properties Case-funds - warp support process developments BY.
84. Classification Case-funds by types.
85. Classification Case-funds by category.
86. Classification Case-funds by levels.
87. Tools developments software tools.
88. Classification features of development and maintenance tool environments for software tools.
89. Main classes of tool environments developments and software support .
90. Instrumental Wednesday programming.
91. Changes V vital cycle software funds at use computer technologies.
92. Standards Life Cycle BY
93. Main features instrumental systems technologies programming.
94. Compilers. Their types And appointment.
95. Phases compilations.
96. Interpreters. Their types and purpose.
97. Interpreters compiling type.

98. Cross-platform compilation.
99. Common Wednesday programming.
100. Cross-platform system Qt.
101. Methods functional modeling software systems.
102. Development informational models software systems.
103. Object-oriented approach To modeling software systems.
104. Tools that implement a structured approach to system modeling.
105. Tools that implement an object-oriented approach to system modeling.
106. Language UML. Advantages And flaws.
107. UML diagram options use.
108. UML diagram classes.
109. UML diagram sequences.
110. Tools to support the software testing process .
111. Advantages And flaws automation process testing.
112. Functions funds management testing.
113. Process accompaniment software provision.
114. Systems management versions.
115. Tasks solved by means of automated generation of technical documentation.
116. Resources automatic generations technical documentation.
117. Case – tools. Main capabilities of CASE-tools. Modern CASE-technologies. Technology of development and implementation of CASE-tools. Evaluation and characteristics of modern CASE-tools. Classification of CASE-tools.
118. IDEF0
119. IDEF3
120. IDEF1, IDEF1x
121. Entity Relationship Diagram, ERD
122. DB, DBMS
123. UML. Diagram options use
124. UML. Diagram sequences (sequence diagram)
125. UML. Diagram interactions (interaction diagram)
126. UML. Diagram coppers (collaboration diagram)
127. UML. Diagram classes
128. UML. Diagram activities
129. Methods testing programs
130. DFD
131. BPMN
132. Tools to support the software testing process.
133. Instrumental means, implementing structural approach To modeling systems.
134. Tools that implement an object-oriented approach to system modeling.
135. Compilers. Their types And appointment.
136. Phases compilations.
137. Interpreters. Their types and purpose.
138. Interpreters compiling type.
139. Cross-platform compilation.
140. Common Wednesday programming.
141. Cross-platform system Qt.
142. Technology — ASP.NET
143. Technology NetCF — .NET Compact Framework 3.5
144. Technology — Silverlight
145. VSTO Technology — Visual Studio Tools for Office
146. Technology — WinForms
147. Technology WPF Windows Presentation Foundation
148. Technology XNA
149. Visual Studio Team System Database Edition
150. Visual Studio Team System Development Edition

151. Visual Studio Team System Architect Edition
152. Visual Studio Team System Test Edition
153. Visual Studio Team Suite
154. ASP.NET Web Service
155. XML (Extensible Markup Language, expandable markup language)
156. Model threats And principles provision security software provision
157. Formal methods proof correctness programs And their specifications.
158. Methods And means analysis security software .
159. Methods of ensuring the reliability of programs for monitoring their technological safety.
160. Standards And other normative documents, regulatory security of software and processed information.